

Chapitre 1 — Dossier de Revue

Histoire de la thèse · Figures · Code source

Nathan Corroyez

2026-05-05

Contents

1	Histoire de la thèse	2
1.1	Histoire de la thèse	2
1.2	Introduction Générale	2
1.3	Chapitre 2 : Évaluation et réduction des divergences radiatives d'estimation du LAI inter-capteurs (ALS et Sentinel-2)	4
1.4	Chapitre 3 : Cartographie dynamique et suivi spatio-temporel de l'atténuation microclimatique	5
2	Figures — Résultats principaux	7
3	Figures — Annexes	14
4	Code source R	17
4.1	main.R — Orchestrateur principal (pipeline 7 étapes)	17
4.2	R/config.R — Configuration, chemins et flags	37
4.3	R/scenarios.R — Définition des scénarios H1/H2/REF	38
4.4	R/lad.R — Constructeurs de profils LAD	41
4.5	R/musica.R — Exécution MuSICA et post-traitement NetCDF	43
4.6	R/forest.R — Extraction des métriques forestières LiDAR	47
4.7	R/fpca.R — Analyse en Composantes Principales Fonctionnelles	50
4.8	R/h1_analysis.R — Analyse H1 : attribution structurelle	53
4.9	R/h2_analysis.R — Analyse H2 : forme verticale LAD	65
4.10	R/gamm.R — Émulateur GAMM	71
4.11	R/validation.R — Validation aux capteurs HOBO	74
4.12	R/io.R — Entrées/sorties (forcing ERA5, HOBO CSV)	78
4.13	R/profiles.R — Profils verticaux MuSICA	80
4.14	R/sentinel.R — Annexe Sentinel-2/FORMS-H	93

1 Histoire de la thèse

1.1 Histoire de la thèse

Ce document présente l'articulation scientifique du manuscrit de thèse. Il est construit selon une progression logique visant à lever le verrou méthodologique lié à la caractérisation spatio-temporelle de la canopée pour la modélisation biophysique du microclimat forestier. Le raisonnement s'articule autour de l'évaluation du poids de la structure tridimensionnelle sur les simulations microclimatiques (Chapitre 1), de la résolution des divergences radiatives inter-capteurs nécessaires à l'acquisition d'une dimension temporelle (Chapitre 2), et de l'application de ce couplage pour la prédiction spatialisée et continue (Chapitre 3).

1.2 Introduction Générale

Face à l'intensification des anomalies thermiques, les écosystèmes forestiers assurent une fonction de régulation climatique déterminante, générant des micro-refuges thermiques essentiels au maintien de la biodiversité sous-jacente et à la résilience globale des peuplements. Toutefois, spatialiser la modélisation mécaniste de ces transferts d'énergie sur de vastes massifs forestiers nécessite la description de variables structurelles complexes. Le modèle biophysique MuSICA, qui simule ces flux à l'échelle de la parcelle, requiert notamment une caractérisation fine de la structure verticale de la végétation.

Dans ce contexte, les approches par télédétection offrent un potentiel majeur pour acquérir ces paramètres structurels de manière exhaustive et continue sur de grandes superficies. Elles se heurtent néanmoins à des limites inhérentes aux différents types de capteurs :

- Le LiDAR aéroporté (ALS) fournit une description structurelle à haute résolution (hauteur, distribution verticale de la densité foliaire), mais sa résolution temporelle reste limitée.
- L'imagerie optique satellitaire (Sentinel-2) dispose d'une haute fréquence de revisite, mais la sensibilité de son signal de réflectance aux variations de la surface foliaire (saturation optique du LAI) diminue fortement pour les couverts denses, limitant sa capacité à caractériser l'hétérogénéité architecturale intra-canopée.
- L'altimétrie laser spatiale (ex : mission GEDI) apporte une dimension structurelle répétée dans le temps, mais son échantillonnage par empreintes discontinues et ses incertitudes de géolocalisation restreignent la production de cartographies continues à très haute résolution à l'échelle du paysage.

L'objectif de cette thèse est de fusionner l'exactitude structurelle de l'ALS et la continuité temporelle de Sentinel-2 afin de fournir aux modèles microclimatiques les paramètres de peuplement nécessaires à la simulation spatialisée et dynamique des températures sous couvert.

1.2.0.1 Chapitre 1 : Sensibilité du microclimat forestier à l'architecture 3D : L'illusion optique face à la réalité structurelle *(Préliminaire : Dans le cadre de cette étude, le macroclimat désigne les conditions météorologiques régionales de référence prévalant en milieu ouvert, tandis que le microclimat correspond aux conditions thermiques locales spécifiquement modifiées sous le couvert forestier. L'analyse se concentre sur le rafraîchissement maximal diurne, défini comme l'écart thermique $\Delta T_{\max}$, afin d'isoler la performance intrinsèque de la forêt).*

Questions de recherche : Quelle est la sensibilité des températures sous couvert simulées par le modèle mécaniste MuSICA aux variations de la distribution spatiale (verticale et horizontale) de la végétation ? Les approches satellitaires optiques (Sentinel-2) sont-elles suffisantes pour cartographier ces refuges thermiques continus, ou la caractérisation de la géométrie 3D interne (LiDAR) est-elle techniquement indispensable ?

Hypothèses :

- **H1** : Le profil vertical de densité foliaire (LAD) est un forçage physique de premier ordre. Il module significativement les transferts radiatifs et turbulents, ce qui le rend supérieur aux seules métriques intégrées (LAI total, hauteur maximale) pour simuler fidèlement le pouvoir tampon spatial.
- **H2** : À valeurs de LAI et de hauteur constantes, la forme pure du profil vertical redistribue l'énergie. Les profils présentant une concentration de biomasse dans les strates supérieures (canopées "Top-Heavy") interceptent le rayonnement incident plus loin du sol, déplaçant ainsi les sources de chaleur vers le sommet du couvert tout en favorisant la ventilation du sous-bois.
- **H3** : Remplacer la structure 3D réelle par des proxys issus de l'imagerie satellitaire optique lisse la variance spatiale intra-forestière, créant une "illusion thermique" qui sature et échoue à prédire l'intensité réelle des extrêmes microclimatiques locaux.

Données mobilisées : Forçages météorologiques physiques (ERA5 : Rayonnement, Vent, Température, Déficit de Pression de Vapeur), métriques structurelles et profils LAD extraits de données LiDAR aéroporté (ALS) agrégées à 20 mètres de résolution spatiale, séries temporelles de réflectance Sentinel-2, et températures sous couvert issues d'un réseau in-situ (53 capteurs HOBO).

Méthodes :

- **Plan d'Expérience Spatial et Mécaniste** : Échantillonnage par Hypercube Latin Conditionné (cLHS) sur l'espace structurel LiDAR afin d'extraire un jeu de 300 parcelles statistiquement équilibrées. Application d'une approche par scénarios itératifs (*Forward inclusion*) dans MuSICA pour isoler le poids mécaniste de chaque variable (LAI, Hmax, fCover), couplée à un contraste apparié isolant la distribution verticale (LAD uniforme vs réel).
- **Caractérisation Fonctionnelle de l'Architecture** : Normalisation des profils LAD et application d'une Analyse en Composantes Principales Fonctionnelles (FPCA) *post-échantillonnage* pour extraire la géométrie pure sans biais de représentativité (FPC1: Asymétrie Top/Bottom-Heavy).
- **Émulation Statistique Hybride** : Entraînement d'un Modèle Additif Généralisé Mixte (GAMM) couplant l'architecture 3D et les variables macroclimatiques physiques (Vent, Rayonnement, VPD) pour isoler rigoureusement l'effet de la canopée du bruit météorologique temporel.

Résultats & Discussion :

- **La hiérarchie structurelle et l'empreinte de la géométrie (H1 & H2)** : L'approche mécaniste prouve que le LAI dicte l'amplitude globale de l'interception, mais que l'affinage 3D (Hmax, fCover, profil LAD) est indispensable pour maximiser la fidélité spatiale. À biomasse égale, la forme verticale pure redistribue spatialement le refroidissement au sol de $\sim 1^\circ\text{C}$, agissant comme un régulateur local de l'énergie.
- **La respiration climatique** : L'émulateur GAMM confirme que la structure 3D (LAI, Hmax, FPC1) conserve un rôle physique déterminant ($p < 2 \times 10^{-16}$) indépendamment de la météorologie. Il met en lumière les mécanismes physiques : la destruction du microclimat par le vent, et l'amplification du rafraîchissement par transpiration lorsque le VPD (atmosphère sèche) augmente.
- **Le plafond de verre structurel** : Confrontée aux capteurs in-situ, la modélisation 3D classe excellentement les parcelles (R^2 grimpe jusqu'à 0.48) mais révèle un biais chaud systématique. La physique de la canopée supérieure capte la dynamique spatiale (elle sait *qui* est plus frais que *qui*), mais sous-estime l'amplitude absolue des refuges réels (biais de $+2.08^\circ\text{C}$). Cela souligne le rôle critique de processus de surface non modélisés ici (évapotranspiration de la sous-strate, inertie du sol, topographie).

- **L'Illusion Optique (H3) :** Face aux prédictions LiDAR qui déverrouillent la variance spatiale, le scénario basé sur la télédétection optique (Sentinel-2) sature rapidement. Le signal optique "écrase" la variance, moyennise le microclimat forestier, et nous rend complètement aveugles à la réalité des extrêmes thermiques locaux.

Conclusion du Chapitre 1 :

Ce chapitre établit qu'une paramétrisation 3D fine (issue d'un capteur actif) est indispensable pour orchestrer la variance spatiale du microclimat. L'imagerie optique seule (Sentinel-2) crée une "illusion" d'homogénéité sous couvert. Toutefois, la donnée LiDAR reste statique, alors que le suivi continu du microclimat nécessite l'intégration de la dynamique saisonnière du feuillage. Étant donné l'échec de l'intégration naïve du signal optique, il devient impératif d'étalonner les relations profondes entre la réflectance satellitaire et la géométrie 3D de la canopée. Le Chapitre 2 vise par conséquent à résoudre ces divergences d'estimation inter-capteurs avant toute assimilation temporelle.

1.3 Chapitre 2 : Évaluation et réduction des divergences radiatives d'estimation du LAI inter-capteurs (ALS et Sentinel-2)

- **Questions de recherche :** Comment évaluer et réduire les divergences d'estimation du LAI entre un capteur actif (ALS) et un capteur passif (Sentinel-2) dans les peuplements feuillus tempérés afin de générer des séries temporelles structurellement cohérentes ?
- **Données mobilisées :** Rasters LiDAR ALS et Sentinel-2 (pour l'instant, 10 mètres de résolution spatiale) sur trois forêts domaniales (tempérées feuillus).

1.3.1 Sous-chapitre 2.1 : Compréhension des mécanismes de divergence

- **Hypothèses :**
 - **H1 :** L'utilisation d'un paramétrage approprié des méthodes d'inversion s'appuyant sur des modèles physiques de transfert radiatif permet une meilleure comparabilité des estimations de LAI issues de Sentinel-2 et de l'ALS.
 - **H2 :** La cohérence limitée entre ces estimations, associée à la perte de sensibilité de la réflectance multispectrale pour les couverts denses, peut être partiellement corrigée en restreignant l'intégration LiDAR à la seule fraction supérieure de la canopée influençant la réflectance.
- **Méthodes :**
 1. Création d'un pipeline d'alignement inter-capteurs.
 2. Optimisation de la paramétrisation du modèle de transfert radiatif PROSAIL.
 3. Identification de la profondeur d'intégration optimale (d_{opt}) du nuage de points LiDAR pour correspondre à la sensibilité réelle de Sentinel-2.
- **Résultats & Discussion :** L'influence de la végétation sur le signal de réflectance mesuré par Sentinel-2 devient négligeable en deçà des strates supérieures. L'optimisation de PROSAIL sur l'information ALS restreinte à cette d_{opt} réduit significativement les biais d'estimation.

1.3.2 Sous-chapitre 2.2 : Correction des séries temporelles optiques

- **Méthodes :** Application des acquis théoriques du sous-chapitre 2.1 pour la correction opérationnelle du signal temporel Sentinel-2. Comparaison d'approches de correction (modèles empiriques type Random Forest versus inversion physique paramétrée sur la profondeur d'intégration d_{opt}).

- *Baseline Statique* : LiDAR fixe (LAI d'été appliqué sur toute la saison).
 - *Baseline Naïve* : Sentinel-2 brut temporel (avec lissage/gap-filling simple).
 - *Scénarios Corrigés* : Sentinel-2 temporel corrigé via les acquis du Chapitre 2 (ex: application de fonctions de correction empiriques, random forest, ou inversion PRO-SAIL paramétrée sur d_{opt}). A discuter
- **Résultats & Discussion (Attendus)** : L'approche permet de générer des séries temporelles de LAI lissées et affranchies de leurs biais de saturation initiaux. La méthode de correction retenue restitue fidèlement la dynamique phénologique du peuplement tout en conservant une magnitude physiquement cohérente avec la donnée LiDAR de référence.
 - **Conclusion du Chapitre 2** : La réduction des incertitudes inter-capteurs permet la production de séries temporelles de LAI Sentinel-2 "corrigées", validées structurellement, et directement utilisables pour caractériser la dynamique du couvert végétal dans les modèles biophysiques.

1.4 Chapitre 3 : Cartographie dynamique et suivi spatio-temporel de l'atténuation microclimatique

- **Questions de recherche** : Le couplage des séries temporelles optiques corrigées (Chapitre 2) aux variables structurelles LiDAR (Chapitre 1) permet-il de modéliser et de spatialiser la dynamique de l'atténuation thermique durant la période en feuilles ? Quelle méthode d'intégration temporelle offre les prédictions les plus robustes lors des anomalies thermiques estivales ?
- **Hypothèses** :
 - **H1** : L'intégration de la géométrie de la canopée (ALS) garantit le paramétrage adéquat des transferts radiatifs et turbulents, tandis que l'évolution temporelle du LAI (Sentinel-2) régit la dynamique saisonnière.
 - **H2** : Bien que la valeur absolue du LAI fluctue durant la période végétative, la distribution relative de la surface foliaire au sein du profil vertical demeure suffisamment stable pour justifier une mise à l'échelle (scaling) homothétique du LAD.
- **Données mobilisées** : Séries temporelles lissées de LAI Sentinel-2 corrigé, profils LiDAR statiques, forçage climatique horaire (ERA5), séries temporelles in-situ (HOBO).
- **Méthodes** :
 1. Modulation dynamique du profil vertical de référence (LAD statique extrait de l'ALS) par la courbe d'évolution temporelle du LAI Sentinel-2 au sein de MuSICA.
 2. Évaluation des scénarios d'assimilation (Baseline statique LiDAR versus Scénarios dynamiques couplés) via des métriques d'erreur standards (RMSE sur T_{max} , T_{min} , Delta T_{max}) sur la période en feuilles.
 3. *Si le temps le permet* : Production de cartes continues de Delta T_{max} à l'échelle du massif forestier.
- **Résultats & Discussion (Attendus)** : Sur-performance du scénario couplé pour capturer les décalages temporels (*lag*) du pic de chaleur sous couvert, particulièrement lors des séquences caniculaires. La discussion mettra en perspective ces résultats spatialisés continus face aux approches alternatives basées sur le LiDAR spatial (GEDI), en soulignant les avantages du couplage ALS/S2 pour s'affranchir des limites d'interpolation entre les empreintes satellitaires.
- **Conclusion du Chapitre 3 (Attendue)** : Le couplage des approches radiatives et géométriques lève les verrous inhérents à la caractérisation continue des couverts forestiers. Cette méthode offre un cadre robuste pour la cartographie spatio-temporelle à haute résolution des refuges thermiques, proposant une alternative spatialement continue aux limites d'échantillonnage de l'altimétrie spatiale.

1.4.1 Analyses Annexes et Perspectives

- **Évaluation de la donnée optique brute :** L'intégration "naïve" de données Sentinel-2 standardisées dans MuSICA fera l'objet d'une analyse différenciée (potentiellement appuyée par des bases de données de hauteur indépendantes de type FORMS-H) afin d'isoler les limites intrinsèques de ce type d'imagerie satellitaire.
- **Sensibilité au forçage climatique :** Comparaison des sorties du modèle sous forçage de réanalyse (ERA5) versus forçage localisé (SAFRAN).
- **Généralisation multi-sites :** Validation des méthodologies sur les massifs de l'Aigoual et de Mormal.
- **Comparaison inter-modèles :** Confrontation des estimations MuSICA avec le modèle microclimf.

2 Figures — Résultats principaux

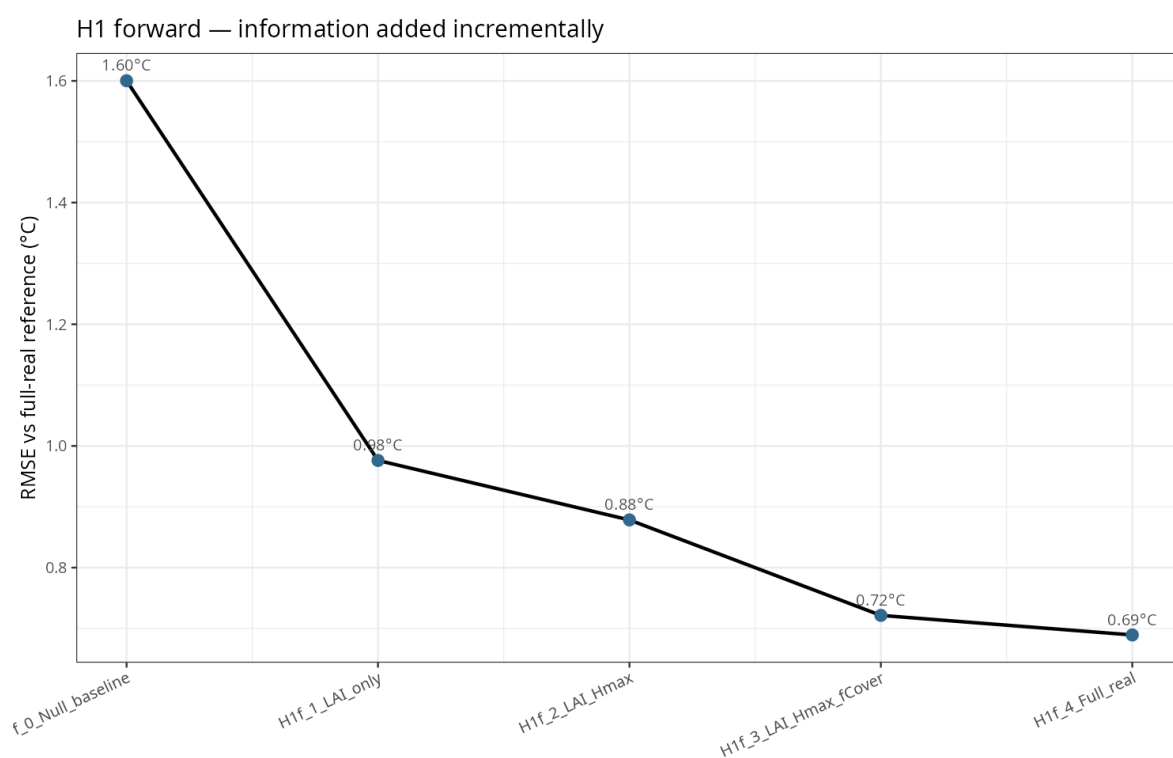


Figure 1: Fig. 1 — Courbe d'inclusion progressive H1 (SC0→SC4) : gain cumulatif de R^2 à chaque ajout structurel.

H1 — Robustesse du ranking : Forward vs LOO

Concordance Forward/LOO = ranking indépendant de l'ordre | 2 barres LOO pour Profil LAD = sensibilité à la baseline

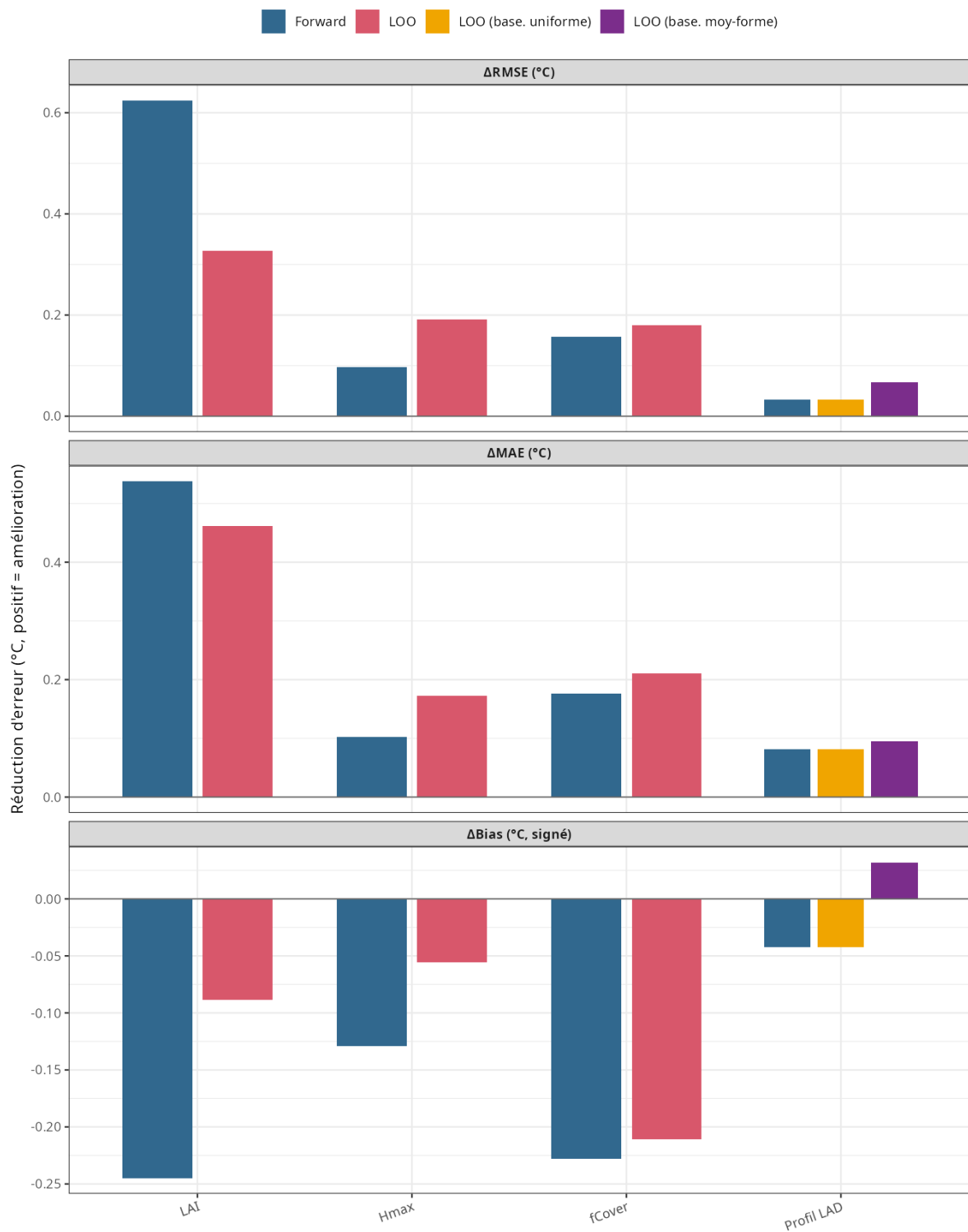


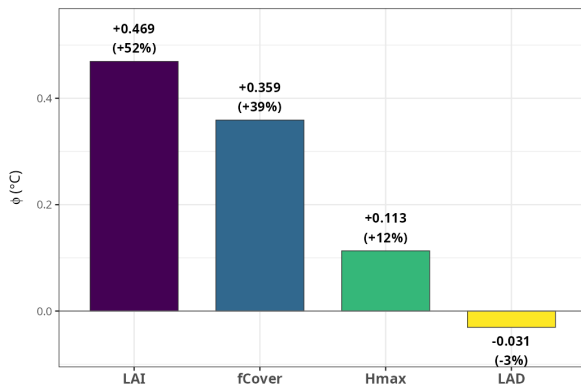
Figure 2: Fig. 2 — Classement des contributions : forward / LOO / factorial.

Shapley attribution of canopy structural drivers on ΔT_{max}

Blois oak forest | MuSICA simulations | Summer 2021-2022

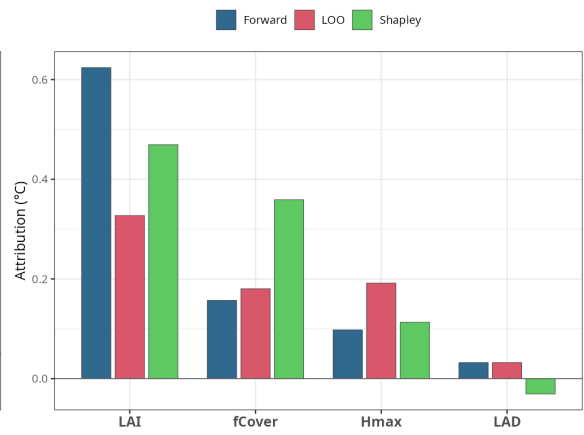
A. Shapley attribution of canopy structural drivers

Total buffering $v(N) = 0.911\text{ }^{\circ}\text{C}$ | $\sum \phi = 0.9109$



B. Method comparison : Forward / LOO / Shapley

Shapley averages over all permutations — resolves Forward/LOO divergence



C. The $2^4 = 16$ coalitions of the Shapley lattice

$v(S) = \text{RMSE}(\emptyset) - \text{RMSE}(S)$ | L=LAI, H=Hmax, F=fCover, D=LAD

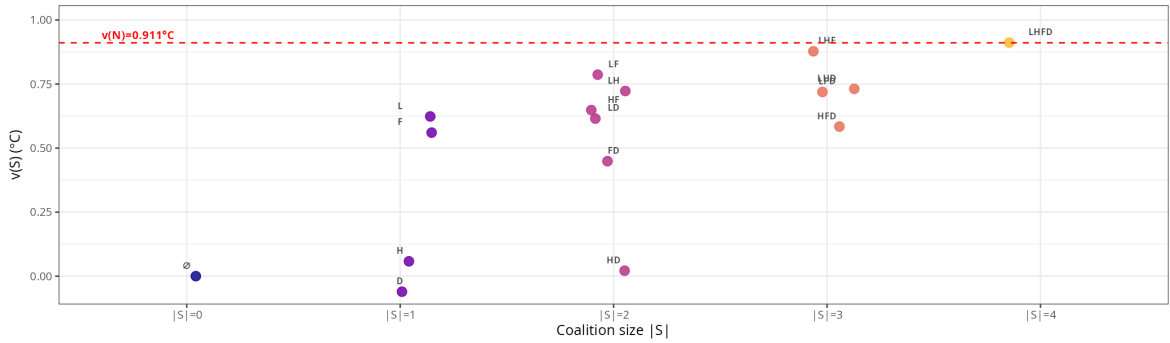


Figure 3: Fig. 3 — Attribution exacte de Shapley sur le treillis 2^4 .

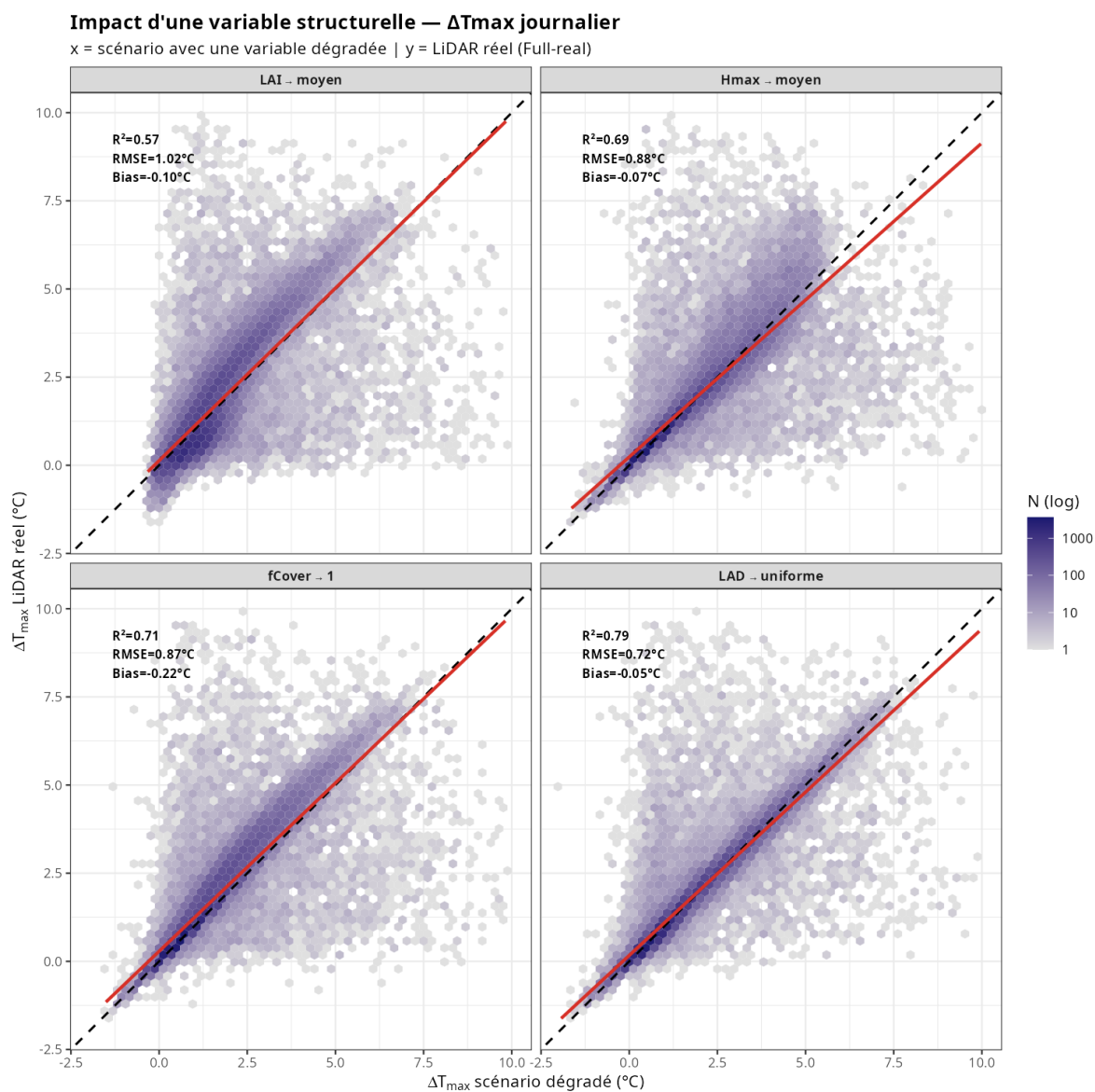


Figure 4: Fig. 4 — LOO : nuages de points $\Delta T_{\max}$ par scénario dégradé.

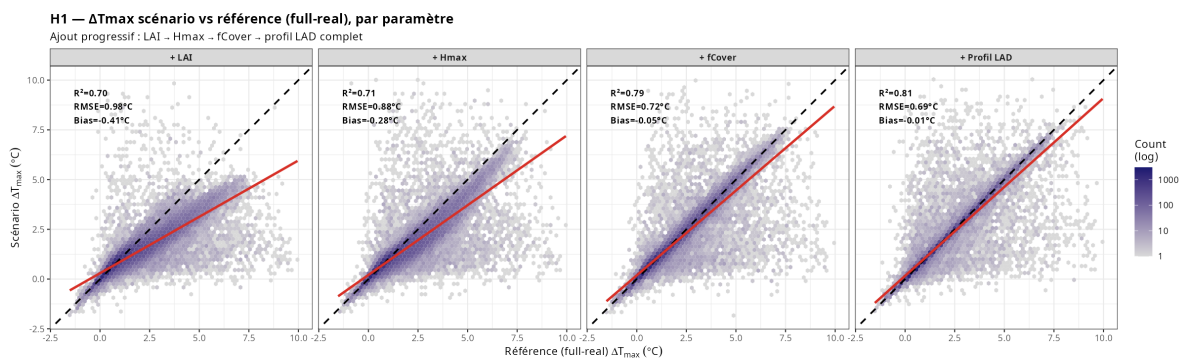


Figure 5: Fig. 5 — Scénarios forward vs référence (REF_all_real).

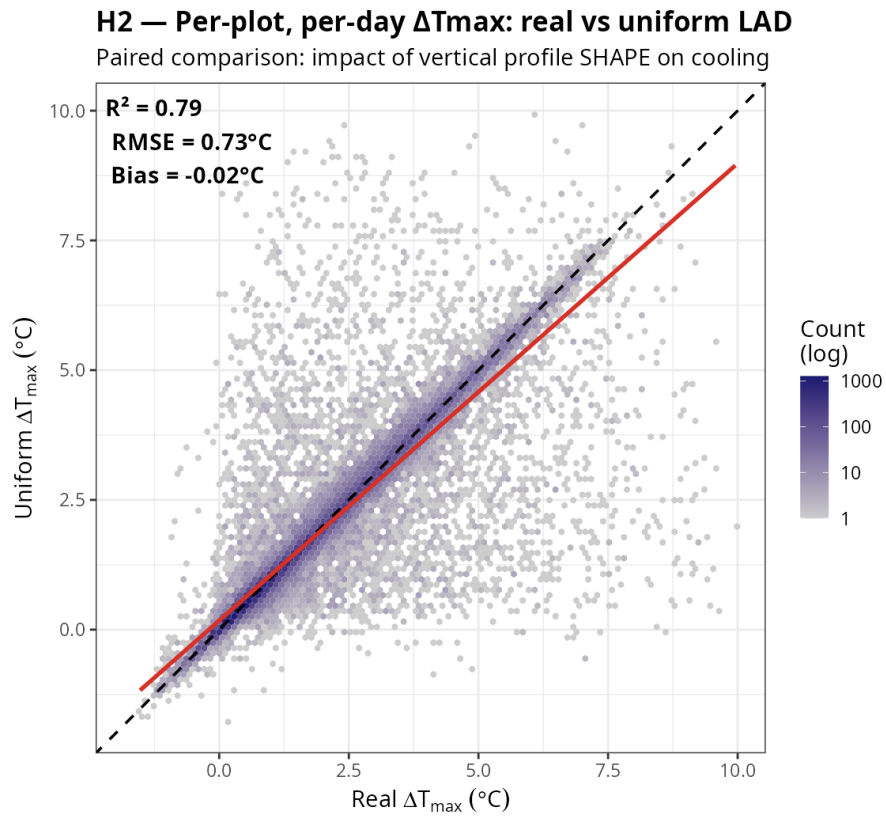


Figure 6: Fig. 6 — H2 : LAD réel vs LAD uniforme — comparaison appariée.

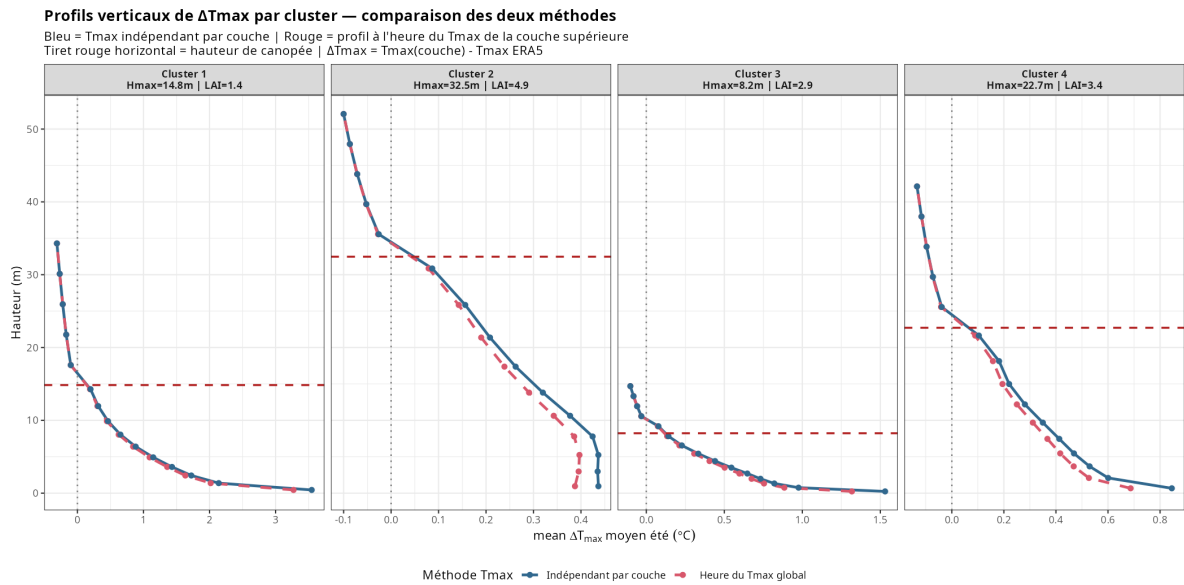


Figure 7: Fig. 7 — Profils verticaux $\Delta T_{\max}$ par cluster structurel.

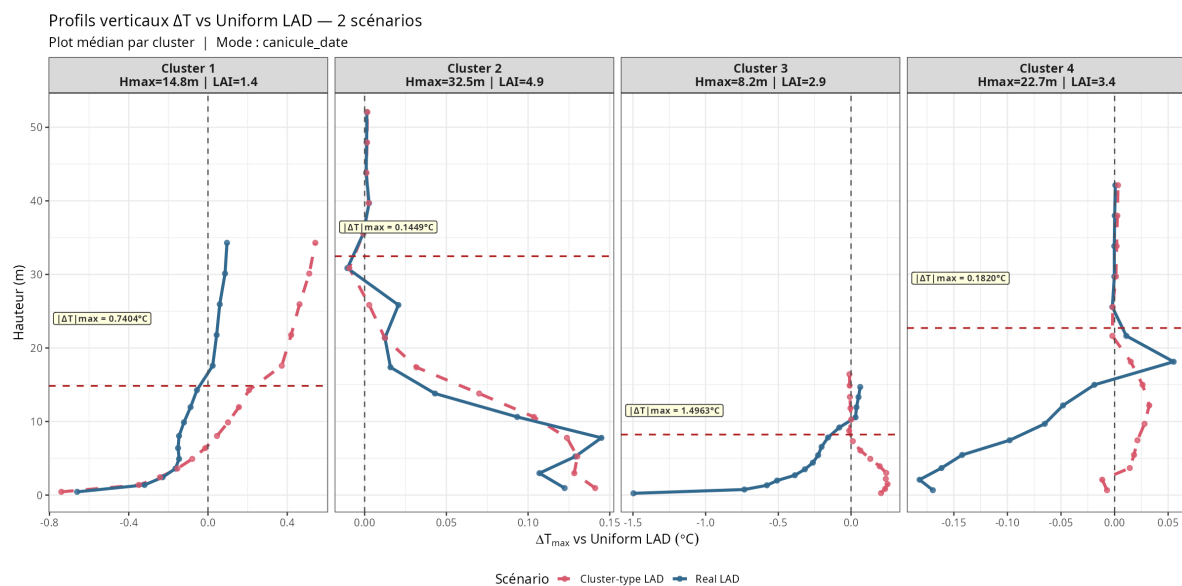


Figure 8: Fig. 8 — Comparaison 3 voies (réel / type-cluster / uniforme) en canicule.

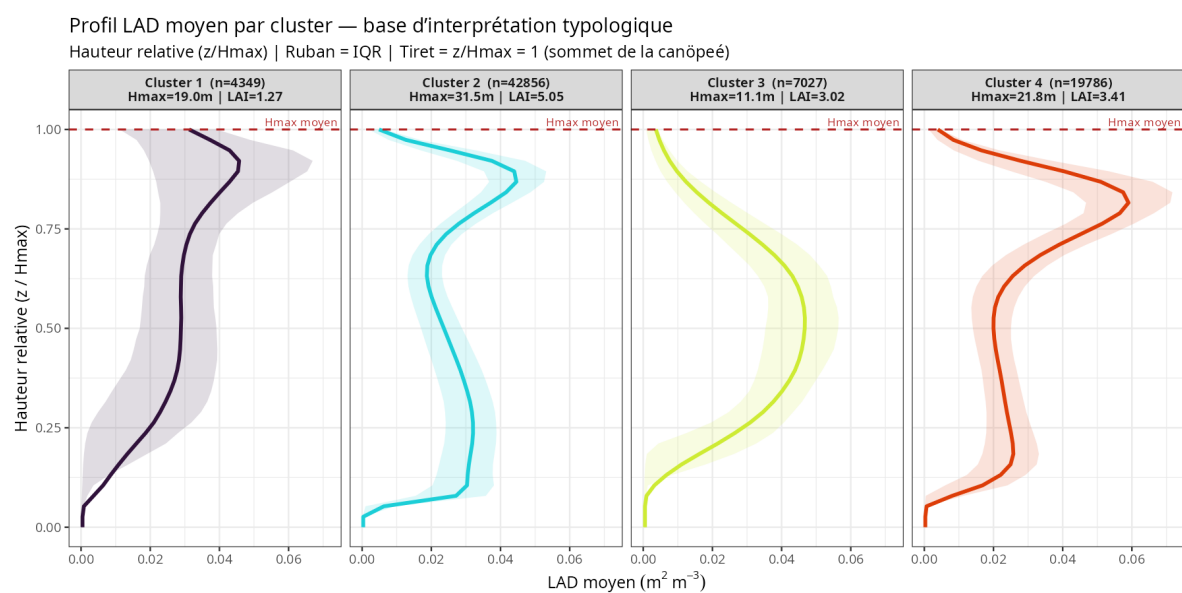


Figure 9: Fig. 9 — Profils LAD moyens par cluster (k=4).

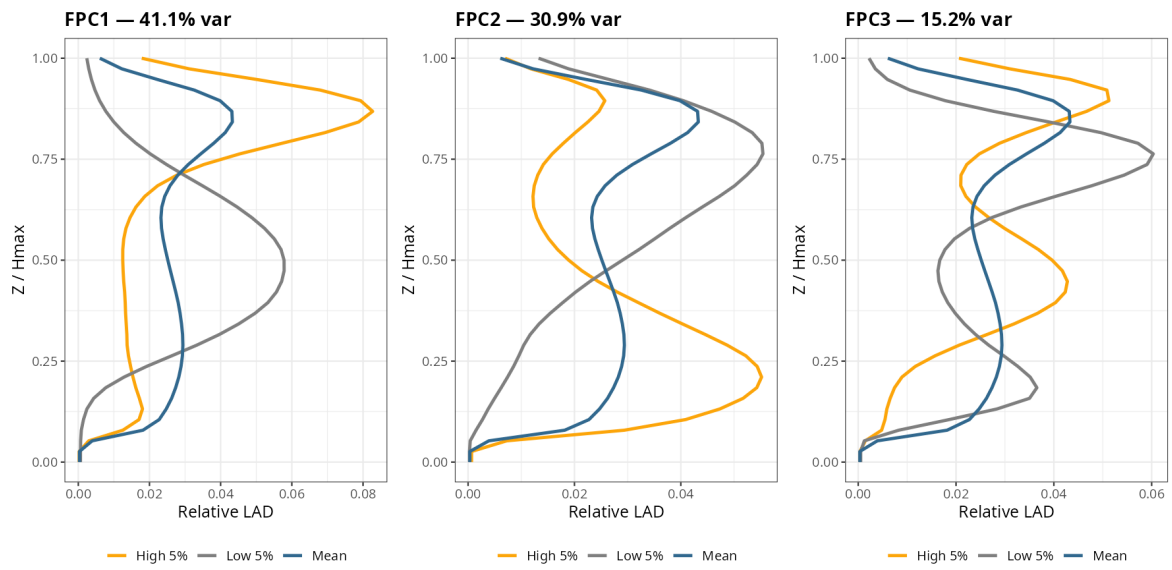


Figure 10: Fig. 10 — Harmoniques ACPF : FPC1/FPC2 décrivent l'asymétrie verticale.

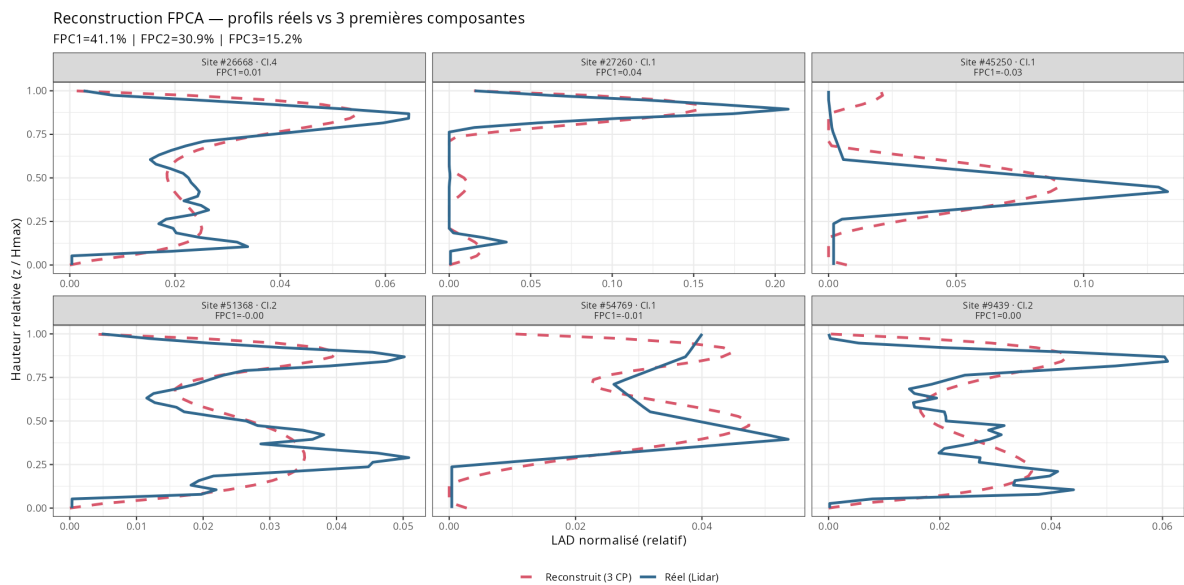


Figure 11: Fig. 11 — Reconstruction ACPF sur 6 sites représentatifs.

3 Figures — Annexes

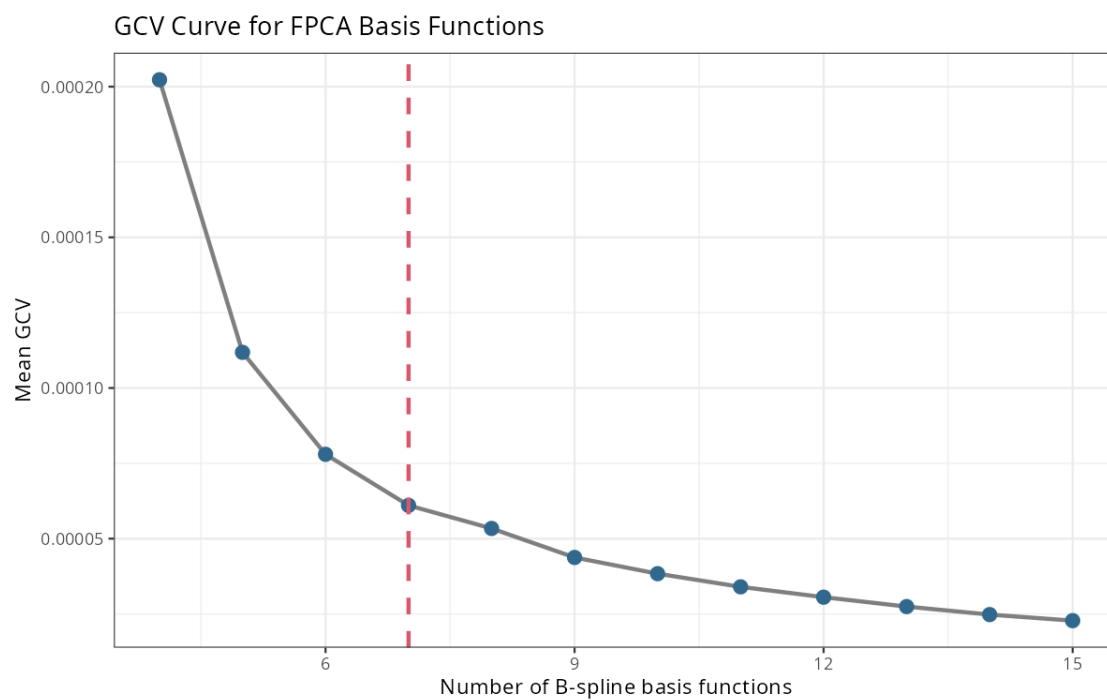


Figure 12: Fig. A1 — Critère GCV pour la sélection du lissage ACPF.

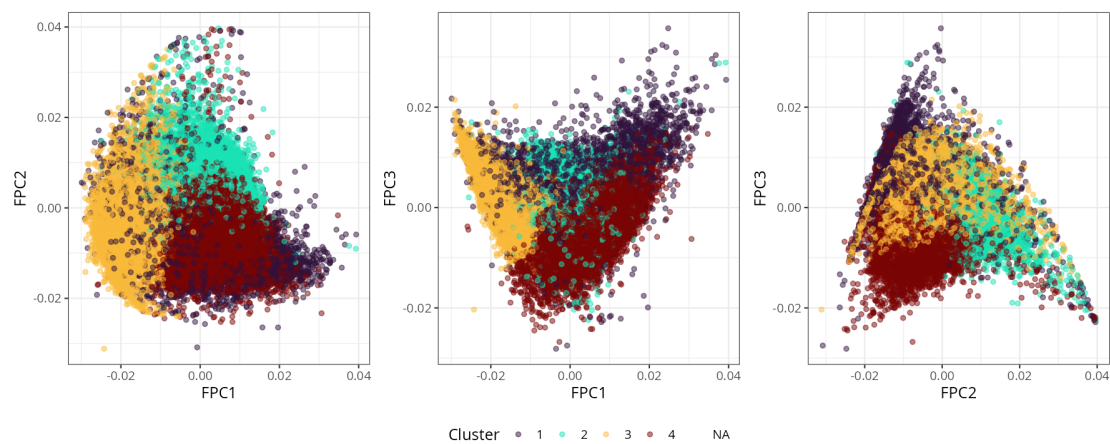


Figure 13: Fig. A2 — Scores FPC colorés par cluster.

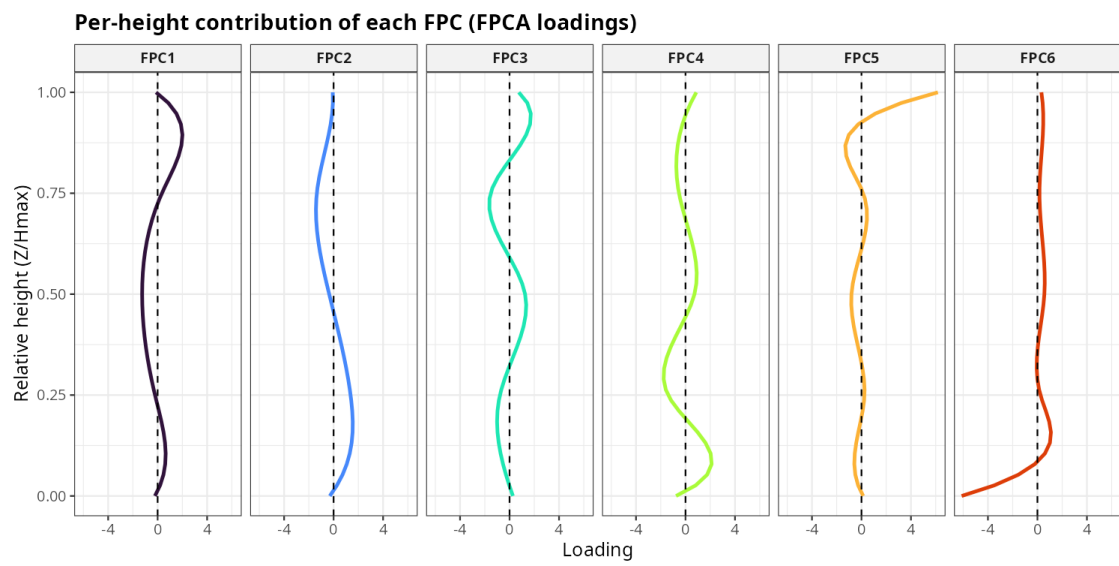


Figure 14: Fig. A3 — Chargements FPC1/FPC2.

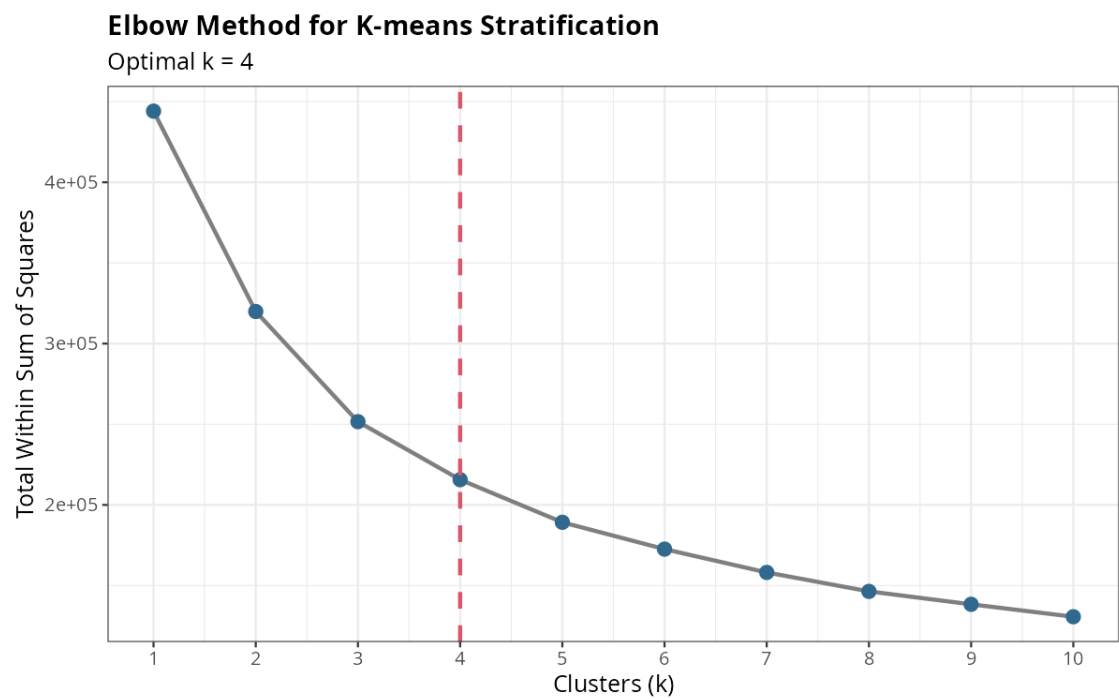


Figure 15: Fig. A4 — Courbe coude k-means (sélection $k=4$).

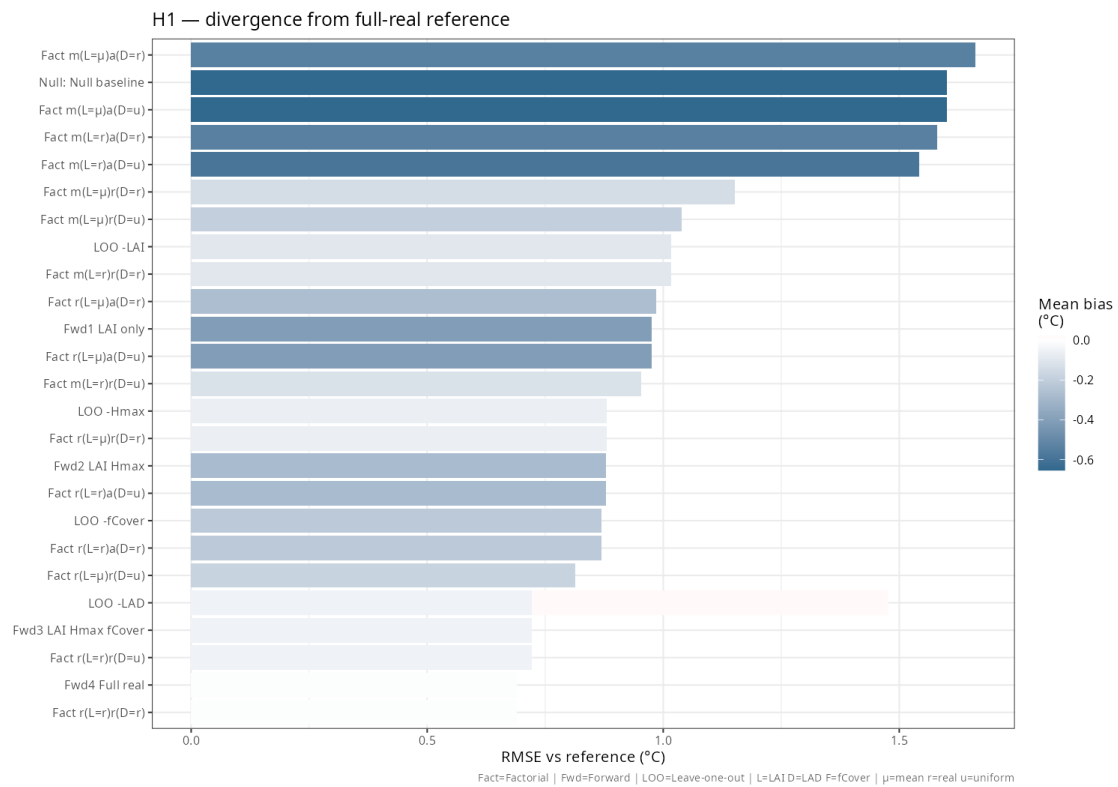


Figure 16: Fig. A5 — Hiérarchie des scénarios H1.

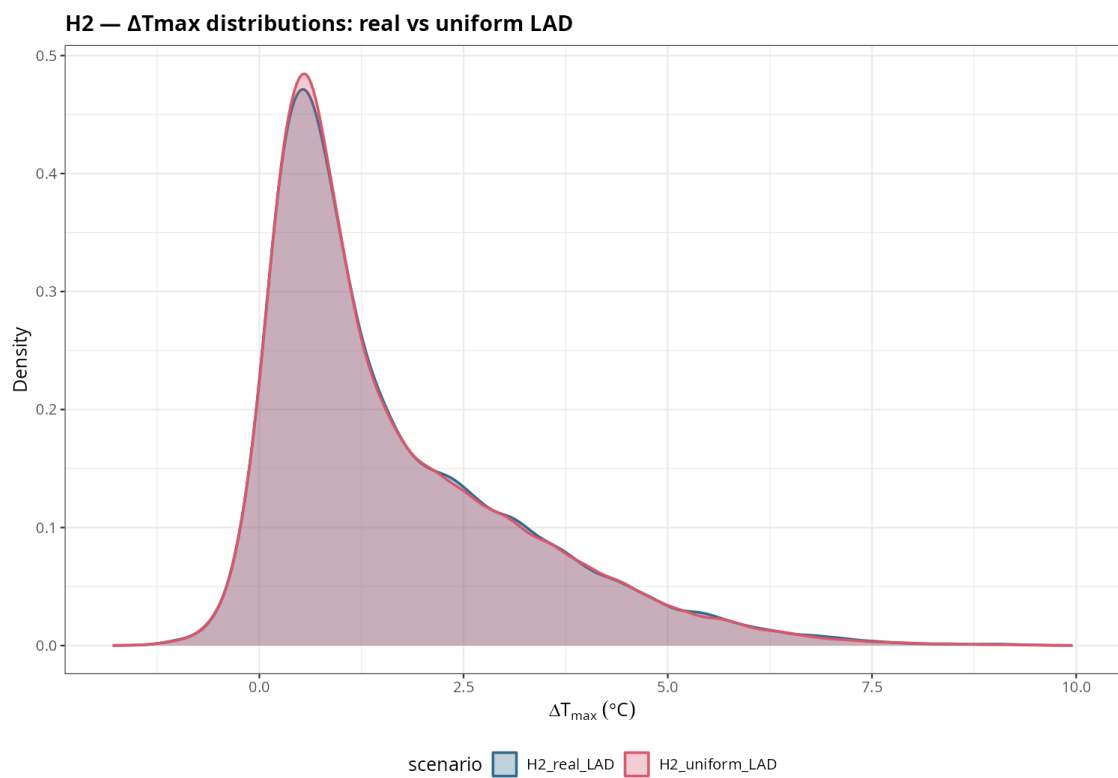


Figure 17: Fig. A6 — Distributions des différences H2 (réel–uniforme).

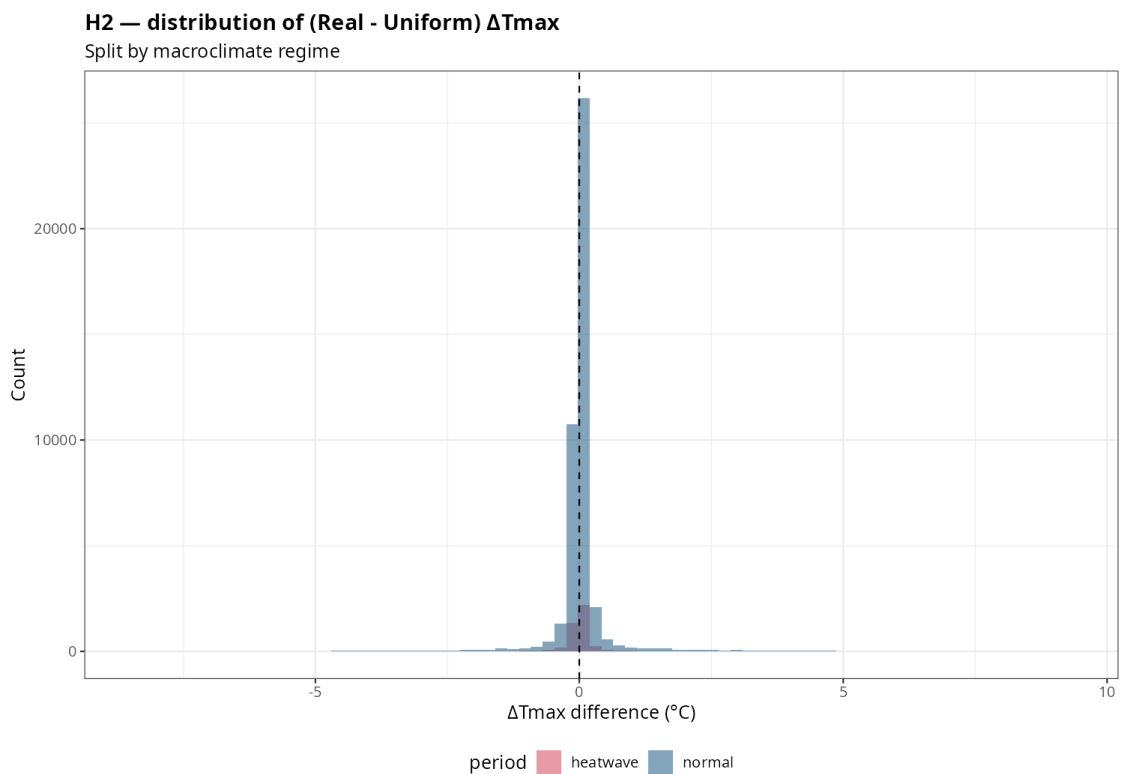


Figure 18: Fig. A7 — Analyse H2 en période caniculaire.

4 Code source R

Tous les chemins sont relatifs à la racine du projet.

4.1 main.R — Orchestrateur principal (pipeline 7 étapes)

```
# =====
# Chapter 1 – Entry point
# =====

if (rstudioapi::isAvailable()) {
  setwd(dirname(rstudioapi::getSourceEditorContext()$path))
}

suppressPackageStartupMessages({
  library(musica.tools)
  library(rmusica)
  library(ncdf4)
  library(tidyverse)
  library(lubridate)
  library(broom.mixed)
  library(ggeffects)
  library(patchwork)
  library(viridis)
  library(mgcv)
  library(terra)
  library(fda)
  library(corrplot)
  library(sf)
  library(clhs)
```

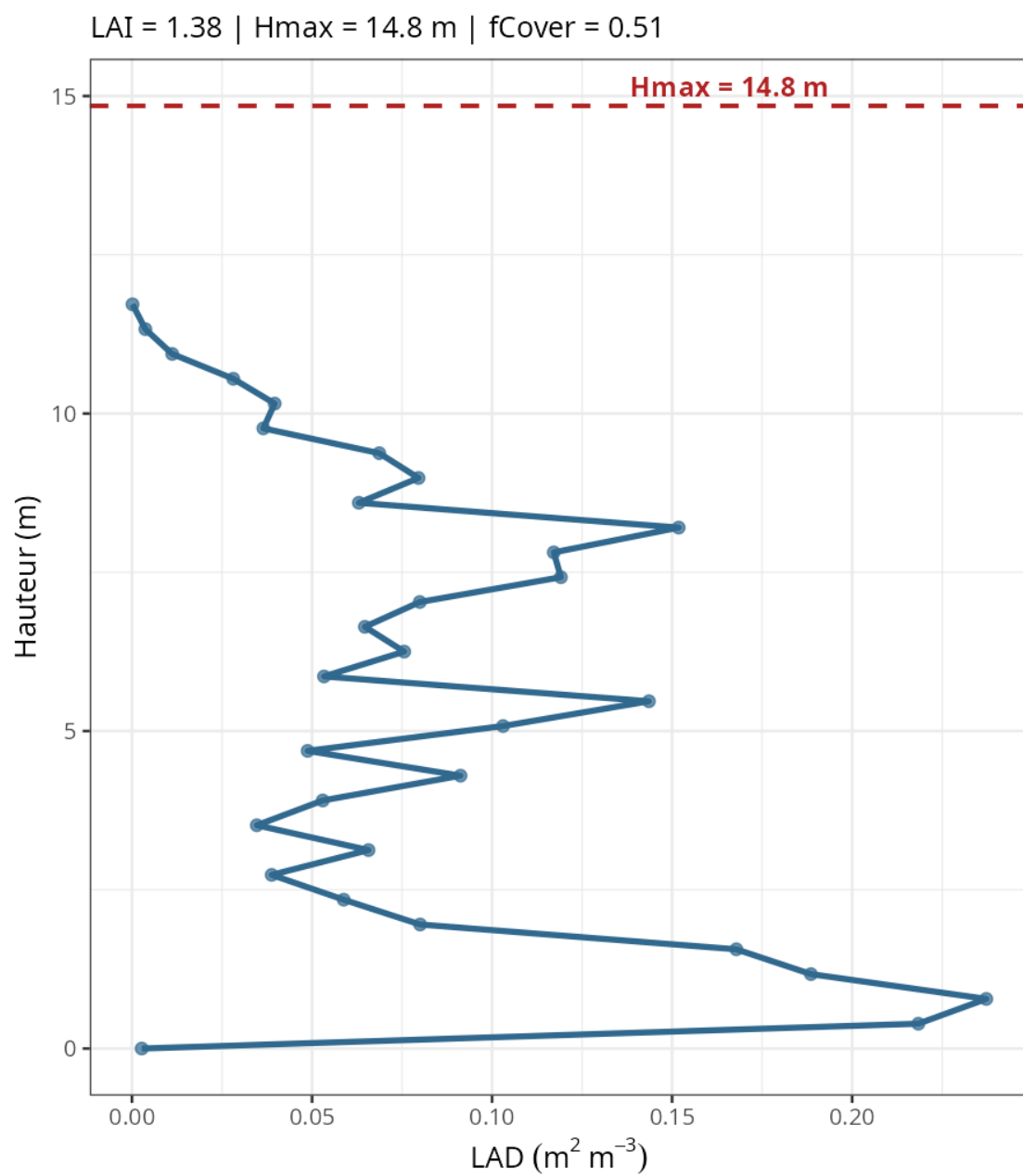


Figure 19: Fig. A8a — Profil LAD médian cluster 1.

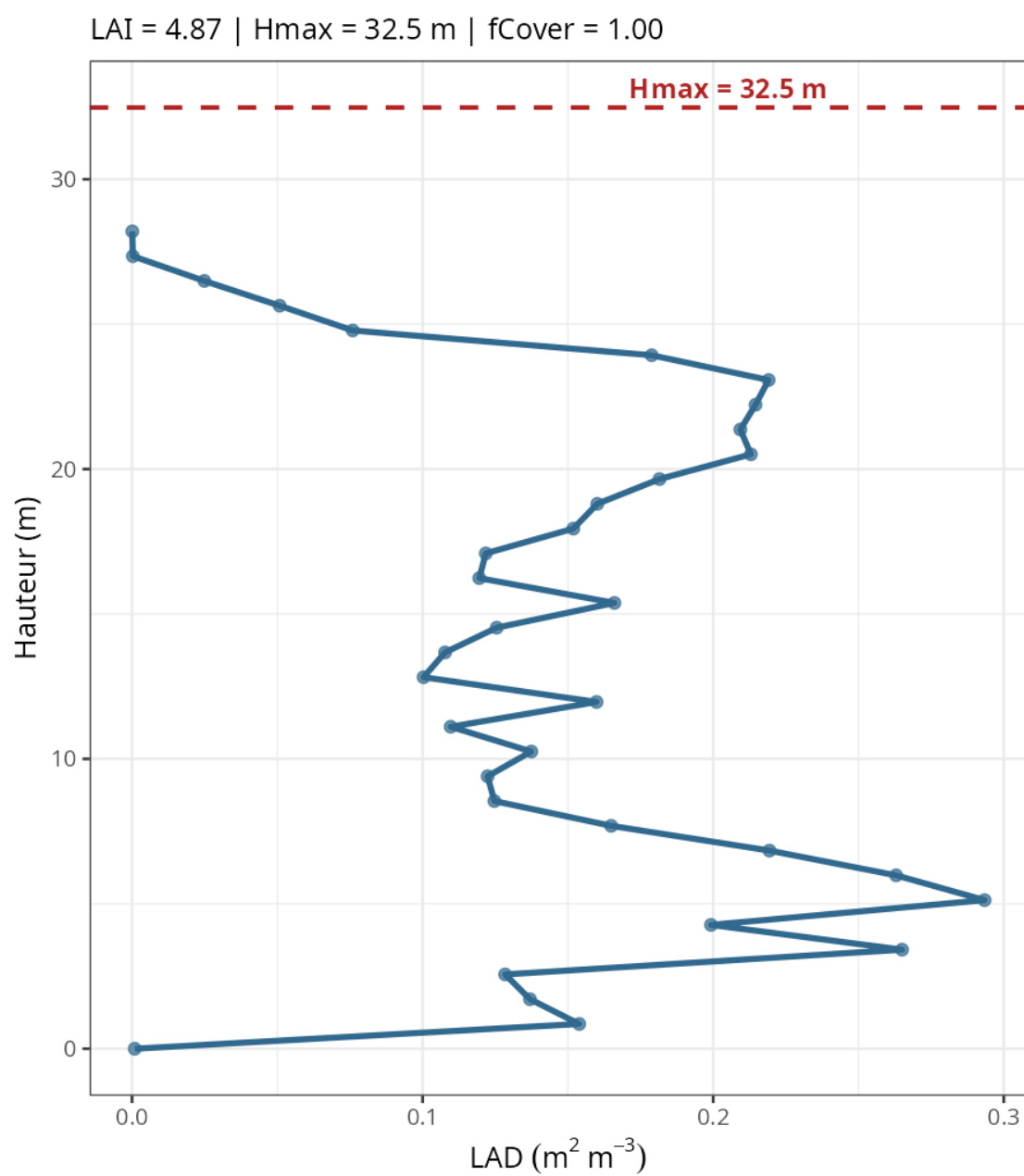


Figure 20: Fig. A8b — Profil LAD médian cluster 2.

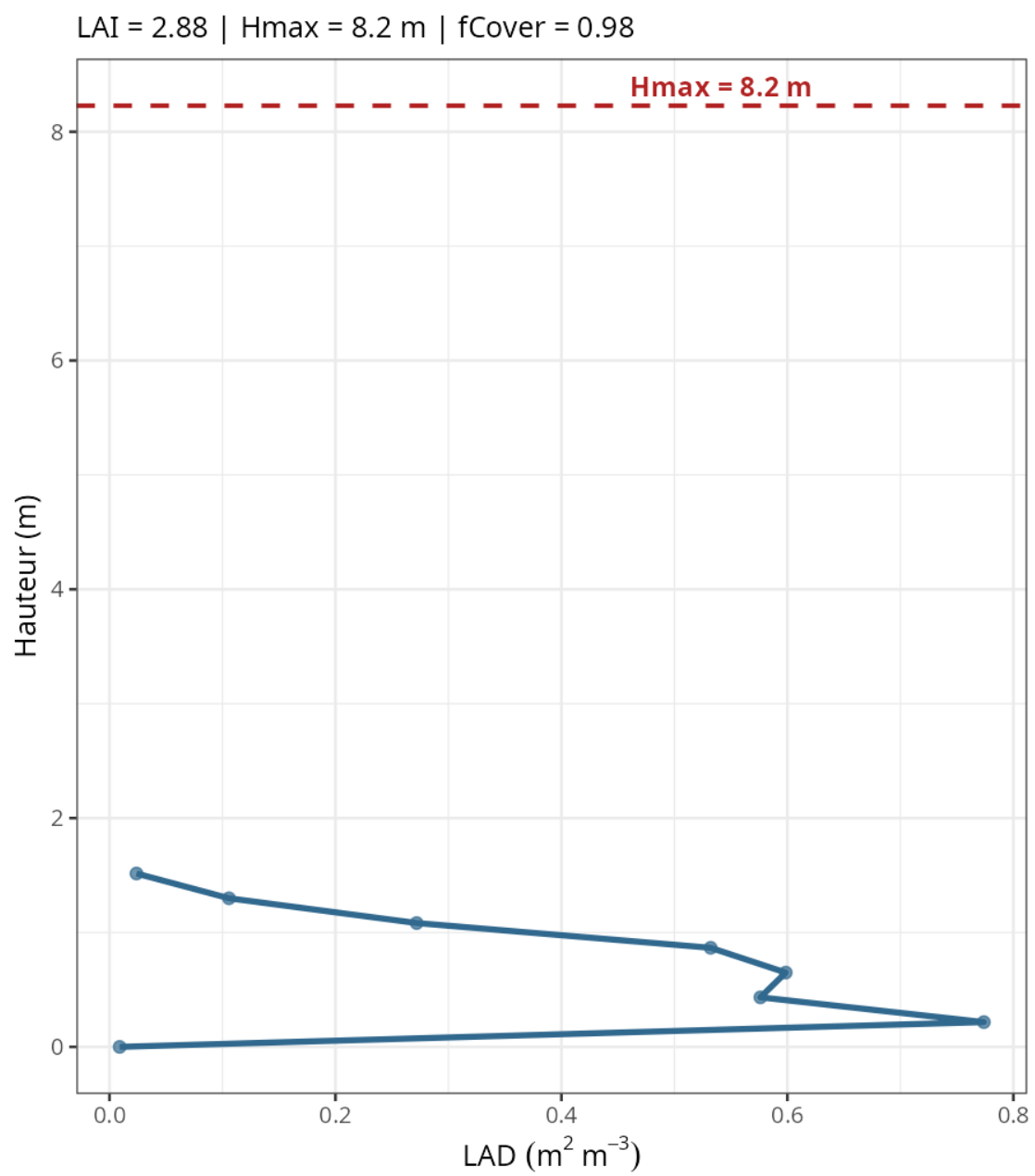


Figure 21: Fig. A8c — Profil LAD médian cluster 3.

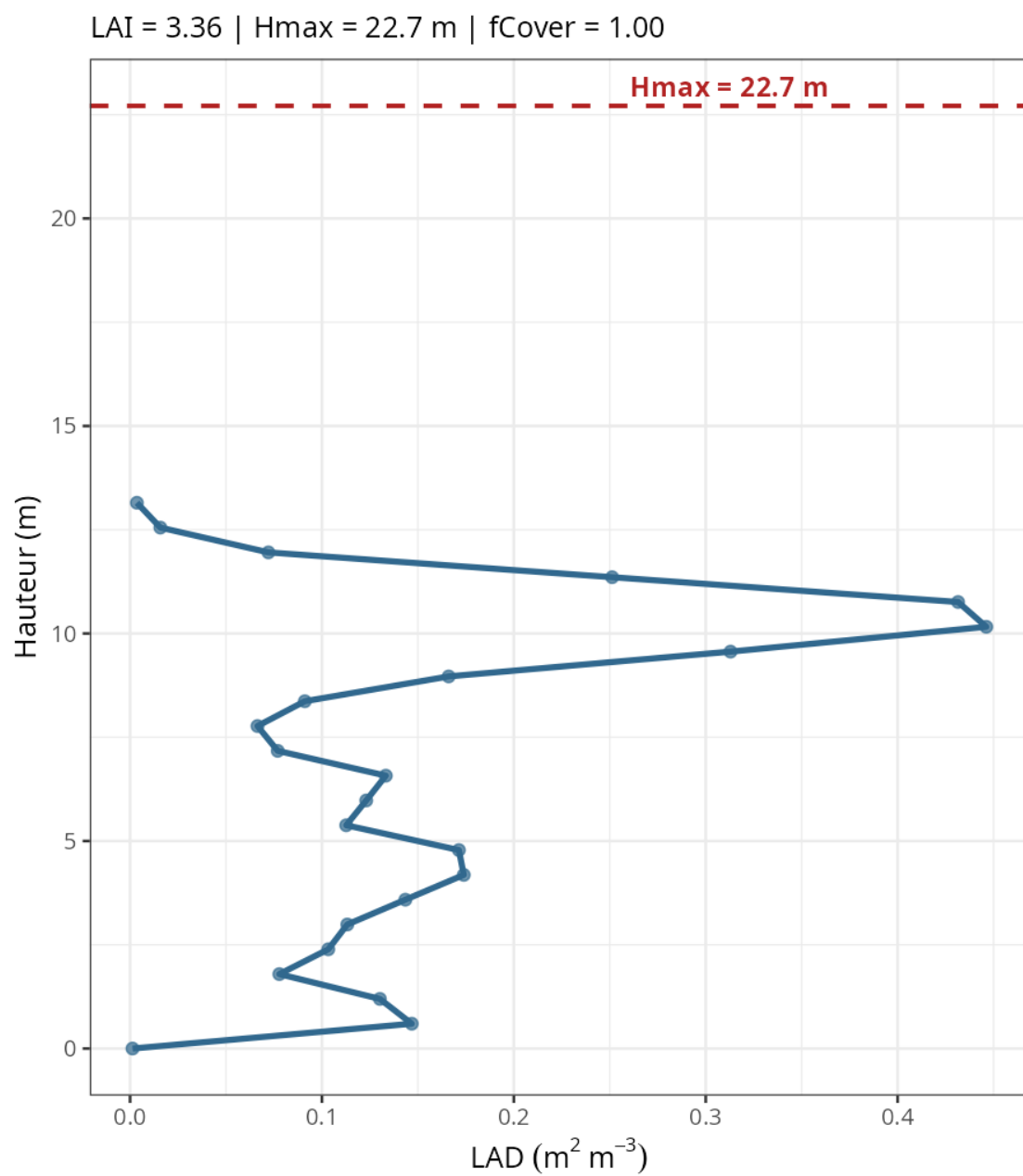


Figure 22: Fig. A8d — Profil LAD médian cluster 4.

H2 — Per-plot, per-day $\Delta T_{\max}$: real vs cluster-type LAD

Paired comparison: impact of LAD shape typology on cooling

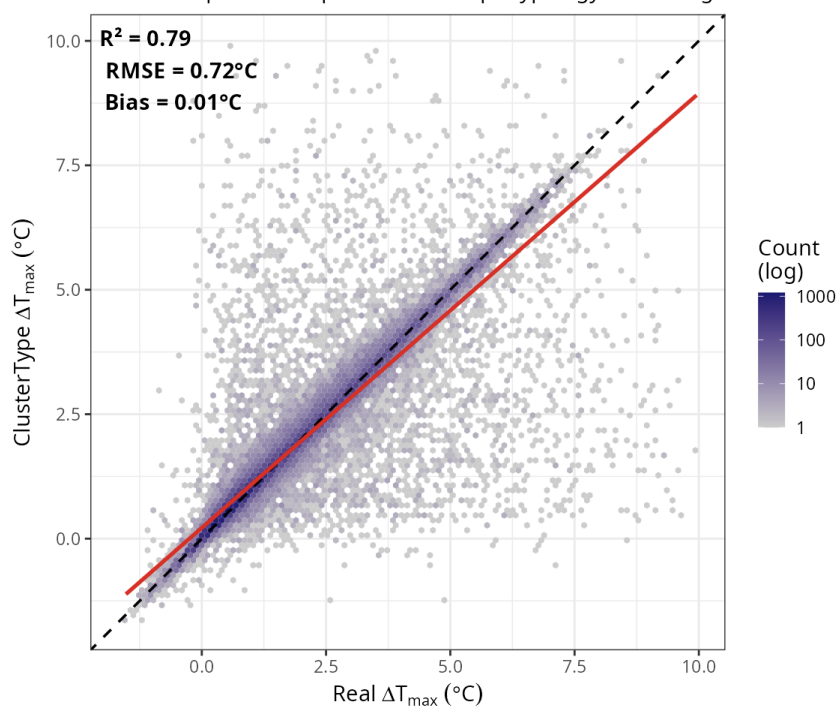


Figure 23: Fig. A9 — LAD type-cluster vs LAD réel (apparié).

GAMM Marginal Effects

Holding all other variables at their mean

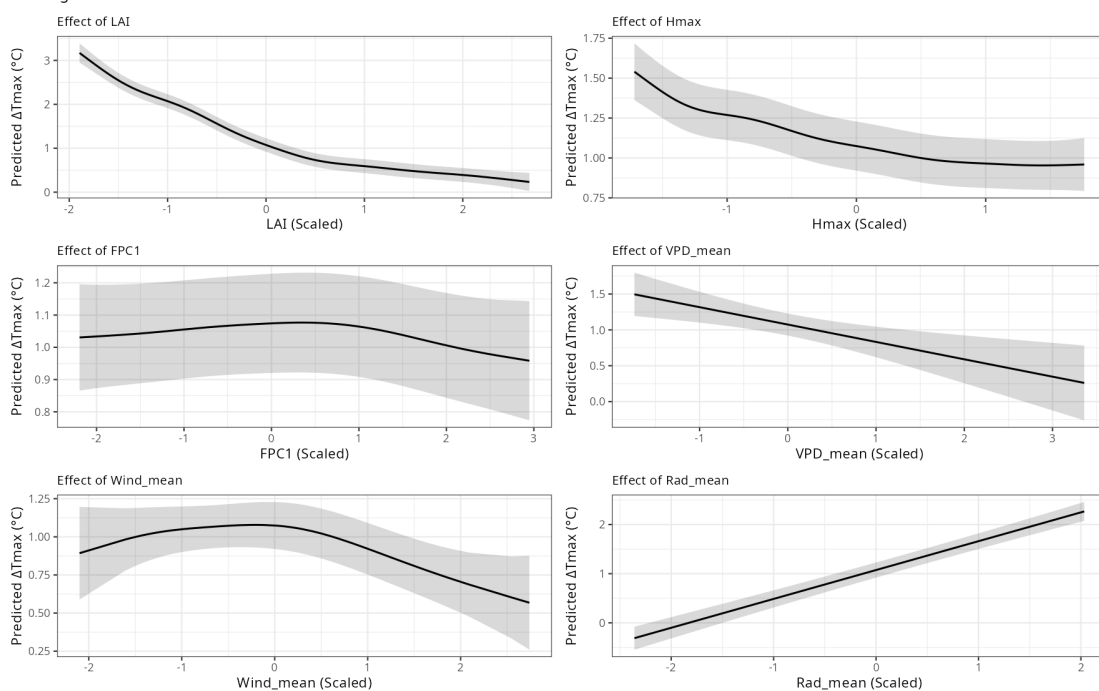


Figure 24: Fig. A10 — Effets marginaux GAMM (LAI, Hmax, fCover, FPCs).

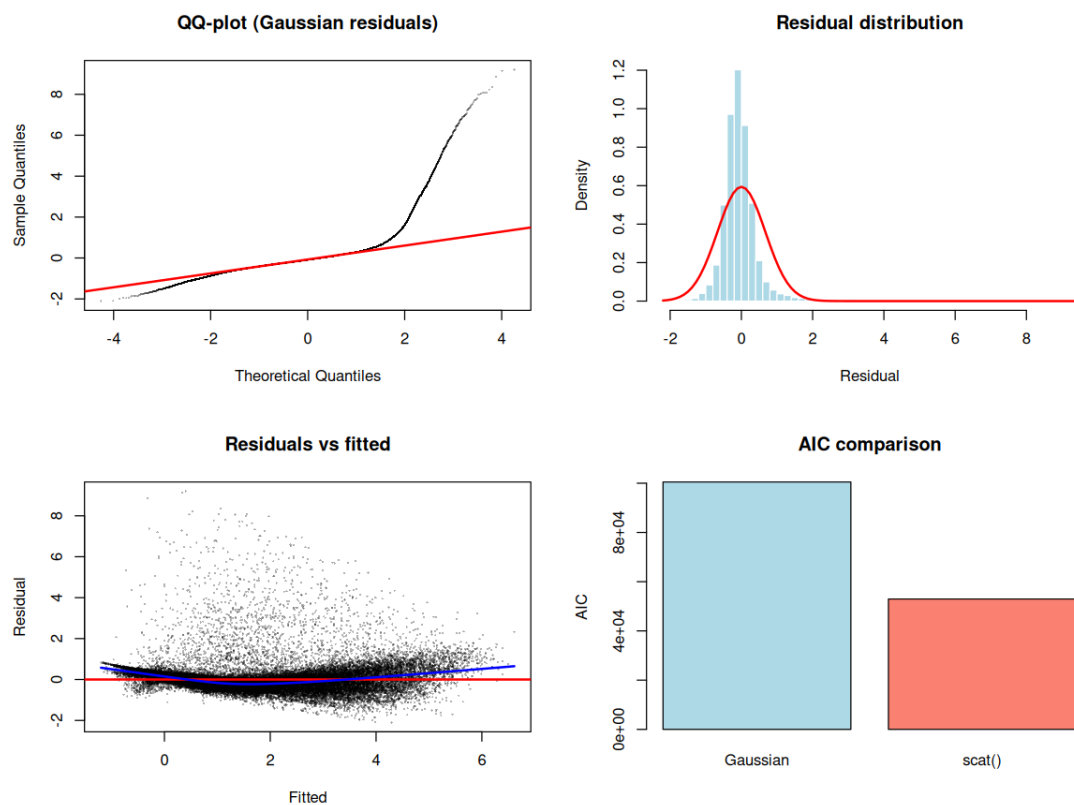


Figure 25: Fig. A11 — Diagnostics résiduels GAMM.

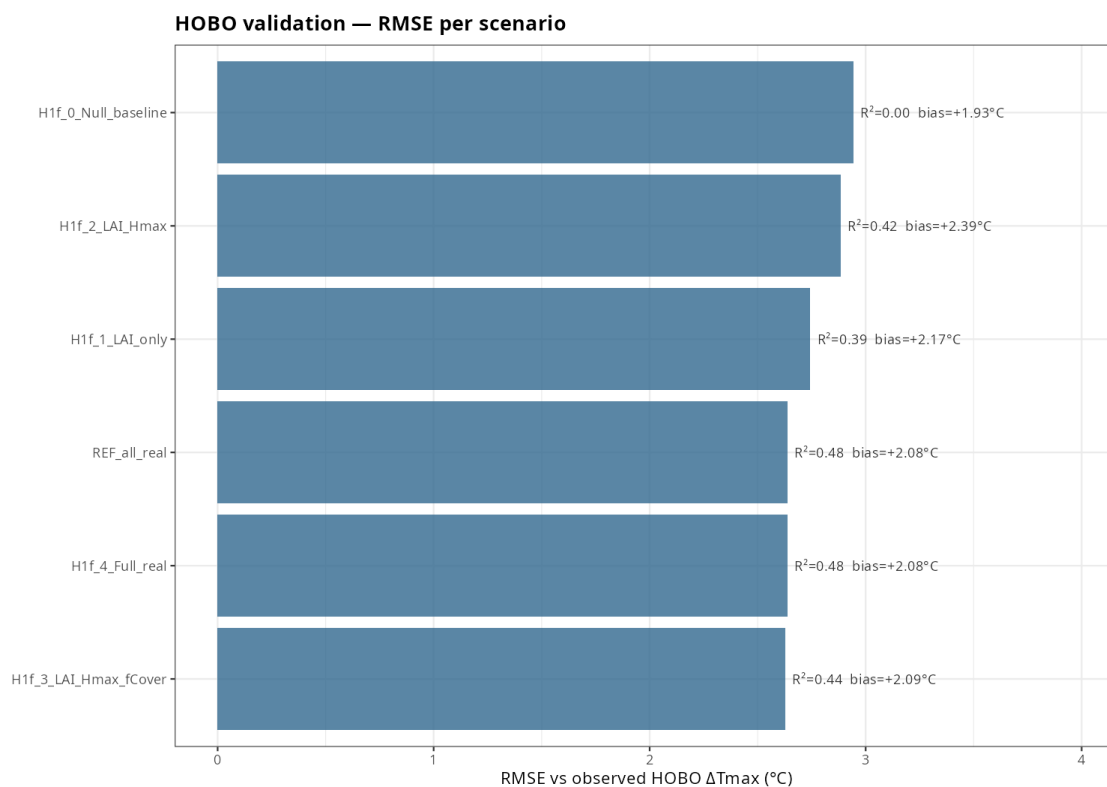


Figure 26: Fig. A12 — Validation HOB0 : métriques par scénario.

Time-series — Selected HOBOS vs MuSICA

Plots chosen by their average buffering capacity during summer

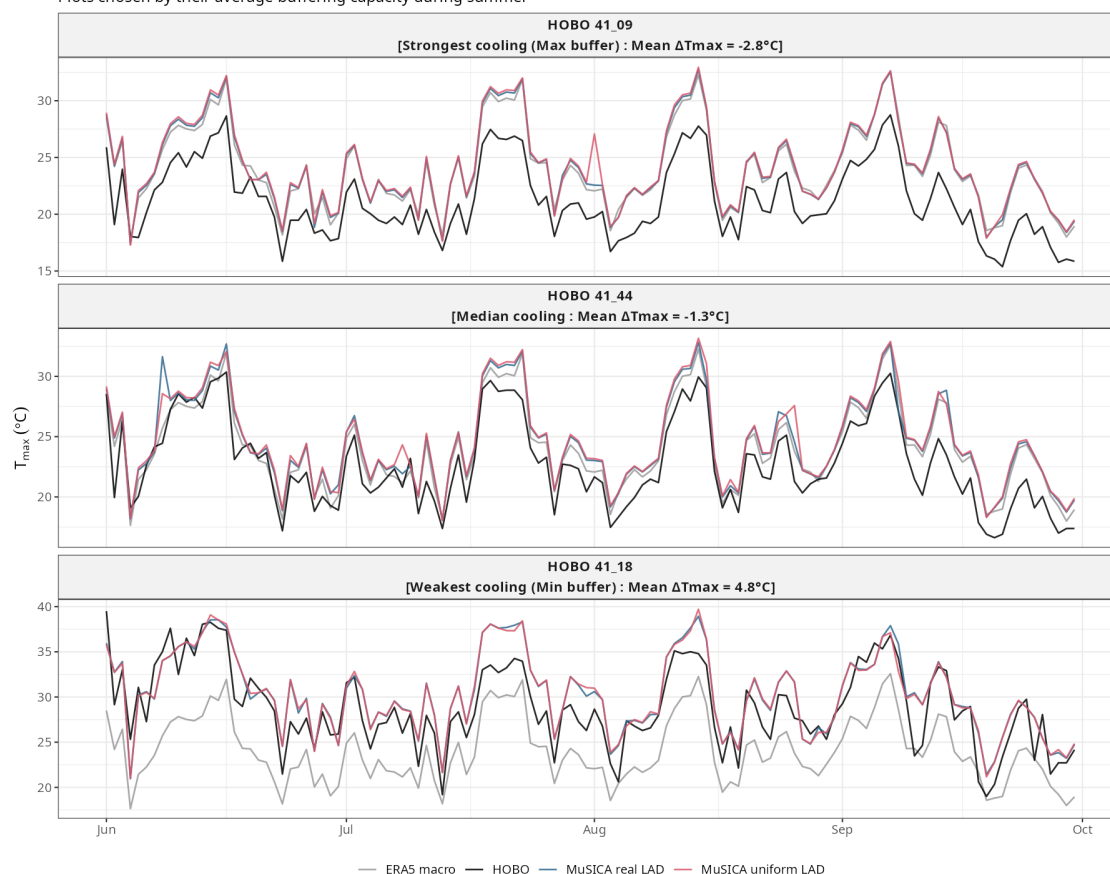


Figure 27: Fig. A13 — Séries temporelles HOBOS — 3 sites représentatifs.

Annex H3 — Optical Proxy vs Structural Reality (Density)

$R^2 = 0.34$ | $RMSE = 1.53^{\circ}\text{C}$ | $Bias = -0.87^{\circ}\text{C}$

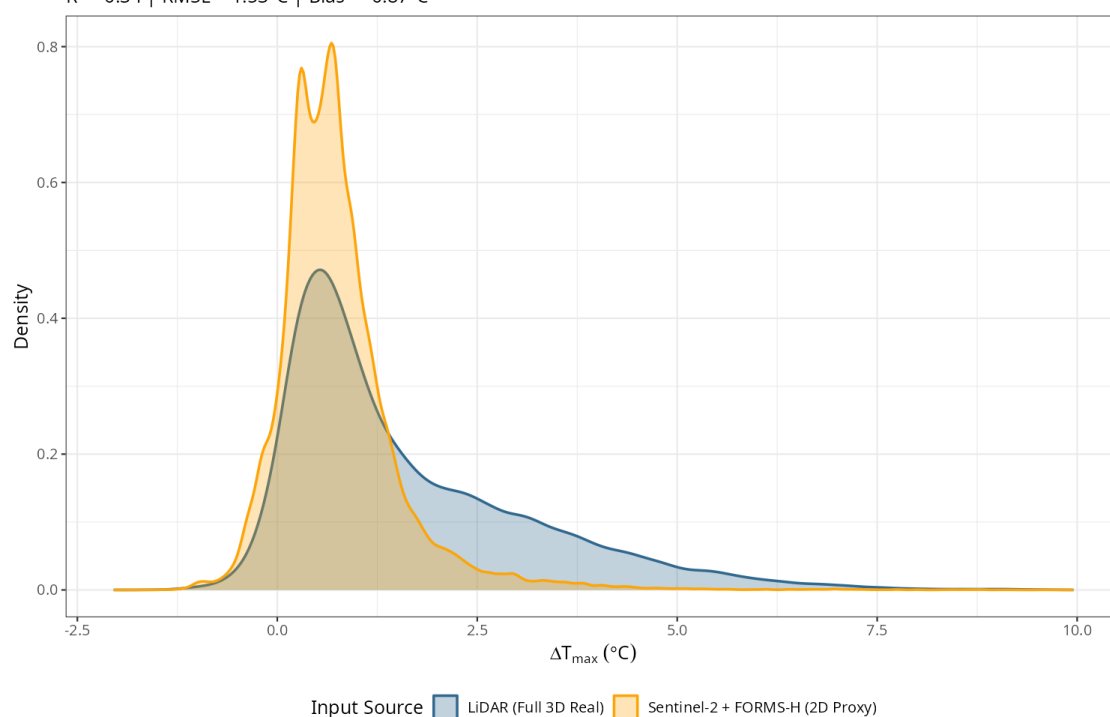


Figure 28: Fig. A14 — Proxy Sentinel-2 vs densité LiDAR.

Annex H3 — Per-plot, per-day $\Delta T_{\max}$: S2 Proxy vs LiDAR

Paired comparison: impact of 2D optical proxy on spatial prediction

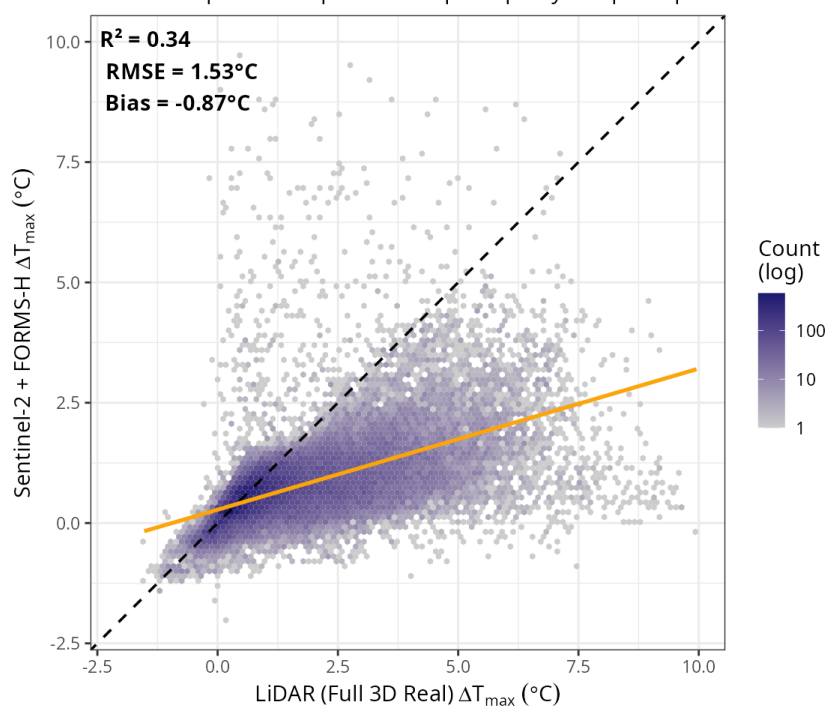


Figure 29: Fig. A15 — Sentinel-2 vs LiDAR : comparaison appariée par plot×jour.

Contrôle cohérence : REF_all_real ↔ H2_real_LAD

Mêmes paramètres LiDAR — écart attendu ≈ 0

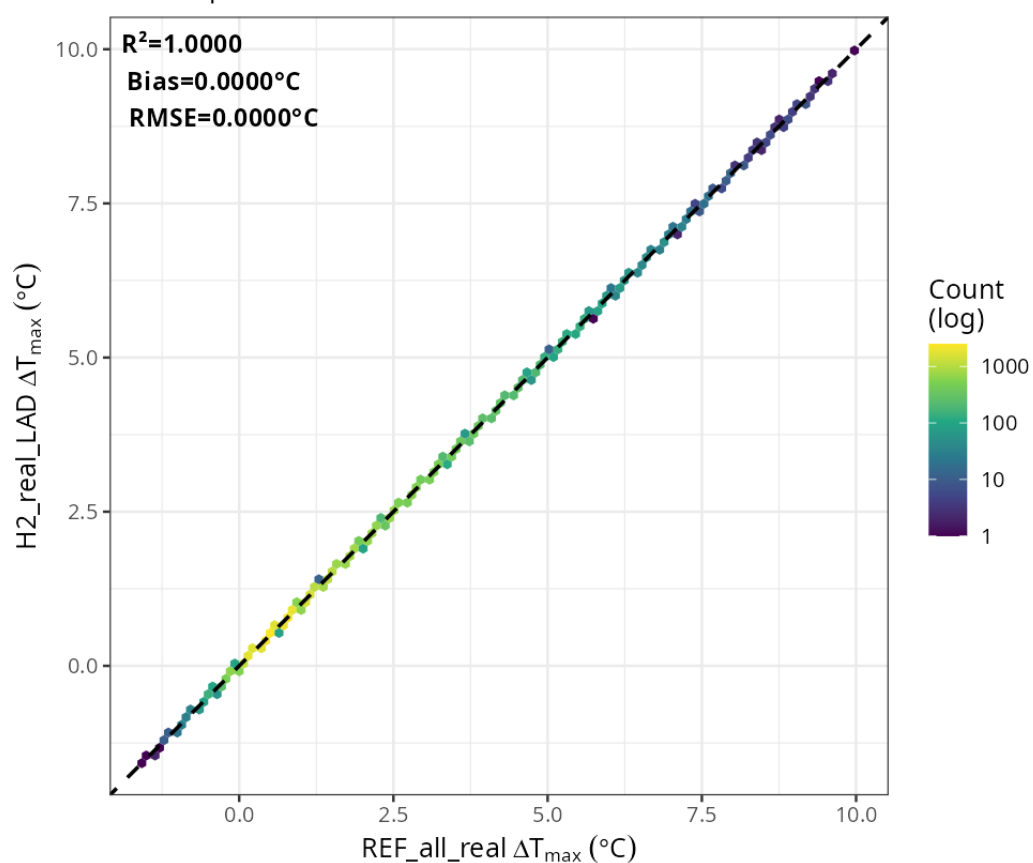


Figure 30: Fig. A16 — Contrôle cohérence REF_all_real ↔ H2_real_LAD.

H2 — Stabilité saisonnière de l'effet profil LAD (Real - Uniform)

Effet moyen : Canicule = -0.01°C | Normal = 0.02°C

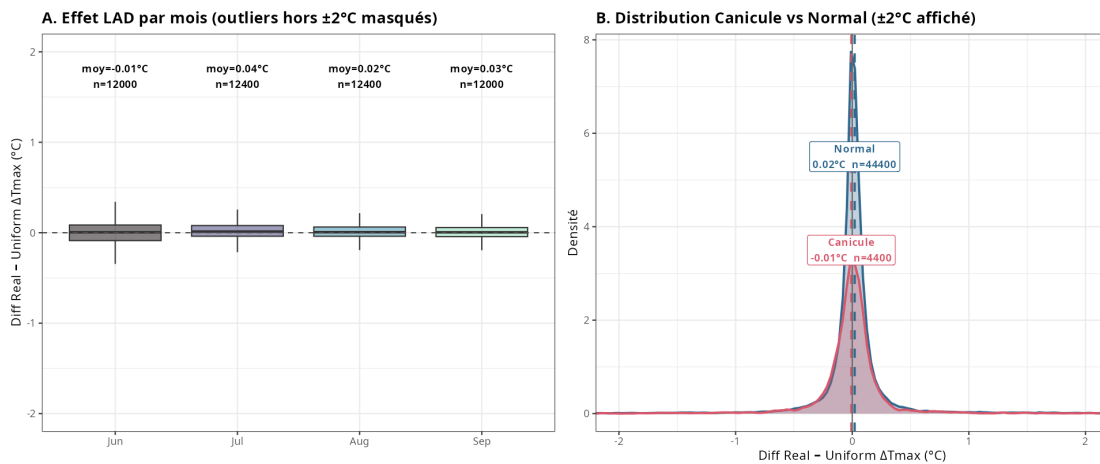


Figure 31: Fig. A17 — Stabilité saisonnière de l'effet profil LAD (H2).

Shapley bootstrap — stabilité sur 5 sous-échantillons

Barres = moyenne $\pm \sigma$ | points = tirages individuels | $\blacklozenge$ = Shapley exact (échantillon complet)

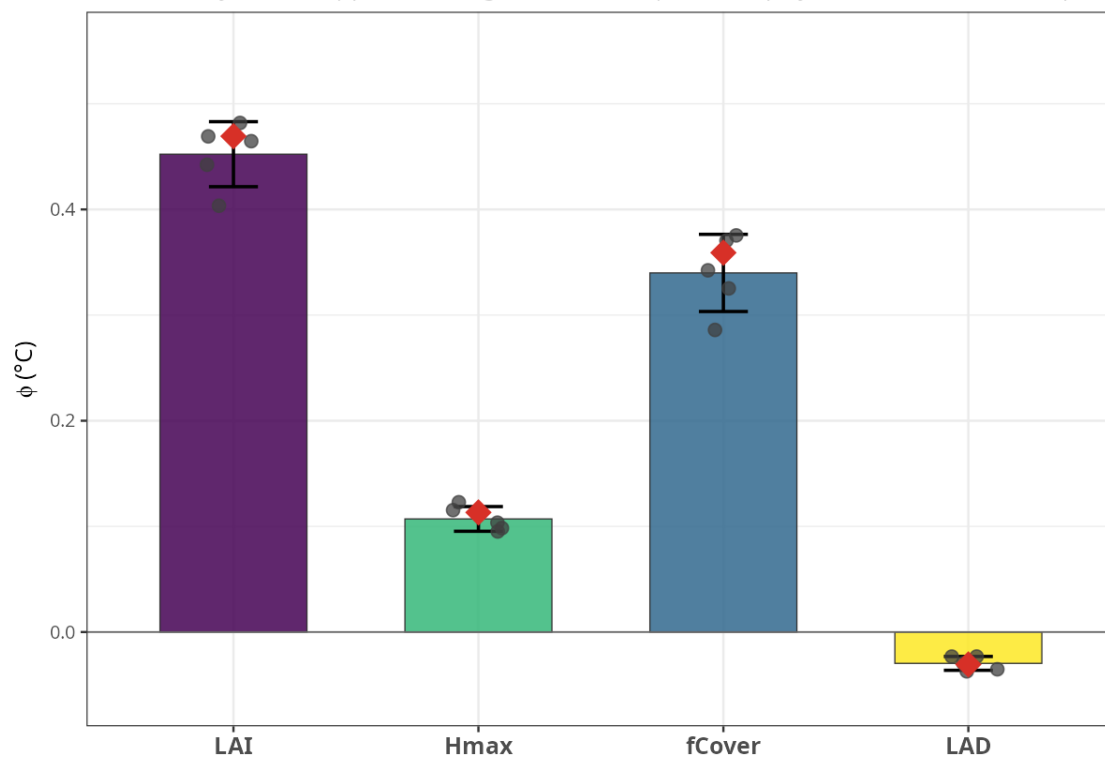


Figure 32: Fig. A18 — Stabilité bootstrap Shapley (5 répliques, $n/2$).

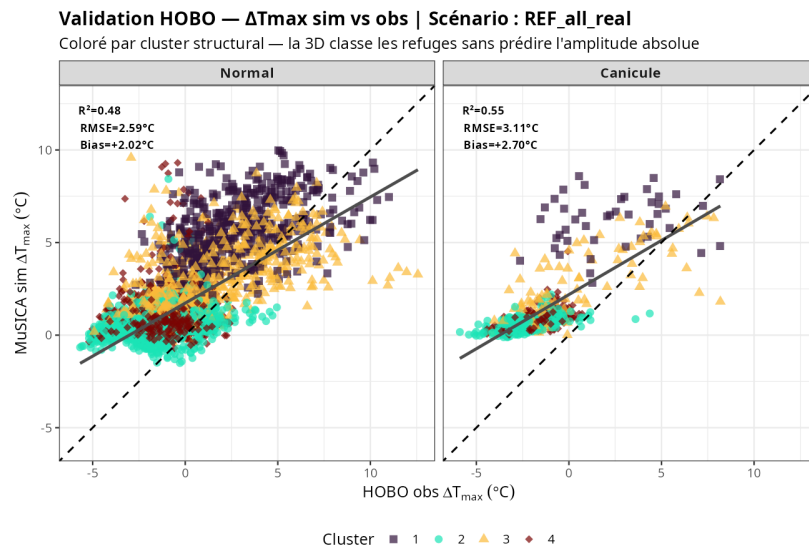


Figure 33: Fig. A19 — HOBO sim vs obs, coloré par cluster structurel.

```
library(vip)
library(Metrics)
library(tidyterra)
})

theme_set(
  theme_bw() +
  theme(
    text = element_text(size = 12),
    plot.title = element_text(face = "bold", size = 14),
    legend.position = "bottom",
    strip.background = element_rect(fill = "grey95"),
    strip.text = element_text(face = "bold")
  )
)

set.seed(42)

source("R/config.R")
source("R/io.R")
source("R/forest.R")
source("R/lad.R")
source("R/scenarios.R")
source("R/musica.R")
source("R/h2_analysis.R")
source("R/h1_analysis.R")
source("R/fpca.R")
source("R/gamm.R")
source("R/validation.R")
source("R/sentinel.R")
source("R/profiles.R")

# =====
# main() — orchestrator
# =====

main <- function() {
  FIG <- CFG$out_figures
```

```

FIG_ANN <- file.path(CFG$out_figures, "annex")

cat("[0/7] Setup output directories...\n")
setup_outputs_dirs()

cat("[1/7] Loading LiDAR rasters...\n")
rasters <- load_lidar_rasters(CFG$in_dir, CFG$agg_factor)

cat("[2/7] Building forest dataframe...\n")
fd <- build_forest_dataframe(rasters$stack)
df_macro <- extract_macro_daily(CFG$forcing_file, CFG$date_seq)

# FPCA on the full forest (before clustering and cLHS) so that FPC1/FPC2
# stratify both the clusters and the sample.
cat("[2b] FPCA sur la forêt entière (avant clustering)...\n")
fpca_res <- compute_fpca(fd$mat_lad, fd$df$Hmax, fd$z_breaks, n_harm = CFG$n_fpc_total)
fpc_var <- fpca_res$varprop
n_fpc_sel <- select_n_fpc(fpc_var, min_marginal = CFG$fpc_min_marginal)
cat(sprintf("    FPC variance (forêt entière) : %s\n",
  paste(round(100 * fpc_var, 1), collapse = " / ")))
cat(sprintf("    FPCs retenus pour clustering/cLHS : %d (seuil gain >= %.0f%%)\n",
  n_fpc_sel, 100 * CFG$fpc_min_marginal))

writeLines(
  c(sprintf("n_fpc_total: %d | n_fpc_selected: %d (min_marginal: %.0f%%)",
    CFG$n_fpc_total, n_fpc_sel, 100 * CFG$fpc_min_marginal),
    sprintf("FPC%d: %.1f%% (cumul %.1f%%) %s",
      seq_along(fpc_var),
      100 * fpc_var,
      100 * cumsum(fpc_var),
      ifelse(seq_along(fpc_var) <= n_fpc_sel, "<-- retenu", ""))),
  "outputs/fpca/fpca_variance.txt"
)

save_plot(fpca_res$gcv_plot, file.path(FIG_ANN, "A01_fpca_gcv.png"), width = 8, height = 5)
print(fpca_res$gcv_plot)

p_fpca_harm <- plot_fpc_harmonics(fpca_res, fd$mat_lad, fd$z_breaks, fd$df$Hmax)
save_plot(p_fpca_harm, file.path(FIG, "09_fpca_harmonics.png"), width = 12, height = 6)
print(p_fpca_harm)

p_fpca_load <- plot_fpc_loadings(fpca_res)
save_plot(p_fpca_load, file.path(FIG_ANN, "A03_fpca_loadings.png"), width = 10, height = 5)
print(p_fpca_load)

fpc_col_names <- paste0("FPC", seq_len(n_fpc_sel))
fpc_mat <- fpca_res$fpca$scores[, seq_len(n_fpc_sel), drop = FALSE]
colnames(fpc_mat) <- fpc_col_names
df_forest_raw <- fd$df %>% bind_cols(as.data.frame(fpc_mat))

cluster_vars <- c("LAI", "Hmax", "fCover", fpc_col_names)
cl_res <- label_clusters(df_forest_raw, k = CFG$k_clusters, vars = cluster_vars)
df_forest <- cl_res$df
if (!is.null(cl_res$elbow_plot)) {
  save_plot(cl_res$elbow_plot, file.path(FIG_ANN, "A04_kmeans_elbow.png"), width = 8, height = 5)
  print(cl_res$elbow_plot)
}

p_fpca_scatter <- plot_fpc_scatters(fpca_res, cluster_vec = df_forest$Cluster)
save_plot(p_fpca_scatter, file.path(FIG_ANN, "A02_fpca_scatters.png"), width = 12, height = 5)

```

```

print(p_fpca_scatter)

p_fpca_recon <- plot_fpca_reconstruction(
  fpca_res,
  n_sites      = 6,
  cluster_vec  = as.character(df_forest$Cluster),
  out_path     = file.path(FIG, "10_fpca_reconstruction.png")
)
print(p_fpca_recon)

p_clust_prof <- plot_cluster_mean_profiles(df_forest, fpca_res$mat_rel, fpca_res$z_rel)
save_plot(p_clust_prof, file.path(FIG, "08_clusters_lad_profiles.png"), width = 12, height = 12)
print(p_clust_prof)

# ----- 3. cLHS sampling -----
if (FLAGS$RUN_CLHS) {
  cat("[3/7] cLHS sampling...\n")
  df_sample <- sample_clhs_per_cluster(df_forest, CFG$n_per_cluster,
                                       vars = cluster_vars)
  saveRDS(df_sample, file.path(CFG$out_dir, "clhs_sample.rds"))
} else {
  df_sample <- readRDS(file.path(CFG$out_dir, "clhs_sample.rds"))
  if ("Archetype" %in% names(df_sample) && !"Cluster" %in% names(df_sample)) {
    df_sample <- df_sample %>% rename(Cluster = Archetype)
  }
}
cat(sprintf("    sample size: %d plots\n", nrow(df_sample)))

idx_sample <- match(paste(df_sample$x, df_sample$y),
                   paste(df_forest$x, df_forest$y))
fpc_scores_sample <- fpca_res$fpca$scores[idx_sample, seq_len(CFG$n_fpc_total), drop = FALSE]

ref_sc <- scenario_reference(df_sample)

# ----- 4. H2 - uniform vs real LAD -----
if (FLAGS$RUN_H2_UNIFORM_VS_REAL) {
  cat("[4/7] H2: uniform vs real LAD...\n")
  h2_scs <- scenarios_h2_uniform_vs_real(df_sample)
  h2_paths <- run_scenarios(df_sample, h2_scs, CFG$out_h2,
                           skip_existing = FLAGS$SKIP_EXISTING_SCENARIOS)
  df_h2 <- extract_all_scenarios(h2_paths, df_macro, CFG$date_seq)
  df_h2_w <- summarise_h2(df_h2)

  p_h2_dist <- plot_h2_distributions(df_h2)
  save_plot(p_h2_dist, file.path(FIG_ANN, "A06_h2_distributions.png"))
  print(p_h2_dist)

  p_h2_pair <- plot_h2_paired(df_h2_w)
  save_plot(p_h2_pair, file.path(FIG, "05_h2_paired_real_vs_uniform.png"))
  print(p_h2_pair)

  cat(sprintf("    mean per-plot diff (real - uniform): %.3f °C\n", mean(df_h2_w$diff, na.rm = TRUE)))
  p_h2_hw <- analyse_h2_distribution(df_h2_w, df_macro, CFG$heatwave_thr)
  save_plot(p_h2_hw, file.path(FIG_ANN, "A07_h2_heatwave.png"))

  cat("[H2 seasonal] Stabilité saisonnière de l'effet profil LAD...\n")
  plot_h2_seasonal(df_h2_w, df_macro, threshold = CFG$heatwave_thr,
                  out_path = file.path(FIG_ANN, "A17_h2_seasonal.png"))

  if (FLAGS$RUN_H2_STRUCTURE_ANALYSIS) {

```

```

cat("[audit] Analyse structure de la diff H2...\n")
fpc_sc_h2 <- if (exists("fpca_res") && !is.null(fpca_res))
  fpca_res$fpca$scores[, 1:3] else NULL
hmed_h2 <- if (exists("h_median_vec") && !is.null(h_median_vec))
  h_median_vec else NULL
analyse_h2_structure(
  df_h2_w, df_sample, df_macro,
  h_median_vec = hmed_h2,
  fpc_scores = fpc_sc_h2,
  out_dir = "outputs/audit"
)
}

if (FLAGS$RUN_VERTICAL_PROFILES) {
cat("[vertical] Profils verticaux Tmax par cluster médian...\n")
plot_vertical_tmax_profiles(
  df_sample,
  real_lad_dir = file.path(CFG$out_h2, "H2_real_LAD"),
  uniform_lad_dir = file.path(CFG$out_h2, "H2_uniform_LAD"),
  date_seq = CFG$date_seq,
  out_dir = "outputs/h2",
  summary_mode = "mean_summer"
)
plot_vertical_tmax_profiles(
  df_sample,
  real_lad_dir = file.path(CFG$out_h2, "H2_real_LAD"),
  uniform_lad_dir = file.path(CFG$out_h2, "H2_uniform_LAD"),
  date_seq = CFG$date_seq,
  out_dir = "outputs/h2",
  summary_mode = "canicule_date"
)
plot_vertical_tmax_profiles(
  df_sample,
  real_lad_dir = file.path(CFG$out_h2, "H2_real_LAD"),
  uniform_lad_dir = file.path(CFG$out_h2, "H2_uniform_LAD"),
  date_seq = CFG$date_seq,
  out_dir = "outputs/h2",
  summary_mode = "median_date"
)

cat("[vertical] Profils LAD par cluster médian (tous clusters)...\n")
medians_all <- select_cluster_median_plots(df_sample)
if (!is.null(medians_all) && nrow(medians_all) > 0) {
  for (cl in unique(medians_all$Cluster)) {
    cl_median <- medians_all %>% filter(Cluster == cl)
    if (nrow(cl_median) == 1) {
      cl_full <- df_sample %>%
        filter(abs(x - cl_median$x) < 1 & abs(y - cl_median$y) < 1) %>%
        slice(1)
      if (nrow(cl_full) == 1) {
        plot_lad_profile_for_plot(
          cl_full,
          sprintf(file.path(FIG_ANN, "A08_c%s_median_lad_profile.png"), cl)
        )
      }
    }
  }
}

cat("[vertical] Diagnostic pic Cluster 1 (verify_cluster1_peak)...\n")

```

```

verify_cluster1_peak(
  df_sample,
  real_lad_dir = file.path(CFG$out_h2, "H2_real_LAD"),
  uniform_lad_dir = file.path(CFG$out_h2, "H2_uniform_LAD"),
  date_seq = CFG$date_seq,
  out_dir = "outputs/h2"
)

cat("[vertical] ΔTmax vertical par cluster (indépendant vs heure globale)...\n")
plot_vertical_delta_by_cluster(
  df_sample,
  real_lad_dir = file.path(CFG$out_h2, "H2_real_LAD"),
  df_macro = df_macro,
  date_seq = CFG$date_seq,
  out_path = file.path(FIG, "06_h2_vertical_delta_by_cluster.png")
)

cat("[vertical] Profils verticaux 3 scénarios H2 (canicule)...\n")
plot_vertical_tmax_profiles_multi(
  scenarios_dict = list(
    "Real LAD" = file.path(CFG$out_h2, "H2_real_LAD"),
    "Cluster-type LAD" = file.path(CFG$out_h2, "H2_cluster_type_LAD"),
    "Uniform LAD" = file.path(CFG$out_h2, "H2_uniform_LAD")
  ),
  df_sample = df_sample,
  date_seq = CFG$date_seq,
  summary_mode = "canicule_date",
  diff_from = "Uniform LAD",
  out_path = file.path(FIG, "07_h2_vertical_3way_canicule.png")
)
}
}

# ----- 4b. H2 – cluster-type LAD vs real LAD -----
if (FLAGS$RUN_H2_CLUSTER_TYPE) {
  cat("[4b/7] H2: cluster-type LAD vs real LAD...\n")
  h2_ct_scs <- scenarios_h2_cluster_type(df_sample)
  h2_ct_paths <- run_scenarios(df_sample, h2_ct_scs, CFG$out_h2,
                              skip_existing = FLAGS$SKIP_EXISTING_SCENARIOS)
  df_h2_ct <- extract_all_scenarios(h2_ct_paths, df_macro, CFG$date_seq)
  df_h2_ct_w <- df_h2_ct %>%
    pivot_wider(id_cols = c(x, y, date),
                 names_from = scenario,
                 values_from = Delta_Tmax) %>%
    rename_with(~ "Real", matches("real_LAD")) %>%
    rename_with(~ "ClusterType", matches("cluster_type_LAD")) %>%
    mutate(diff = Real - ClusterType)

  cat(sprintf("    mean per-plot diff (real - cluster-type): %.3f °C\n",
              mean(df_h2_ct_w$diff, na.rm = TRUE)))
  cat(sprintf("    sd diff: %.3f °C\n",
              sd(df_h2_ct_w$diff, na.rm = TRUE)))

  p_h2_ct_pair <- plot_h2_paired(df_h2_ct_w, col_x = "Real", col_y = "ClusterType",
                                title = "H2 – Per-plot, per-day ΔTmax: real vs cluster-type",
                                subtitle = "Paired comparison: impact of LAD shape typology")
  save_plot(p_h2_ct_pair, file.path(FIG_ANN, "A09_h2_ct_cluster_vs_real.png"))
  print(p_h2_ct_pair)

  summary_ct <- df_h2_ct_w %>%

```

```

left_join(df_sample %>% select(x, y, Cluster), by = c("x", "y")) %>%
group_by(Cluster) %>%
summarise(
  mean_diff = mean(diff, na.rm = TRUE),
  sd_diff = sd(diff, na.rm = TRUE),
  n = n(),
  .groups = "drop"
)
cat("[4b] Diff (real - cluster-type) par cluster :\n")
print(summary_ct)
write.csv(summary_ct,
  "outputs/h2/h2_ct_summary_by_cluster.csv",
  row.names = FALSE)
}

# ---- 5. H1 - forward / L00 / factorial -----
scenario_paths <- list()
ref_path <- file.path(CFG$out_h1_forward, ref_sc$name)
run_musica_scenario(df_sample, ref_sc, ref_path,
  skip_existing = FLAGS$SKIP_EXISTING_SCENARIOS)
scenario_paths[[ref_sc$name]] <- ref_path

if (FLAGS$RUN_H1_FORWARD) {
  cat("[5a/7] H1 forward inclusion...\n")
  fw_scs <- scenarios_h1_forward(df_sample)

  sc0_params <- data.frame(
    variable = c("LAI_mean", "Hmax_mean", "fCover_absent", "LAD_absent"),
    valeur = c(mean(df_sample$LAI, na.rm = TRUE),
      round(mean(floor(df_sample$Hmax) + 1, na.rm = TRUE)),
      1,
      NA_real_),
    note = c("moyenne échantillon cLHS",
      "moyenne échantillon cLHS",
      "canopée fermée homogène (pas de clumping)",
      "profil uniforme 1 m → Hmax, densité = LAI_plot / depth")
  )
  write.csv(sc0_params, "outputs/h1/h1_sc0_baseline_params.csv", row.names = FALSE)
  cat(sprintf(" [SC0] LAI_mean=%.3f Hmax_mean=%.2f m fCover=1 LAD=uniforme\n",
    sc0_params$valeur[1], sc0_params$valeur[2]))
  scenario_paths <- c(scenario_paths,
    run_scenarios(df_sample, fw_scs, CFG$out_h1_forward,
      skip_existing = FLAGS$SKIP_EXISTING_SCENARIOS))
}
if (FLAGS$RUN_H1_L00) {
  cat("[5b/7] H1 leave-one-out...\n")
  loo_scs <- scenarios_h1_loo(df_sample)
  scenario_paths <- c(scenario_paths,
    run_scenarios(df_sample, loo_scs, CFG$out_h1_loo,
      skip_existing = FLAGS$SKIP_EXISTING_SCENARIOS))
}
if (FLAGS$RUN_H1_FACTORIAL) {
  cat("[5c/7] H1 full factorial (HEAVY)...\n")
  fac_scs <- scenarios_h1_factorial(df_sample)

  factorial_preload_map <- c(
    H1f_m_m_a_u = file.path(CFG$out_h1_forward, "H1f_0_Null_baseline"),
    H1f_r_m_a_u = file.path(CFG$out_h1_forward, "H1f_1_LAI_only"),
    H1f_r_r_a_u = file.path(CFG$out_h1_forward, "H1f_2_LAI_Hmax"),
    H1f_r_r_r_u = file.path(CFG$out_h1_forward, "H1f_3_LAI_Hmax_fCover"),

```

```

H1F_r_r_r_r = file.path(CFG$out_h1_forward, "H1f_4_Full_real"),
H1F_m_r_r_r = file.path(CFG$out_h1_loo,      "H1l_dropLAI_mean"),
H1F_r_m_r_r = file.path(CFG$out_h1_loo,      "H1l_dropHmax_mean"),
H1F_r_r_a_r = file.path(CFG$out_h1_loo,      "H1l_dropfCover_mean")
)
preload_factorial_from_existing(factorial_preload_map, CFG$out_h1_factorial)

scenario_paths <- c(scenario_paths,
  run_scenarios(df_sample, fac_scs, CFG$out_h1_factorial,
    skip_existing = FLAGS$SKIP_EXISTING_SCENARIOS))
}

df_all_scenarios <- extract_all_scenarios(scenario_paths, df_macro, CFG$date_seq)
df_scores <- score_scenarios_vs_reference(df_all_scenarios, ref_sc$name)
print(df_scores)
write.csv(df_scores, "outputs/h1/h1_scores.csv", row.names = FALSE)

# REF reconciliation: REF_all_real and H2_real_LAD share the same full-LiDAR
# inputs – any divergence flags a reproducibility issue in the pipeline.
if (FLAGS$RUN_H2_UNIFORM_VS_REAL && exists("df_h2")) {
  cat("[audit] REF reconciliation: REF_all_real ↔ H2_real_LAD...\n")
  df_ref_rec <- df_all_scenarios %>%
    filter(scenario == ref_sc$name) %>%
    dplyr::select(x, y, date, Delta_REF = Delta_Tmax)
  df_h2_real_rec <- df_h2 %>%
    filter(scenario == "H2_real_LAD") %>%
    dplyr::select(x, y, date, Delta_H2 = Delta_Tmax)
  df_rec <- inner_join(df_ref_rec, df_h2_real_rec, by = c("x", "y", "date"))

  if (nrow(df_rec) > 0) {
    r2_rec <- cor(df_rec$Delta_REF, df_rec$Delta_H2, use = "complete.obs")^2
    bias_rec <- mean(df_rec$Delta_H2 - df_rec$Delta_REF, na.rm = TRUE)
    rmse_rec <- sqrt(mean((df_rec$Delta_H2 - df_rec$Delta_REF)^2, na.rm = TRUE))
    cat(sprintf(" [REF recon] R²=%.4f bias=%.4f°C RMSE=%.4f°C\n",
      r2_rec, bias_rec, rmse_rec))

    lims_rec <- range(c(df_rec$Delta_REF, df_rec$Delta_H2), na.rm = TRUE)
    p_rec <- ggplot(df_rec, aes(x = Delta_REF, y = Delta_H2)) +
      geom_hex(bins = 80) +
      scale_fill_viridis_c(trans = "log10", name = "Count\n(log)") +
      geom_abline(slope = 1, intercept = 0, linetype = "dashed",
        colour = "black", linewidth = 0.8) +
      coord_fixed(xlim = lims_rec, ylim = lims_rec) +
      annotate("text", x = -Inf, y = Inf,
        label = sprintf("R²=%.4f\nBias=%.4f°C\nRMSE=%.4f°C",
          r2_rec, bias_rec, rmse_rec),
        hjust = -0.1, vjust = 1.2, fontface = "bold", size = 4) +
      labs(
        title = "Contrôle cohérence : REF_all_real ↔ H2_real_LAD",
        subtitle = "Mêmes paramètres LiDAR – écart attendu ≈ 0",
        x = expression("REF_all_real" ~ Delta * T[max] ~ (degree*C)),
        y = expression("H2_real_LAD" ~ Delta * T[max] ~ (degree*C))
      ) +
      theme_bw(base_size = 12) +
      theme(plot.title = element_text(face = "bold"))
    save_plot(p_rec, file.path(FIG_ANN, "A16_ref_h2_reconciliation.png"),
      width = 7, height = 6)
    print(p_rec)
  }
}

```

```

p_h1_hier <- plot_scenario_hierarchy(df_scores, ref_sc$name)
save_plot(p_h1_hier, file.path(FIG_ANN, "A05_h1_hierarchy.png"))
print(p_h1_hier)

if (FLAGS$RUN_H1_FORWARD) {
  forward_order <- vapply(scenarios_h1_forward(df_sample), `[`, "", "name")
  p_h1_fwd <- plot_forward_curve(df_scores, forward_order)
  save_plot(p_h1_fwd, file.path(FIG, "01_h1_forward_curve.png"))
  print(p_h1_fwd)

  forward_sc_names <- setdiff(forward_order, "H1f_0_Null_baseline")
  p_h1_sc <- plot_h1_scenario_scatters(
    df_all_scenarios,
    ref_scenario_name = ref_sc$name,
    forward_names     = forward_sc_names,
    out_path          = file.path(FIG, "04_h1_scatters_vs_ref.png")
  )
  print(p_h1_sc)

  p_h1_rank <- compare_h1_rankings(
    df_scores,
    forward_order = forward_order,
    out_path      = file.path(FIG, "02_h1_ranking.png")
  )
  print(p_h1_rank)
}

if (FLAGS$RUN_H1_LOO) {
  p_loo_2x2 <- plot_loo_scatters_2x2(
    df_all_scenarios,
    ref_scenario_name = ref_sc$name,
    out_path          = file.path(FIG, "03_h1_loo_scatters_2x2.png")
  )
  print(p_loo_2x2)
}

# ---- 5d. Shapley attribution (requires full factorial) -----
if (FLAGS$RUN_H1_FACTORIAL) {
  cat("[5d/7] Shapley exact attribution on 2^4 lattice...\n")
  shap_coalitions <- extract_shapley_coalitions(df_scores)
  shap_res <- compute_shapley_exact(shap_coalitions)
  plot_shapley_attribution(
    shap_res, df_scores,
    out_path = file.path(FIG, "02_shapley_attribution.png")
  )

  if (FLAGS$RUN_SHAPLEY_BOOTSTRAP) {
    cat("[5e/7] Shapley bootstrap stability...\n")
    df_boot_shap <- bootstrap_shapley(
      df_all_scenarios = df_all_scenarios,
      df_sample        = df_sample,
      ref_scenario_name = ref_sc$name,
      n_boot           = 5,
      n_per_boot       = ceiling(nrow(df_sample) / 2)
    )
    if (nrow(df_boot_shap) > 0) {
      write.csv(df_boot_shap,
        "outputs/h1/shapley_bootstrap.csv", row.names = FALSE)
      plot_shapley_bootstrap(
        df_boot_shap, shap_exact = shap_res,

```

```

    out_path = file.path(FIG_ANN, "A18_shapley_bootstrap.png")
  )
}
}

# ----- 6. GAMM emulator -----
if (FLAGS$RUN_GAMM_EMULATOR) {
  cat("[6/7] GAMM emulator on reference scenario...\n")
  df_ref_daily <- df_all_scenarios %>% filter(scenario == ref_sc$name) %>% join_with_sample()

  fpc_scores <- if (FLAGS$GAMM_SHAPE_VAR == "FPC") fpc_scores_sample else NULL
  h_median_vec <- if (FLAGS$GAMM_SHAPE_VAR == "H_MEDIAN") {
    mat_s <- df_sample %>% dplyr::select(starts_with("LAD_Layer_")) %>% as.matrix()
    mat_s[is.na(mat_s)] <- 0
    compute_h_median(mat_s, as.numeric(gsub("LAD_Layer_", "", colnames(mat_s))))
  } else NULL

  df_gamm <- prepare_gamm_data(df_ref_daily, df_sample,
                              fpc_scores = fpc_scores, h_median_vec = h_median_vec)

  gam_ref <- fit_reference_gamm(df_gamm,
                                shape_type = FLAGS$GAMM_SHAPE_VAR,
                                use_macroclimate = FLAGS$USE_MACROCLIMATE_GAMM)
  capture.output(summary(gam_ref), file = "outputs/gamm/gamm_summary.txt")
  print(summary(gam_ref))

  p_gamm_eff <- plot_gamm_marginal_effects(gam_ref,
                                           shape_type = FLAGS$GAMM_SHAPE_VAR,
                                           use_macroclimate = FLAGS$USE_MACROCLIMATE_GAMM)
  save_plot(p_gamm_eff, file.path(FIG_ANN, "A10_gamm_effects.png"), width = 12, height = 8)
  print(p_gamm_eff)

  png(file.path(FIG_ANN, "A11_gamm_residuals.png"), width = 1200, height = 900, res = 120)
  diag_res <- diagnose_residuals(gam_ref, df_gamm)
  dev.off()
  print(diag_res)
  write.csv(diag_res, "outputs/gamm/gamm_residuals_stats.csv", row.names = FALSE)

  conc_res <- run_concurvity_audit(gam_ref, out_dir = "outputs/audit")
}

# ----- OPTIONAL: HOB0 validation -----
if (FLAGS$RUN_HOB0_VALIDATION) {
  cat("[HOB0] Running validation of forward scenarios at HOB0 locations...\n")
  df_hobo_daily <- read_hobo_daily(CFG$hobo_temp_csv, CFG$date_seq, df_macro, CFG$ids_to_remove)
  df_hobo_inputs <- build_hobo_inputs(CFG$hobo_geojson, rasters$stack, CFG$ids_to_remove)
  cat(sprintf("    %d HOB0 plots retained\n", nrow(df_hobo_inputs)))

  hobo_scs <- if (FLAGS$RUN_H1_FORWARD) c(list(REF = ref_sc), scenarios_h1_forward(df_sample))

  hobo_res <- validate_scenarios_at_hobos(df_hobo_inputs, df_hobo_daily, hobo_scs,
                                           parent_dir = CFG$out_hobo,
                                           df_macro = df_macro,
                                           date_seq = CFG$date_seq)

  print(hobo_res$metrics)
  write.csv(hobo_res$metrics, "outputs/hobo/hobo_metrics.csv", row.names = FALSE)

  p_hobo_val <- plot_hobo_validation(hobo_res$metrics)
  save_plot(p_hobo_val, file.path(FIG_ANN, "A12_hobo_validation.png"))
}

```

```

print(p_hobo_val)

fw_names <- vapply(scenarios_h1_forward(df_sample), `[`, "", "name")
rep_hobos <- pick_representative_hobos(df_hobo_daily)

df_real <- hobo_res$daily %>% filter(scenario == fw_names["Full_real"]) %>% mutate(Tmax_
df_unif <- hobo_res$daily %>% filter(scenario == fw_names["LAI_Hmax_fCover"]) %>% mutate

p_ts <- plot_timeseries_faceted(rep_hobos, df_hobo_daily, df_real, df_unif, df_macro)
save_plot(p_ts, file.path(FIG_ANN, "A13_hobo_timeseries.png"), width = 12, height = 10)
print(p_ts)

cat("[H0B0] Scatter sim vs obs coloré par cluster structurel...\n")
plot_hobo_sim_obs_by_cluster(
  hobo_res      = hobo_res,
  df_hobo_inputs = df_hobo_inputs,
  df_forest     = df_forest,
  scenario_name  = ref_sc$name,
  threshold     = CFG$heatwave_thr,
  out_path      = file.path(FIG_ANN, "A19_hobo_cluster_scatter.png")
)
}

# ----- OPTIONAL: Sentinel-2 + FORMS-H annex -----
if (FLAGS$RUN_S2_ANNEX) {
  cat("[annex] Running S2 + FORMS-H naive scenario...\n")
  annex <- make_s2_formsh_scenario(df_sample, CFG$in_dir, CFG$agg_factor)
  p_annex <- run_musica_scenario(annex$df_sample, annex$scenario, CFG$out_s2_annex)
  df_annex <- extract_deltatmax_scenario(p_annex, df_macro, CFG$date_seq)

  cat("    [annex] Computing metrics S2 vs Full LiDAR...\n")
  df_ref_compare <- df_all_scenarios %>%
    filter(scenario == ref_sc$name) %>%
    dplyr::select(x, y, date, Delta_ref = Delta_Tmax)
  df_s2_matched <- df_annex %>% inner_join(df_ref_compare, by = c("x", "y", "date"))

  s2_r2 <- cor(df_s2_matched$Delta_Tmax, df_s2_matched$Delta_ref, use = "complete.obs")^2
  s2_rmse <- sqrt(mean((df_s2_matched$Delta_Tmax - df_s2_matched$Delta_ref)^2, na.rm = TRUE))
  s2_bias <- mean(df_s2_matched$Delta_Tmax - df_s2_matched$Delta_ref, na.rm = TRUE)

  stats_label_s2 <- sprintf("R² = %.2f\nRMSE = %.2f°C\nBias = %.2f°C", s2_r2, s2_rmse, s2_bias)

  df_s2_eval <- bind_rows(df_annex, df_all_scenarios %>% filter(scenario == ref_sc$name))
  df_plot_s2 <- df_s2_eval %>%
    mutate(scenario_label = ifelse(scenario == ref_sc$name,
                                   "LiDAR (Full 3D Real)",
                                   "Sentinel-2 + FORMS-H (2D Proxy)"))

  p_s2_density <- ggplot(df_plot_s2,
    aes(x = Delta_Tmax, fill = scenario_label, colour = scenario_label))
  geom_density(alpha = 0.3, linewidth = 0.8) +
  scale_fill_manual(values = c("LiDAR (Full 3D Real)" = "#31688e",
                              "Sentinel-2 + FORMS-H (2D Proxy)" = "#fca50a")) +
  scale_colour_manual(values = c("LiDAR (Full 3D Real)" = "#31688e",
                                 "Sentinel-2 + FORMS-H (2D Proxy)" = "#fca50a")) +
  labs(title = "Annex H3 – Optical Proxy vs Structural Reality (Density)",
    subtitle = sprintf("R² = %.2f | RMSE = %.2f°C | Bias = %.2f°C", s2_r2, s2_rmse, s2_bias),
    x = expression(Delta * T[max] ~ (degree*C)),
    y = "Density",
    fill = "Input Source",

```

```

    colour = "Input Source") +
    theme(legend.position = "bottom")

save_plot(p_s2_density, file.path(FIG_ANN, "A14_s2_density.png"))
print(p_s2_density)

axis_lims_s2 <- range(c(df_s2_matched$Delta_ref, df_s2_matched$Delta_Tmax), na.rm = TRUE)

p_s2_paired <- ggplot(df_s2_matched, aes(x = Delta_ref, y = Delta_Tmax)) +
  geom_hex(bins = 100) +
  scale_fill_gradient(low = "grey80", high = "midnightblue",
    trans = "log10", name = "Count\n(log)") +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed",
    colour = "black", linewidth = 0.8) +
  geom_smooth(method = "lm", colour = "#fca50a", se = FALSE, linewidth = 1.2) +
  coord_fixed(xlim = axis_lims_s2, ylim = axis_lims_s2) +
  annotate("text", x = -Inf, y = Inf, label = stats_label_s2,
    hjust = -0.1, vjust = 1.2, fontface = "bold", size = 5, colour = "black") +
  labs(title = "Annex H3 – Per-plot, per-day ΔTmax: S2 Proxy vs LiDAR",
    subtitle = "Paired comparison: impact of 2D optical proxy on spatial prediction",
    x = expression("LiDAR (Full 3D Real)" ~ Delta * T[max] ~ (degree*C)),
    y = expression("Sentinel-2 + FORMS-H" ~ Delta * T[max] ~ (degree*C))) +
  theme_bw(base_size = 14) +
  theme(legend.position = "right", plot.title = element_text(face = "bold"))

save_plot(p_s2_paired, file.path(FIG_ANN, "A15_s2_paired.png"))
print(p_s2_paired)

writelines(
  sprintf("R2: %.4f\nRMSE: %.4f deg C\nBias: %.4f deg C", s2_r2, s2_rmse, s2_bias),
  "outputs/s2_annex/s2_metrics.txt"
)
}

cat("\n[done]\n")
invisible(NULL)
}

if (interactive() || identical(Sys.getenv("RUN_MAIN"), "1")) {
  main()
}

```

4.2 R/config.R — Configuration, chemins et flags

```

# =====
# Chapter 1 – Configuration : paths and run flags
# =====

# ---- Paths -----
CFG <- list(
  in_dir      = "in_files",
  out_dir     = "out_files/Sensitivity_Analysis",
  forcing_file = "in_files/musica_in_Blois.nc",
  hobo_geojson = "in_files/data_Blois_utm31n.geojson",
  hobo_temp_csv = "in_files/Blois_data_temperature.csv",
  musica_cmd   = normalizePath("in_files/model-3.2.3/musica", mustWork = FALSE),
  date_seq     = seq(as.Date("2021-06-01"), as.Date("2021-09-30"), by = "day"),
  ids_to_remove = c("41_13", "41_14", "41_20", "41_41", "41_50", "41_51", "41_53"),

```

```

agg_factor      = 2,          # 10 m → 20 m aggregation
n_per_cluster   = 100,       # cLHS sample size per cluster
k_clusters      = NULL,      # K-means cluster assignment per plot
n_fpc_total     = 6,         # FPCs calculés par compute_fpca (scree plot complet)
fpc_min_marginal = 0.10,     # seuil gain marginal : inclure FPC_k si varprop[k] >= seuil
heatwave_thr    = 30,        # °C threshold for heatwave subset
out_h2          = "out_files/H2_uniform_vs_real",
out_h1_forward  = "out_files/H1_forward",
out_h1_loo      = "out_files/H1_loo",
out_h1_factorial = "out_files/H1_factorial",
out_hobo        = "out_files/musica_hobo_validation",
out_s2_annex    = "out_files/Annex_S2_FORMSH",
out_figures     = "outputs/figures"
)

# ---- Run flags -----
FLAGS <- list(
  RUN_CLHS              = TRUE, # FALSE = réutilise clhs_sample.rds existant
  SKIP_EXISTING_SCENARIOS = TRUE, # TRUE = skip tout répertoire scénario déjà peuplé
  RUN_H2_UNIFORM_VS_REAL = TRUE, # H2 first pass
  RUN_H2_CLUSTER_TYPE    = TRUE, # H2 cluster-type LAD vs real LAD
  RUN_H1_FORWARD         = TRUE, # forward inclusion
  RUN_H1_LOO             = TRUE, # leave-one-out

  RUN_GAMM_EMULATOR    = TRUE, # Statistical emulator
  USE_MACROCLIMATE_GAMM = TRUE, # Include physical meteorology in GAMM
  GAMM_SHAPE_VAR        = "FPC", # Shape predictor: "FPC", "CLUSTER" (cat.), "H_MEDIAN"

  RUN_VERTICAL_PROFILES = TRUE, # profils verticaux Tmax par cluster médian (H2)
  RUN_H2_STRUCTURE_ANALYSIS = TRUE, # post-processing léger (< 30 s, pas de simulation)
  RUN_HOBO_VALIDATION    = TRUE, # MuSICA at HOB0 locations
  RUN_S2_ANNEX           = TRUE, # Sentinel-2 annex (H3)
  RUN_H1_FACTORIAL       = TRUE, # heavy, off by default
  RUN_SHAPLEY_BOOTSTRAP  = TRUE, # bootstrap Shapley stability (requires RUN_H1_FACTORIAL)
)

dir.create(CFG$out_dir, recursive = TRUE, showWarnings = FALSE)

```

4.3 R/scenarios.R — Définition des scénarios H1/H2/REF

```

# =====
# Chapter 1 – Scenario registry
#
# Each scenario is a named list:
#   $name      : character, used as directory name for outputs
#   $lai_fn     : function(plot_row) → numeric
#   $hmax_fn    : function(plot_row) → numeric
#   $fcover_fn  : function(plot_row) → numeric
#   $lad_fn     : function(plot_row, hmax, lai) → data.frame(height, density)
# =====

# ---- Helper closures -----

fn_real <- function(varname) { force(varname); function(plot_row) as.numeric(plot_row[[varname]]) }
fn_mean <- function(df_sample, varname) { m <- mean(df_sample[[varname]], na.rm = TRUE); function(plot_row) m }

# ---- Reference scenario (all inputs from LiDAR) -----

```

```

#' Build the full-real reference scenario (all inputs from LiDAR).
#'
#' @param df_sample cLHS sample dataframe.
#' @return Named scenario list.
scenario_reference <- function(df_sample) {
  list(name = "REF_all_real", lai_fn = fn_real("LAI"), hmax_fn = fn_real("Hmax"),
        fcover_fn = fn_real("fCover"), lad_fn = make_lad_real)
}

# ---- H2 scenarios (uniform vs real LAD) -----

#' H2 scenarios: real vs uniform LAD, all other inputs held real.
#'
#' @param df_sample cLHS sample dataframe.
#' @return Named list of two scenarios.
scenarios_h2_uniform_vs_real <- function(df_sample) {
  list(
    REF_real_LAD = list(name = "H2_real_LAD", lai_fn = fn_real("LAI"),
                        hmax_fn = fn_real("Hmax"), fcover_fn = fn_real("fCover"),
                        lad_fn = make_lad_real),
    UNIFORM_LAD = list(name = "H2_uniform_LAD", lai_fn = fn_real("LAI"),
                       hmax_fn = fn_real("Hmax"), fcover_fn = fn_real("fCover"),
                       lad_fn = make_lad_uniform)
  )
}

#' H2 scenarios: real vs cluster-type LAD.
#'
#' @param df_sample cLHS sample dataframe.
#' @return Named list of two scenarios.
scenarios_h2_cluster_type <- function(df_sample) {
  lad_cluster_fn <- make_lad_cluster_type_factory(df_sample)
  list(
    REF_real_LAD = list(
      name = "H2_real_LAD",
      lai_fn = fn_real("LAI"), hmax_fn = fn_real("Hmax"),
      fcover_fn = fn_real("fCover"), lad_fn = make_lad_real
    ),
    CLUSTER_TYPE_LAD = list(
      name = "H2_cluster_type_LAD",
      lai_fn = fn_real("LAI"), hmax_fn = fn_real("Hmax"),
      fcover_fn = fn_real("fCover"), lad_fn = lad_cluster_fn
    )
  )
}

# ---- H1 scenarios (forward inclusion) -----

#' H1 forward inclusion scenarios: SC0 → SC4 (null baseline to full-real).
#'
#' @param df_sample cLHS sample dataframe.
#' @return Named list of 5 scenarios.
scenarios_h1_forward <- function(df_sample) {
  lai_mean <- fn_mean(df_sample, "LAI")
  hmax_mean_val <- round(mean(floor(df_sample$Hmax) + 1, na.rm = TRUE))
  hmax_mean <- function(plot_row) hmax_mean_val
  fcover_absent <- function(plot_row) 1 # canopée fermée homogène, pas de clumping
  lad_uniform <- make_lad_uniform
  list(
    Null_baseline = list(name = "H1f_0_Null_baseline", lai_fn = lai_mean, hmax_fn = hmax_mean,

```

```

LAI_only      = list(name = "H1f_1_LAI_only",      lai_fn = fn_real("LAI"), hma
LAI_Hmax      = list(name = "H1f_2_LAI_Hmax",      lai_fn = fn_real("LAI"), hma
LAI_Hmax_fCover = list(name = "H1f_3_LAI_Hmax_fCover", lai_fn = fn_real("LAI"), hma
Full_real     = list(name = "H1f_4_Full_real",      lai_fn = fn_real("LAI"), hm
)
}

# ---- H1 scenarios (leave-one-out) -----

#' H1 leave-one-out scenarios: each variable degraded in turn.
#'
#' @param df_sample cLHS sample dataframe.
#' @return Named list of 5 scenarios.
scenarios_h1_loo <- function(df_sample) {
  lai_mean      <- fn_mean(df_sample, "LAI")
  hmax_mean_val <- round(mean(floor(df_sample$Hmax) + 1, na.rm = TRUE))
  hmax_mean     <- function(plot_row) hmax_mean_val
  fcover_absent <- function(plot_row) 1 # cohérent avec baseline forward
  lad_mean_fn   <- make_lad_mean_factory(df_sample)
  list(
    drop_LAI_mean      = list(name = "H1l_dropLAI_mean",      lai_fn = lai_mean,      hmax_
    drop_Hmax_mean     = list(name = "H1l_dropHmax_mean",     lai_fn = fn_real("LAI"), hmax_
    drop_fCover_mean   = list(name = "H1l_dropfCover_mean",   lai_fn = fn_real("LAI"), hmax_
    drop_LAD_uniform   = list(name = "H1l_dropLAD_uniform",   lai_fn = fn_real("LAI"), hmax_
    drop_LAD_meanShape = list(name = "H1l_dropLAD_meanShape", lai_fn = fn_real("LAI"), hmax_
  )
}

# ---- H1 full factorial (2^4 = 16 scenarios) -----

#' H1 full factorial scenarios (2^4 lattice for Shapley computation).
#'
#' Generates all 16 combinations of {real, mean/absent/uniform} for
#' {LAI, Hmax, fCover, LAD}.
#'
#' @param df_sample cLHS sample dataframe.
#' @return Named list of 16 scenarios.
scenarios_h1_factorial <- function(df_sample) {
  lai_mean      <- fn_mean(df_sample, "LAI")
  hmax_mean_val <- round(mean(floor(df_sample$Hmax) + 1, na.rm = TRUE))
  hmax_mean     <- function(plot_row) hmax_mean_val
  fcover_absent <- function(plot_row) 1 # cohérent avec baseline forward/L00
  grid <- expand.grid(LAI = c("real", "mean"), Hmax = c("real", "mean"),
    fCover = c("real", "absent"), LAD = c("real", "uniform"),
    stringsAsFactors = FALSE)
  scenarios <- vector("list", nrow(grid))
  for (i in seq_len(nrow(grid))) {
    g <- grid[i, ]
    nm <- sprintf("H1F_%s_%s_%s_%s",
      substr(g$LAI, 1, 1), substr(g$Hmax, 1, 1),
      substr(g$fCover, 1, 1), substr(g$LAD, 1, 1))
    scenarios[[i]] <- list(
      name      = nm,
      lai_fn    = if (g$LAI == "real") fn_real("LAI") else lai_mean,
      hmax_fn    = if (g$Hmax == "real") fn_real("Hmax") else hmax_mean,
      fcover_fn  = if (g$fCover == "real") fn_real("fCover") else fcover_absent,
      lad_fn     = if (g$LAD == "real") make_lad_real else make_lad_uniform,
      grid_row   = g
    )
  }
}

```

```

setNames(scenarios, vapply(scenarios, `[`, "", "name"))
}

```

4.4 R/lad.R — Constructeurs de profils LAD

```

# =====
# Chapter 1 – LAD profile constructors
# Chapter 3 – d_opt-truncated LAD constructor
# =====

# Internal helper: rescale a LAD profile to a new (hmax, LAI) target.
.lad_rescale <- function(z_real, d, hmax_ref, hmax_tgt, lai_tgt) {
  z_rel <- if (hmax_ref > 0) z_real / hmax_ref else z_real
  z_new <- z_rel * hmax_tgt
  z_grid <- seq(1L, ceiling(hmax_tgt))
  d_new <- approx(z_new, d, z_grid, rule = 2)$y
  d_new[is.na(d_new)] <- 0
  if (sum(d_new) > 1e-9) d_new <- d_new * (lai_tgt / sum(d_new))
  data.frame(height = z_grid, density = d_new)
}

#' Build a real LAD profile from a plot row (with optional Hmax/LAI override).
#'
#' @param plot_row Single-row dataframe with LAD_Layer_* columns and Hmax.
#' @param hmax      Optional Hmax override (m).
#' @param lai       Optional LAI override.
#' @return Dataframe with columns height (m) and density (m2 m-3).
make_lad_real <- function(plot_row, hmax = NULL, lai = NULL) {
  lad_cols <- grep("LAD_Layer_", names(plot_row), value = TRUE)
  z_real <- as.numeric(gsub("LAD_Layer_", "", lad_cols))
  d <- as.numeric(plot_row[lad_cols]); d[is.na(d)] <- 0
  hmax_real <- as.numeric(plot_row$Hmax)
  if (is.null(hmax)) hmax <- hmax_real
  if (is.null(lai)) lai <- sum(d)
  .lad_rescale(z_real, d, hmax_real, hmax, lai)
}

#' Build a uniform LAD profile (constant density from canopy_base to Hmax).
#'
#' @param plot_row Single-row dataframe with Hmax and LAI columns.
#' @param hmax      Optional Hmax override (m).
#' @param lai       Optional LAI override.
#' @param canopy_base Height of canopy base in metres (default 1).
#' @return Dataframe with columns height (m) and density (m2 m-3).
make_lad_uniform <- function(plot_row, hmax = NULL, lai = NULL, canopy_base = 1) {
  if (is.null(hmax)) hmax <- as.numeric(plot_row$Hmax)
  if (is.null(lai)) lai <- as.numeric(plot_row$LAI)
  max_layer <- ceiling(hmax)
  if (max_layer <= canopy_base) return(data.frame(height = 1L, density = 0))
  depth <- max_layer - canopy_base + 1L
  density <- lai / depth
  heights <- seq_len(max_layer)
  data.frame(height = heights, density = ifelse(heights >= canopy_base, density, 0))
}

#' Factory: build a mean-shape LAD constructor from the full sample.
#'
#' Returns a function with the same signature as make_lad_real/uniform that

```

```

#' produces the sample-mean profile rescaled to each plot's (Hmax, LAI).
#'
#' @param df_sample cLHS sample dataframe with LAD_Layer_* columns.
#' @return Function(plot_row, hmax, lai) → data.frame(height, density).
make_lad_mean_factory <- function(df_sample) {
  lad_cols <- grep("LAD_Layer_", names(df_sample), value = TRUE)
  z_real <- as.numeric(gsub("LAD_Layer_", "", lad_cols))
  mat <- as.matrix(df_sample[, lad_cols]); mat[is.na(mat)] <- 0
  mean_prof <- colMeans(mat, na.rm = TRUE)
  hmax_mean_ref <- mean(df_sample$Hmax, na.rm = TRUE)

  function(plot_row, hmax = NULL, lai = NULL) {
    if (is.null(hmax)) hmax <- as.numeric(plot_row$Hmax)
    if (is.null(lai)) lai <- as.numeric(plot_row$LAI)
    .lad_rescale(z_real, mean_prof, hmax_mean_ref, hmax, lai)
  }
}

#' Factory: build a cluster-type LAD constructor from the full sample.
#'
#' Returns a function that substitutes each plot's real shape with the mean
#' profile of its cluster, rescaled to its own (Hmax, LAI).
#'
#' @param df_sample cLHS sample with LAD_Layer_* and Cluster columns.
#' @return Function(plot_row, hmax, lai) → data.frame(height, density).
make_lad_cluster_type_factory <- function(df_sample) {
  lad_cols <- grep("LAD_Layer_", names(df_sample), value = TRUE)
  z_real <- as.numeric(gsub("LAD_Layer_", "", lad_cols))

  cluster_profiles <- df_sample %>%
    filter(!is.na(Cluster)) %>%
    split(.$Cluster) %>%
    purrr::map(function(df_sub) {
      mat <- as.matrix(df_sub[, lad_cols]); mat[is.na(mat)] <- 0
      list(
        profile = colMeans(mat, na.rm = TRUE),
        hmax_mean_ref = mean(df_sub$Hmax, na.rm = TRUE)
      )
    })

  function(plot_row, hmax = NULL, lai = NULL) {
    cl <- as.character(plot_row$Cluster)
    if (is.na(cl) || !(cl %in% names(cluster_profiles)))
      return(data.frame(height = z_real, density = rep(0, length(z_real))))
    cp <- cluster_profiles[[cl]]
    if (is.null(hmax)) hmax <- as.numeric(plot_row$Hmax)
    if (is.null(lai)) lai <- as.numeric(plot_row$LAI)
    .lad_rescale(z_real, cp$profile, cp$hmax_mean_ref, hmax, lai)
  }
}

# =====
# Chapter 3 – d_opt-truncated LAD constructor
# =====

#' Build a real LAD profile truncated to d_opt height layers from the canopy top.
#'
#' @title d_opt-truncated LAD profile
#' @description Implements the physical interpretation of d_opt: Sentinel-2
#' reflectance is sensitive only to the top `d_opt` metres of the canopy.

```

```

#' This function zeroes out all LAD layers below `Hmax - d_opt` metres,
#' retaining only the canopy layers that S2 actually sees, then rescales to
#' `lai_tgt` so the total LAI matches the chosen LAI source.
#'
#' Use this when pairing with `LAI_S2_DOPT` so that both the LAI amplitude
#' (S2-corrected for depth) and the LAD shape (depth-truncated) are physically
#' consistent with the effective canopy depth seen by S2.
#'
#' @param plot_row Single-row dataframe with `LAD_Layer_*` columns and `Hmax`.
#' @param hmax      Optional Hmax override (m).
#' @param lai       Optional LAI override (applied to truncated layers only).
#' @param d_opt     Effective canopy depth seen by S2 in metres (default 6).
#' @return Dataframe with columns `height` (m) and `density` (m2 m-3).
#' Layers below `Hmax - d_opt` have density = 0.
#' @seealso [make_lad_real()]
make_lad_dopt <- function(plot_row, hmax = NULL, lai = NULL, d_opt = 6) {
  lad_cols <- grep("LAD_Layer_", names(plot_row), value = TRUE)
  z_real <- as.numeric(gsub("LAD_Layer_", "", lad_cols))
  d <- as.numeric(plot_row[lad_cols]); d[is.na(d)] <- 0
  hmax_real <- as.numeric(plot_row$Hmax)
  if (is.null(hmax)) hmax <- hmax_real
  if (is.null(lai)) lai <- sum(d)

  # First rescale to target hmax/lai (same as make_lad_real)
  lad_full <- .lad_rescale(z_real, d, hmax_real, hmax, lai)

  # Then zero out layers below (Hmax - d_opt) – keep only top d_opt metres
  h_thresh <- hmax - d_opt
  lad_full$density[lad_full$height <= h_thresh] <- 0

  # Re-normalise so the remaining layers integrate to lai_tgt
  tot <- sum(lad_full$density)
  if (tot > 1e-9) lad_full$density <- lad_full$density * (lai / tot)

  lad_full
}

#' Factory: build a make_lad_dopt closure with a fixed d_opt value.
#'
#' @title d_opt LAD factory
#' @description Returns a function with the same signature as `make_lad_real()`
#' but using a fixed `d_opt` depth truncation. Useful for embedding d_opt into
#' a scenario list without repeating the argument.
#'
#' @param d_opt Effective canopy depth (m). Default 6 (Pareto Blois).
#' @return Function `f(plot_row, hmax, lai) → data.frame(height, density)`.
make_lad_dopt_factory <- function(d_opt = 6) {
  force(d_opt)
  function(plot_row, hmax = NULL, lai = NULL) {
    make_lad_dopt(plot_row, hmax = hmax, lai = lai, d_opt = d_opt)
  }
}

```

4.5 R/musica.R — Exécution MuSICA et post-traitement NetCDF

```

# =====
# Chapter 1 – MuSICA batch runner and post-processing (NetCDF → daily ΔTmax)
# =====

```

```

#' Copy outputs from existing scenarios (forward/L00) into the factorial folder.
#'
#' Avoids re-running simulations already available elsewhere.
#'
#' @param preload_map Named character vector: name = factorial target name,
#'   value = path to source directory (forward or L00).
#' @param out_dir      Factorial output directory (CFG$out_h1_factorial).
#' @return Character vector of factorial names that were successfully preloaded.
preload_factorial_from_existing <- function(preload_map, out_dir) {
  dir.create(out_dir, recursive = TRUE, showWarnings = FALSE)
  preloaded <- character(0)
  for (nm in names(preload_map)) {
    src <- preload_map[[nm]]
    dst <- file.path(out_dir, nm)
    if (dir.exists(dst)) {
      cat(sprintf(" [factorial preload] %s – déjà présent, skip\n", nm))
      preloaded <- c(preloaded, nm)
    } else if (dir.exists(src)) {
      ok <- file.copy(src, out_dir, recursive = TRUE)
      if (ok) {
        file.rename(file.path(out_dir, basename(src)), dst)
        cat(sprintf(" [factorial preload] %s ← copié depuis %s\n", nm, basename(src)))
        preloaded <- c(preloaded, nm)
      } else {
        cat(sprintf(" [factorial preload] ECHEC copie %s\n", nm))
      }
    } else {
      cat(sprintf(" [factorial preload] %s – source absente (%s), sera simulé\n",
        nm, basename(src)))
    }
  }
  preloaded
}

#' Run a single MuSICA simulation for one plot × one scenario.
#'
#' @param plot_row      Single-row dataframe with structural covariates.
#' @param scenario      Scenario list (from scenarios.R).
#' @param out_nc_file   Path for the output NetCDF.
#' @param forcing_file  Path to the ERA5 forcing NetCDF.
#' @param musica_cmd    Shell command to call MuSICA.
#' @return Invisible path to out_nc_file, or NULL on error.
run_musica_one <- function(plot_row, scenario, out_nc_file, forcing_file, musica_cmd,
                           musica_nml = "musica.nml",
                           musica_variables = "./variables.csv",
                           extra_setup = list()) {
  if (file.exists(out_nc_file)) return(invisible(out_nc_file))
  lai <- scenario$lai_fn(plot_row)
  hmax <- scenario$hmax_fn(plot_row)
  fcover <- scenario$fcover_fn(plot_row)
  if (is.na(lai) || is.na(hmax) || is.na(fcover)) return(invisible(NULL))
  allom <- scenario$lad_fn(plot_row, hmax = hmax, lai = lai)

  # Phenology: S2-driven if phenology_fn provided, otherwise parametric fallback.
  # phenology_fn may return NULL for a given plot (missing S2 data) – also falls back.
  phenology <- NULL
  if (!is.null(scenario$phenology_fn)) {
    phenology <- scenario$phenology_fn(plot_row)
  }
  if (is.null(phenology)) {

```

```

phenology <- calc_phenology(
  list.year = c(2021, 2022), nleafage = 1,
  budburst_date = 115, leaf_age_max_in = 0.56,
  relative_age_firstmax = 0.10, relative_age_lastmax = 0.75,
  LAI_max_per_cohort = lai
)
}

setup_param <- c(
  list("clumping_factor" = fcover,
       "forcing_filename" = forcing_file,
       "history_filename" = out_nc_file,
       "forcing_height" = hmax + 2),
  extra_setup
)

tryCatch({
  callmusica(
    musica.file = musica_nml,
    musica.variables = musica_variables,
    musica.param = list(setupctl = setup_param),
    leaf.param = list(musica_veg1 = list("phenology" = phenology, "allometry" = allometry,
                                         "leafallometryctl" = list("canopy_height_t",
                                                                    "leafallometryctl")),
    musica.cmd = musica_cmd, keep.tmp = FALSE, out.netcdf = TRUE, out.df = FALSE
  )
}, error = function(e) cat(sprintf("ERROR on %s: %s\n", basename(out_nc_file), e$message))
invisible(out_nc_file)
}

#' Run all plots for a single scenario, writing NetCDFs to out_dir.
#'
#' @param df_sample cLHS sample dataframe.
#' @param scenario Scenario list (from scenarios.R).
#' @param out_dir Output directory for this scenario's NetCDF files.
#' @param forcing_file ERA5 forcing file path (default CFG$forcing_file).
#' @param musica_cmd MuSICA shell command (default CFG$musica_cmd).
#' @param id_prefix Prefix for simulation IDs (default "Sim").
#' @param force Re-run even if output exists (default FALSE).
#' @param skip_existing Skip the whole directory if any NC files exist (default FALSE).
#' @return Invisible out_dir.
run_musica_scenario <- function(df_sample, scenario, out_dir, forcing_file = CFG$forcing_file) {
  dir.create(out_dir, recursive = TRUE, showWarnings = FALSE)
  existing <- list.files(out_dir, pattern = "\\nc$")
  if (!force && skip_existing && length(existing) > 0) {
    cat(sprintf(" [skip] %s : %d NC existants – répertoire conservé tel quel\n",
               scenario$name, length(existing)))
    return(invisible(out_dir))
  }
  if (!force && length(existing) >= nrow(df_sample)) return(invisible(out_dir))
  cat(sprintf("\n[scenario: %s] %d plots → %s (%d done)\n", scenario$name, nrow(df_sample),
             for (i in seq_len(nrow(df_sample))) {
               plot_row <- df_sample[i, ]; sim_id <- sprintf("%s_%03d_X%d_Y%d", id_prefix, i, round(plot_row$leaf_age_max_in, 2), round(plot_row$leaf_age_max_out, 2))
               out_file <- file.path(out_dir, paste0("musica_out_", sim_id, ".nc"))
               if (file.exists(out_file)) next
               run_musica_one(plot_row, scenario, out_file, forcing_file, musica_cmd)
             }
  invisible(out_dir)
}

#' Run a list of scenarios sequentially, returning their output paths.

```

```

#'
#' @param df_sample      cLHS sample dataframe.
#' @param scenarios      Named list of scenario lists.
#' @param parent_dir     Parent directory; each scenario gets a sub-directory.
#' @param force          Re-run existing outputs (default FALSE).
#' @param skip_existing  Skip populated scenario directories (default FALSE).
#' @return Named character vector of output directory paths.
run_scenarios <- function(df_sample, scenarios, parent_dir, force = FALSE,
                          skip_existing = getOption("skip_existing_scenarios", FALSE)) {
  paths <- character(length(scenarios)); names(paths) <- names(scenarios)
  for (nm in names(scenarios)) {
    out_d <- file.path(parent_dir, scenarios[[nm]]$name)
    run_musica_scenario(df_sample, scenarios[[nm]], out_d,
                        force = force, skip_existing = skip_existing)
    paths[nm] <- out_d
  }
  paths
}

# ---- Post-processing: NetCDF → daily ΔTmax -----

#' Extract daily ΔTmax from a single MuSICA NetCDF output file.
#'
#' @param nc_file  Path to a MuSICA NetCDF output.
#' @param df_macro Daily macroclimate (from extract_macro_daily).
#' @param date_seq Dates of interest.
#' @return Tibble with date, Tmax_micro, Delta_Tmax, or NULL on failure.
extract_deltatmax_one <- function(nc_file, df_macro, date_seq) {
  nc <- try(nc_open(nc_file), silent = TRUE); if (inherits(nc, "try-error")) return(NULL)
  raw <- try(get_variable(nc, "Tair_z"), silent = TRUE); nc_close(nc); if (inherits(raw, "try-error")) return(NULL)
  raw %>% filter(nair == 1) %>% mutate(Tair_sim = Tair_z - 273.15, time = time - hours(2), d
    filter(date %in% date_seq) %>% group_by(date) %>% summarise(Tmax_micro = max(Tair_sim, na.rm = TRUE))
    inner_join(df_macro, by = "date") %>% mutate(Delta_Tmax = Tmax_micro - Tmax_macro)
  }

#' Extract daily ΔTmax for all NetCDF files in a scenario directory.
#'
#' @param scenario_dir Directory containing MuSICA NetCDF outputs.
#' @param df_macro      Daily macroclimate dataframe.
#' @param date_seq      Dates of interest.
#' @return Stacked tibble with x, y, scenario columns added.
extract_deltatmax_scenario <- function(scenario_dir, df_macro, date_seq) {
  nc_files <- list.files(scenario_dir, pattern = "\\..nc$", full.names = TRUE); if (length(nc_files) == 0) return(NULL)
  map_df(nc_files, function(f) {
    res <- extract_deltatmax_one(f, df_macro, date_seq); if (is.null(res)) return(NULL)
    res %>% mutate(x = as.numeric(str_extract(basename(f), "(?<=X)\\d+")), y = as.numeric(str_extract(basename(f), "\\d+")))
  })
}

#' Extract daily ΔTmax across all scenarios.
#'
#' @param scenario_paths Named character vector of scenario directory paths.
#' @param df_macro       Daily macroclimate dataframe.
#' @param date_seq       Dates of interest.
#' @return Stacked tibble with all scenarios.
extract_all_scenarios <- function(scenario_paths, df_macro, date_seq) {
  map_df(scenario_paths, extract_deltatmax_scenario, df_macro = df_macro, date_seq = date_seq)
}

#' Join daily scenario results with the cLHS sample metadata.

```

```

#'
#' @param df_daily Stacked scenario results (from extract_all_scenarios).
#' @param df_sample cLHS sample dataframe.
#' @return Tibble with structural covariates attached.
join_with_sample <- function(df_daily, df_sample) {
  df_meta <- df_sample %>% dplyr::select(x, y, Cluster, LAI, Hmax, fCover, VCI, starts_with(
  df_daily %>% inner_join(df_meta, by = c("x", "y"))
}

```

4.6 R/forest.R — Extraction des métriques forestières LiDAR

```

# =====
# Chapter 1 – Forest dataframe construction, clustering, and cLHS sampling
# =====

#' Build the forest dataframe from a stacked LiDAR raster.
#'
#' Filters out pixels where the LAD profile sum is below a minimum threshold
#' and returns the pixel coordinates, scalar metrics, and LAD matrix.
#'
#' @param r_stack Terra SpatRaster stack (LAI, VCI, Hmax, fCover, LAD_Layer_*).
#' @return List with elements $df (dataframe), $mat_lad (matrix), $z_breaks (vector).
build_forest_dataframe <- function(r_stack) {
  df_all <- as.data.frame(r_stack, xy = TRUE, na.rm = FALSE) %>%
    filter(if_any(-c(x, y), ~ !is.na(.)))

  mat_lad <- df_all %>% dplyr::select(starts_with("LAD_Layer_")) %>% as.matrix()
  row_sums <- rowSums(mat_lad, na.rm = TRUE)
  valid <- row_sums > 0.5

  df_forest <- df_all[valid, ]
  mat_lad <- mat_lad[valid, ]
  mat_lad[is.na(mat_lad)] <- 0

  list(df = df_forest, mat_lad = mat_lad,
        z_breaks = as.numeric(gsub("LAD_Layer_", "", colnames(mat_lad))))
}

#' Find the optimal number of k-means clusters via the elbow method.
#'
#' Uses maximum perpendicular distance from the WSS curve to the line
#' connecting its endpoints.
#'
#' @param df_scaled Scaled numeric matrix or dataframe.
#' @param k_range Integer range to test (default 1:10).
#' @return List with elements $opt_k, $wss, $k_range, $plot.
optimise_k_elbow <- function(df_scaled, k_range = 1:10) {
  wss <- vapply(k_range, function(k) {
    kmeans(df_scaled, centers = k, nstart = 25, iter.max = 100)$tot.withinss
  }, numeric(1))

  p1 <- c(k_range[1], wss[1]); p2 <- c(k_range[length(k_range)], wss[length(wss)])
  d <- vapply(seq_along(k_range), function(i) {
    p0 <- c(k_range[i], wss[i])
    abs((p2[2]-p1[2])*p0[1] - (p2[1]-p1[1])*p0[2] + p2[1]*p1[2] - p2[2]*p1[1]) /
      sqrt((p2[2]-p1[2])^2 + (p2[1]-p1[1])^2)
  }, numeric(1))
}

```

```

opt_k <- k_range[which.max(d)]

p <- ggplot(data.frame(k = k_range, WSS = wss), aes(x = k, y = WSS)) +
  geom_line(colour = "grey50", linewidth = 1) +
  geom_point(size = 3, colour = "#31688e") +
  geom_vline(xintercept = opt_k, linetype = "dashed", colour = "#d8576b", linewidth = 1) +
  scale_x_continuous(breaks = k_range) +
  labs(title = "Elbow Method for K-means Stratification",
        subtitle = sprintf("Optimal k = %d", opt_k),
        x = "Clusters (k)", y = "Total Within Sum of Squares")

list(opt_k = opt_k, wss = wss, k_range = k_range, plot = p)
}

#' Assign k-means cluster labels to the forest dataframe.
#'
#' If k is NULL the optimal number is determined automatically via
#' optimise_k_elbow().
#'
#' @param df_forest Forest dataframe from build_forest_dataframe.
#' @param k          Fixed number of clusters, or NULL for automatic selection.
#' @param vars       Variables used for clustering (default LAI, Hmax, fCover).
#' @param max_k      Upper bound for automatic k selection (default 10).
#' @return List with elements $df (df_forest with Cluster column) and $elbow_plot.
label_clusters <- function(df_forest, k = NULL, vars = c("LAI", "Hmax", "fCover"), max_k = 10) {
  df_in <- df_forest %>% dplyr::select(all_of(vars))
  ok <- complete.cases(df_in)
  df_scaled <- scale(df_in[ok, ])

  elbow_plot <- NULL
  if (is.null(k)) {
    cat(" [Auto-K] Determining optimal number of clusters via elbow method...\n")
    elbow_res <- optimise_k_elbow(df_scaled, k_range = 1:max_k)
    k <- elbow_res$opt_k
    elbow_plot <- elbow_res$plot
    cat(sprintf(" [Auto-K] Optimal k selected: %d\n", k))
  }

  km <- kmeans(df_scaled, centers = k, iter.max = 100, nstart = 25)
  df_forest$Cluster <- NA_integer_
  df_forest$Cluster[ok] <- km$cluster
  df_forest$Cluster <- factor(df_forest$Cluster)
  list(df = df_forest, elbow_plot = elbow_plot)
}

#' Plot mean LAD profile per cluster in relative height space.
#'
#' Profil LAD moyen par cluster – base d'interprétation typologique.
#' Permet de nommer visuellement les clusters (bottom-heavy, top-heavy, etc.)
#' en affichant le profil moyen de chaque groupe dans l'espace de hauteur absolu.
#'
#' @param df_forest Dataframe avec colonnes x, y, Cluster (issu de label_clusters).
#' @param mat_rel    Matrice LAD normalisée en hauteur relative (de compute_fpca).
#' @param z_rel      Vecteur de hauteurs relatives [0,1].
#' @return ggplot object.
plot_cluster_mean_profiles <- function(df_forest, mat_rel, z_rel) {
  valid <- !is.na(df_forest$Cluster)
  clusters <- if (is.factor(df_forest$Cluster)) levels(droplevels(df_forest$Cluster[valid]))
  else sort(unique(as.character(df_forest$Cluster[valid])))
}

```

```

df_profiles <- lapply(clusters, function(cl) {
  idx <- which(df_forest$Cluster == cl & valid)
  mat <- mat_rel[idx, , drop = FALSE]
  q25 <- apply(mat, 2, quantile, 0.25, na.rm = TRUE)
  q75 <- apply(mat, 2, quantile, 0.75, na.rm = TRUE)
  data.frame(
    rel_z = z_rel,
    density = colMeans(mat, na.rm = TRUE),
    q25 = q25,
    q75 = q75,
    Cluster = cl,
    n = length(idx),
    Hmax_mean = round(mean(df_forest$Hmax[idx], na.rm = TRUE), 1),
    LAI_mean = round(mean(df_forest$LAI[idx], na.rm = TRUE), 2)
  )
})
df_profiles <- bind_rows(df_profiles) %>%
  mutate(label = sprintf("Cluster %s (n=%d)\nHmax=%.1fm | LAI=%.2f",
    Cluster, n, Hmax_mean, LAI_mean))

ggplot(df_profiles, aes(x = density, y = rel_z, colour = Cluster, fill = Cluster)) +
  geom_ribbon(aes(xmin = q25, xmax = q75), alpha = 0.15, colour = NA) +
  geom_path(linewidth = 1.3) +
  geom_hline(yintercept = 1, linetype = "dashed", colour = "firebrick",
    linewidth = 0.7) +
  annotate("text", x = Inf, y = 1, label = "Hmax moyen",
    hjust = 1.05, vjust = -0.4, colour = "firebrick", size = 3) +
  facet_wrap(~ label, nrow = 1) +
  scale_colour_viridis_d(option = "turbo", end = 0.85) +
  scale_fill_viridis_d(option = "turbo", end = 0.85) +
  labs(
    title = "Profil LAD moyen par cluster – base d'interprétation typologique",
    subtitle = "Hauteur relative (z/Hmax) | Ruban = IQR | Tiret = z/Hmax = 1 (sommet de la",
    x = expression("LAD moyen" ~ (m^2 ~ m^{-3})),
    y = "Hauteur relative (z / Hmax)"
  ) +
  theme_bw(base_size = 12) +
  theme(legend.position = "none",
    strip.text = element_text(face = "bold"))
}

#' Conditioned Latin Hypercube Sampling (cLHS) stratified per cluster.
#'
#' @param df_forest Forest dataframe with Cluster column.
#' @param n_per_cluster Target sample size per cluster (default 100).
#' @param vars Variables to include in the LHS design.
#' @param iter Number of cLHS iterations (default 10000).
#' @return Dataframe of sampled plots.
sample_clhs_per_cluster <- function(df_forest, n_per_cluster = 100, vars = c("LAI", "Hmax",
  df_forest %>%
    filter(!is.na(Cluster)) %>%
    split(.$Cluster) %>%
    map_dfr(function(df_sub) {
      df_lhs <- df_sub %>% dplyr::select(x, y, all_of(vars))
      n_target <- min(n_per_cluster, nrow(df_sub))
      lhs_res <- clhs::clhs(df_lhs, size = n_target, iter = iter, simple = FALSE, progress = FALSE)
      df_sub[lhs_res$index_samples, ]
    })
}

```

4.7 R/fpca.R — Analyse en Composantes Principales Fonctionnelles

```
# =====
# Chapter 1 – Shape metrics: FPCA and Height of Median LAI
# =====

#' Sélection automatique du nombre de FPCs à inclure dans le clustering / cLHS.
#'
#' Critère : gain marginal de variance expliquée par FPC_k. On inclut FPC_k
#' tant que varprop[k] > min_marginal.
#'
#' @param varprop Vecteur de proportions de variance (issu de fpca$varprop).
#' @param min_marginal Seuil de gain marginal minimal (défaut 0.05 = 5 %).
#' @return Entier : nombre de FPCs à retenir (au moins 1).
select_n_fpc <- function(varprop, min_marginal = 0.05) {
  n <- sum(varprop >= min_marginal)
  max(1L, n)
}

#' Find the optimal number of B-spline basis functions via GCV elbow.
#'
#' @param arg_vals Vector of argument values (e.g. relative heights).
#' @param mat_lad_norm Normalised LAD matrix (rows = sites, cols = heights).
#' @param nbasis_range Range of nbasis values to test (default 4:15).
#' @return List with elements $opt, $gcv, $range, $plot.
optimise_nbasis <- function(arg_vals, mat_lad_norm, nbasis_range = 4:15) {
  gcv <- vapply(nbasis_range, function(nb) {
    bb <- create.bspline.basis(c(min(arg_vals), max(arg_vals)), nbasis = nb)
    mean(smooth.basis(arg_vals, t(mat_lad_norm), bb)$gcv)
  }, numeric(1))

  p1 <- c(nbasis_range[1], gcv[1]); p2 <- c(nbasis_range[length(nbasis_range)], gcv[length(gcv)])
  d <- vapply(seq_along(nbasis_range), function(i) {
    p0 <- c(nbasis_range[i], gcv[i])
    abs((p2[2]-p1[2])*p0[1] - (p2[1]-p1[1])*p0[2] + p2[1]*p1[2] - p2[2]*p1[1]) /
      sqrt((p2[2]-p1[2])^2 + (p2[1]-p1[1])^2)
  }, numeric(1))
  opt_nb <- nbasis_range[which.max(d)]

  p <- ggplot(data.frame(nbasis = nbasis_range, GCV = gcv), aes(x = nbasis, y = GCV)) +
    geom_line(colour = "grey50", linewidth = 1) +
    geom_point(size = 3, colour = "#31688e") +
    geom_vline(xintercept = opt_nb, linetype = "dashed", colour = "#d8576b", linewidth = 1) +
    labs(title = "GCV Curve for FPCA Basis Functions",
         x = "Number of B-spline basis functions", y = "Mean GCV") +
    theme_bw()

  list(opt = opt_nb, gcv = gcv, range = nbasis_range, plot = p)
}

#' Compute FPCA on the LAD profiles after normalisation to relative height.
#'
#' Profiles are normalised by total LAI and interpolated to a common relative
#' height grid [0,1] before B-spline smoothing and PCA.
#'
#' @param mat_lad LAD matrix (rows = sites, cols = height layers).
#' @param hmax_vec Vector of Hmax values aligned with mat_lad rows.
#' @param z_breaks Height breaks corresponding to the columns of mat_lad.
#' @param n_harm Number of harmonics to retain (default 3).
#' @return List with elements $fpca, $fd_obj, $mat_rel, $z_rel,
```

```

#' $nbasis_opt, $varprop, $gcv_plot.
compute_fpca <- function(mat_lad, hmax_vec, z_breaks, n_harm = 3) {
  row_sums <- rowSums(mat_lad, na.rm = TRUE)
  mat_norm <- sweep(mat_lad, 1, pmax(row_sums, 1e-9), "/")
  n_bins <- length(z_breaks); z_rel <- seq(0, 1, length.out = n_bins)
  mat_rel <- matrix(0, nrow = nrow(mat_norm), ncol = n_bins)
  for (i in seq_len(nrow(mat_norm))) {
    h_i <- max(hmax_vec[i], 1); z_i_rel <- z_breaks / h_i
    mat_rel[i, ] <- pmax(approx(z_i_rel, mat_norm[i, ], z_rel, rule = 2)$y, 0)
  }
  rs <- rowSums(mat_rel); rs[rs == 0] <- 1; mat_rel <- sweep(mat_rel, 1, rs, "/")
  opt <- optimise_nbasis(z_rel, mat_rel)
  basis <- create.bspline.basis(c(0, 1), nbasis = opt$opt)
  fd_obj <- Data2fd(z_rel, t(mat_rel), basis)
  fpca <- pca.fd(fd_obj, nharm = n_harm)
  list(fpca = fpca, fd_obj = fd_obj, mat_rel = mat_rel, z_rel = z_rel,
       nbasis_opt = opt$opt, varprop = fpca$varprop, gcv_plot = opt$plot)
}

#' Profile plots for each FPC: mean curve + Low 5% vs High 5% groups.
#'
#' @param fpca_res Result of compute_fpca.
#' @param mat_lad_real Not used directly but kept for signature compatibility.
#' @param z_breaks Height breaks (not used directly here).
#' @param hmax_vec Hmax values (not used directly here).
#' @return patchwork ggplot of three harmonic panels.
plot_fpc_harmonics <- function(fpca_res, mat_lad_real, z_breaks, hmax_vec) {
  scores <- fpca_res$fpca$scores; z_rel <- fpca_res$z_rel; mat_r <- fpca_res$mat_rel
  one <- function(k, label) {
    q <- quantile(scores[, k], c(0.05, 0.95), na.rm = TRUE)
    lo <- which(scores[, k] <= q[1]); hi <- which(scores[, k] >= q[2])
    df <- bind_rows(data.frame(rel_z = z_rel, lad = colMeans(mat_r[lo, , drop = FALSE])), grp = "L")
    ggplot(df, aes(x = lad, y = rel_z, colour = grp)) + geom_path(linewidth = 1.1) + scale_y_continuous(
    )
  }
  one(1, "FPC1") | one(2, "FPC2") | one(3, "FPC3")
}

#' Plot FPCA loading functions (contribution of each height to each FPC).
#'
#' @param fpca_res Result of compute_fpca.
#' @return ggplot object with facets per FPC.
plot_fpc_loadings <- function(fpca_res) {
  harmonics <- fpca_res$fpca$harmonics; z_rel <- fpca_res$z_rel; vals <- eval.fd(z_rel, harmonics)
  df <- as.data.frame(vals); names(df) <- paste0("FPC", seq_len(ncol(df))); df$rel_z <- z_rel
  df_long <- pivot_longer(df, -rel_z, names_to = "FPC", values_to = "loading")
  ggplot(df_long, aes(x = loading, y = rel_z, colour = FPC)) + geom_path(linewidth = 1.1) +
  facet_wrap(~FPC)
}

#' Scatter plots of FPC score pairs, optionally coloured by cluster.
#'
#' @param fpca_res Result of compute_fpca.
#' @param cluster_vec Optional factor vector of cluster labels.
#' @return patchwork ggplot with FPC1×FPC2, FPC1×FPC3, FPC2×FPC3.
plot_fpc_scatters <- function(fpca_res, cluster_vec = NULL) {
  scores <- as.data.frame(fpca_res$fpca$scores[, 1:3])
  names(scores) <- c("FPC1", "FPC2", "FPC3")
  if (!is.null(cluster_vec)) scores$Cluster <- factor(cluster_vec)

  base <- function(xv, yv) {
    p <- ggplot(scores, aes(x = .data[[xv]], y = .data[[yv]]))
    if (!is.null(cluster_vec)) p <- p + aes(colour = Cluster)
    p
  }
  base("FPC1", "FPC2") | base("FPC1", "FPC3") | base("FPC2", "FPC3")
}

```

```

    if (!is.null(cluster_vec)) {
      p <- p + geom_point(aes(colour = Cluster), alpha = 0.5) + scale_colour_viridis_d(option)
    } else {
      p <- p + geom_point(alpha = 0.4, colour = "#31688e")
    }
  }
  p + labs(x = xv, y = yv)
}
(base("FPC1", "FPC2") | base("FPC1", "FPC3") | base("FPC2", "FPC3")) +
  plot_layout(guides = "collect") & theme(legend.position = "bottom")
}

#' Qualité de reconstruction FPCA : profils réels vs 3 premières composantes.
#'
#' Pour chaque site sélectionné, superpose le profil LAD normalisé réel
#' et sa reconstruction :  $\text{mean}(z) + \sum_{k=1}^3 \text{score}_k \times \text{FPC}_k(z)$ .
#'
#' @param fpca_res Résultat de compute_fpca() (contient mat_rel, z_rel, fpca).
#' @param idx_sites Indices explicites. Si NULL : n_sites sites espacés sur FPC1.
#' @param n_sites Nombre de sites si idx_sites est NULL (défaut 6).
#' @param cluster_vec Vecteur optionnel de labels Cluster pour annoter les panneaux.
#' @param out_path Chemin PNG de sortie.
#' @return Invisible ggplot.
plot_fpca_reconstruction <- function(fpca_res, idx_sites = NULL, n_sites = 6,
                                     cluster_vec = NULL,
                                     out_path = "outputs/figures/10_fpca_reconstruction.png") {
  z_rel <- fpca_res$z_rel
  mat_real <- fpca_res$mat_rel
  scores <- fpca_res$fpca$scores
  mean_curve <- as.numeric(eval.fd(z_rel, fpca_res$fpca$meanfd))
  harm_matrix <- eval.fd(z_rel, fpca_res$fpca$harmonics)

  n_plots <- nrow(mat_real)
  if (is.null(idx_sites)) {
    fpc1_order <- order(scores[, 1])
    idx_sites <- fpc1_order[round(seq(1, n_plots, length.out = n_sites))]
  }
  idx_sites <- idx_sites[idx_sites >= 1 & idx_sites <= n_plots]

  df_all <- purrr::imap_dfr(idx_sites, function(i, ii) {
    recon <- pmax(as.numeric(mean_curve + harm_matrix[, 1:3] %*% scores[i, 1:3]), 0)
    cl_label <- if (!is.null(cluster_vec)) sprintf(" · Cl.%s", cluster_vec[i]) else ""
    site_label <- sprintf("Site #%d%s\nFPC1=%.2f", i, cl_label, scores[i, 1])
    bind_rows(
      data.frame(rel_z = z_rel, lad = mat_real[i, ],
                 source = "Réal (Lidar)", site = site_label),
      data.frame(rel_z = z_rel, lad = recon,
                 source = "Reconstruit (3 CP)", site = site_label)
    )
  })

  var_str <- paste(sprintf("FPC%d=%.1f%%", 1:3, 100 * fpca_res$varprop[1:3]),
                   collapse = " | ")

  p <- ggplot(df_all, aes(x = lad, y = rel_z, colour = source, linetype = source)) +
    geom_path(linewidth = 1.1) +
    facet_wrap(~ site, nrow = 2, scales = "free_x") +
    scale_colour_manual(
      values = c("Réal (Lidar)" = "#31688e", "Reconstruit (3 CP)" = "#d8576b")
    ) +
    scale_linetype_manual(

```

```

    values = c("Réel (Lidar)" = "solid", "Reconstruit (3 CP)" = "dashed")
  ) +
  labs(
    title = "Reconstruction FPCA – profils réels vs 3 premières composantes",
    subtitle = var_str,
    x = "LAD normalisé (relatif)",
    y = "Hauteur relative (z / Hmax)",
    colour = NULL, linetype = NULL
  ) +
  theme_bw(base_size = 11) +
  theme(legend.position = "bottom", strip.text = element_text(size = 8))

dir.create(dirname(out_path), recursive = TRUE, showWarnings = FALSE)
save_plot(p, out_path, width = 14, height = 7)
cat(sprintf(" [FPCA reconstruction] %d sites – %s\n", length(idx_sites), out_path))
invisible(p)
}

#' Compute the Height of Median LAI (centre of gravity of the LAD profile).
#'
#' @param mat_lad LAD matrix (rows = sites, cols = height layers).
#' @param z_breaks Height break values aligned with mat_lad columns.
#' @return Numeric vector of H_median values (one per site).
compute_h_median <- function(mat_lad, z_breaks) {
  apply(mat_lad, 1, function(prof) {
    total_lai <- sum(prof, na.rm = TRUE)
    if (total_lai <= 0) return(0)
    cum_lai <- cumsum(prof)
    idx <- which(cum_lai >= (total_lai / 2))[1]
    return(z_breaks[idx])
  })
}

```

4.8 R/h1_analysis.R — Analyse H1 : attribution structurelle

```

# =====
# Chapter 1 – H1 analysis: forward inclusion, L00, and Shapley attribution
# =====

#' Score scenarios by their divergence from a reference (full-real) scenario.
#'
#' @param df_all Stacked scenario results (x, y, date, Delta_Tmax, scenario).
#' @param ref_scenario_name Name of the reference scenario.
#' @return Tibble sorted by RMSE with columns: scenario, n, mean_diff, mae, rmse.
score_scenarios_vs_reference <- function(df_all, ref_scenario_name) {
  df_ref <- df_all %>% filter(scenario == ref_scenario_name) %>% dplyr::select(x, y, date, D
  df_all %>% inner_join(df_ref, by = c("x", "y", "date")) %>% mutate(diff = Delta_Tmax - Del
    summarise(n = n(), mean_diff = mean(diff, na.rm = TRUE), mae = mean(abs(diff), na.rm = T
  }

#' Bar chart of scenario RMSE vs reference, coloured by mean bias.
#'
#' @param df_scores Tibble from score_scenarios_vs_reference.
#' @param ref_scenario_name Name of the reference scenario to exclude from the plot.
#' @return ggplot object.
plot_scenario_hierarchy <- function(df_scores, ref_scenario_name) {
  readable <- function(s) {
    s %>%

```

```

    sub("^H1F_", "Fact ", .) %>%
    sub("^H1f_0_", "Null: ", .) %>%
    sub("^H1f_([0-9]+)_", "Fwd\\1 ", .) %>%
    sub("^H1l_drop([A-Za-z]+)_.*", "L00 -\\1", .) %>%
    gsub("_m_", "(L=μ)", .) %>%
    gsub("_r_", "(L=r)", .) %>%
    gsub("_m$", "(D=μ)", .) %>%
    gsub("_r$", "(D=r)", .) %>%
    gsub("_u$", "(D=u)", .) %>%
    gsub("_a_", "(F=r)", .) %>%
    gsub("_", " ", .)
  }
df_plot <- df_scores %>%
  filter(scenario != ref_scenario_name) %>%
  mutate(label = readable(scenario))
ggplot(df_plot, aes(x = reorder(label, rmse), y = rmse, fill = mean_diff)) +
  geom_col() +
  coord_flip() +
  scale_fill_gradient2(low = "#31688e", mid = "white", high = "#d8576b",
    midpoint = 0, name = "Mean bias\\n(°C)") +
  labs(
    title = "H1 – divergence from full-real reference",
    caption = "Fact=Factorial | Fwd=Forward | L00=Leave-one-out | L=LAI D=LAD F=fCover | μ",
    x = NULL, y = "RMSE vs reference (°C)"
  ) +
  theme_bw(base_size = 10) +
  theme(
    axis.text.y = element_text(size = 8),
    plot.caption = element_text(size = 7, colour = "grey40")
  )
}

#' Line plot of RMSE along the forward inclusion sequence.
#'
#' @param df_scores Tibble from score_scenarios_vs_reference.
#' @param forward_order Ordered character vector of scenario names.
#' @return ggplot object.
plot_forward_curve <- function(df_scores, forward_order) {
  df_plot <- df_scores %>%
    filter(scenario %in% forward_order) %>%
    mutate(scenario = factor(scenario, levels = forward_order),
      step = as.integer(scenario))
  ggplot(df_plot, aes(x = step, y = rmse)) +
    geom_line(linewidth = 1) +
    geom_point(size = 3, colour = "#31688e") +
    geom_text(aes(label = sprintf("%.2f°C", rmse)),
      vjust = -0.9, size = 3.5, colour = "grey30") +
    scale_x_continuous(breaks = seq_along(forward_order), labels = forward_order) +
    labs(title = "H1 forward – information added incrementally",
      x = NULL, y = "RMSE vs full-real reference (°C)") +
    theme_bw(base_size = 12) +
    theme(axis.text.x = element_text(angle = 25, hjust = 1),
      plot.margin = margin(t = 5, r = 10, b = 50, l = 5, unit = "pt"))
}

#' Scatter plots ΔTmax scénario vs référence – un panneau par étape H1 forward.
#'
#' Demandé lors de la réunion du 17/04 : produire un scatter par paramètre
#' (pas seulement pour le profil complet). Chaque panneau montre la relation
#' entre le ΔTmax du scénario et celui de la référence full-real, avec R²,

```

```

#' RMSE et biais annotés.
#'
#' @param df_all      Dataframe empilé (x, y, date, Delta_Tmax, scenario).
#' @param ref_scenario_name Nom du scénario de référence (ex. "REF_all_real").
#' @param forward_names Vecteur ordonné des noms de scénarios H1 forward à
#'   inclure (sans ref ni Null_baseline). Si NULL, tous sauf la référence.
#' @param out_path     Chemin PNG de sortie.
#' @return Invisible ggplot.
plot_h1_scenario_scatters <- function(df_all, ref_scenario_name,
                                     forward_names = NULL,
                                     out_path = "outputs/figures/04_h1_scatters_vs_ref.png") {
  label_map <- c(
    H1f_1_LAI_only      = "+ LAI",
    H1f_2_LAI_Hmax      = "+ Hmax",
    H1f_3_LAI_Hmax_fCover = "+ fCover",
    H1f_4_Full_real      = "+ Profil LAD"
  )

  df_ref <- df_all %>%
    filter(scenario == ref_scenario_name) %>%
    dplyr::select(x, y, date, Delta_ref = Delta_Tmax)

  sc_names <- if (!is.null(forward_names)) forward_names else
    setdiff(unique(df_all$scenario), c(ref_scenario_name, "H1f_0_Null_baseline"))

  df_joined <- df_all %>%
    filter(scenario %in% sc_names) %>%
    inner_join(df_ref, by = c("x", "y", "date")) %>%
    mutate(
      scenario_lab = dplyr::recode(scenario, !!!label_map, .default = scenario),
      scenario_lab = factor(scenario_lab, levels = label_map[label_map %in% unique(scenario_
    )

  lims <- range(c(df_joined$Delta_Tmax, df_joined$Delta_ref), na.rm = TRUE)
  x_ann <- lims[1] + 0.05 * diff(lims)
  y_ann <- lims[2] - 0.05 * diff(lims)

  df_stats <- df_joined %>%
    group_by(scenario_lab) %>%
    summarise(
      r2 = cor(Delta_Tmax, Delta_ref, use = "complete.obs")^2,
      rmse = sqrt(mean((Delta_Tmax - Delta_ref)^2, na.rm = TRUE)),
      bias = mean(Delta_Tmax - Delta_ref, na.rm = TRUE),
      .groups = "drop"
    ) %>%
    mutate(
      label = sprintf("R²=%.2f\nRMSE=%.2f°C\nBias=%+.2f°C", r2, rmse, bias),
      x_pos = x_ann, y_pos = y_ann
    )

  p <- ggplot(df_joined, aes(x = Delta_ref, y = Delta_Tmax)) +
    geom_hex(bins = 60) +
    scale_fill_gradient(low = "grey85", high = "midnightblue",
                       trans = "log10", name = "Count\n(log)") +
    geom_abline(slope = 1, intercept = 0, linetype = "dashed",
                colour = "black", linewidth = 0.8) +
    geom_smooth(method = "lm", se = FALSE, colour = "#d73027", linewidth = 1.1) +
    geom_text(
      data = df_stats,
      aes(x = x_pos, y = y_pos, label = label),

```

```

    hjust = 0, vjust = 1, fontface = "bold", size = 3.2,
    inherit.aes = FALSE
  ) +
  coord_fixed(xlim = lims, ylim = lims) +
  facet_wrap(~ scenario_lab, nrow = 1) +
  labs(
    title = "H1 - ΔTmax scénario vs référence (full-real), par paramètre",
    subtitle = "Ajout progressif : LAI → Hmax → fCover → profil LAD complet",
    x = expression("Référence (full-real)" ~ Delta * T[max] ~ (degree*C)),
    y = expression("Scénario" ~ Delta * T[max] ~ (degree*C))
  ) +
  theme_bw(base_size = 11) +
  theme(legend.position = "right",
    strip.text = element_text(face = "bold"),
    plot.title = element_text(face = "bold"))

dir.create(dirname(out_path), recursive = TRUE, showWarnings = FALSE)
save_plot(p, out_path, width = 16, height = 6)
cat(sprintf(" [h1_scatters] PNG : %s\n", out_path))
invisible(p)
}

#' Comparaison Forward vs L00 pour valider la robustesse du ranking des variables.
#'
#' Si l'ordre des barres Forward et L00 est cohérent (LAI > Hmax > fCover > profil
#' LAD ou inverse), le ranking n'est pas un artefact de l'ordre d'insertion.
#' Les deux scénarios L00 pour le profil LAD (uniform vs mean-shape) illustrent
#' la sensibilité au choix de la baseline.
#'
#' @param df_scores Dataframe issu de score_scenarios_vs_reference().
#' @param forward_order Vecteur ordonné des noms de scénarios H1 forward.
#' @param out_path Chemin PNG de sortie.
#' @return Invisible ggplot.
compare_h1_rankings <- function(df_scores,
                                forward_order,
                                out_path = "outputs/figures/02_h1_ranking.png") {

  fwd_labels <- c(
    H1f_1_LAI_only = "LAI",
    H1f_2_LAI_Hmax = "Hmax",
    H1f_3_LAI_Hmax_fCover = "fCover",
    H1f_4_Full_real = "Profil LAD"
  )
  loo_labels <- c(
    H1l_dropLAI_mean = "LAI",
    H1l_dropHmax_mean = "Hmax",
    H1l_dropfCover_mean = "fCover",
    H1l_dropLAD_uniform = "Profil LAD",
    H1l_dropLAD_meanShape = "Profil LAD"
  )
  loo_approach <- c(
    H1l_dropLAI_mean = "L00",
    H1l_dropHmax_mean = "L00",
    H1l_dropfCover_mean = "L00",
    H1l_dropLAD_uniform = "L00 (base. uniforme)",
    H1l_dropLAD_meanShape = "L00 (base. moy-forme)"
  )
  var_order <- c("LAI", "Hmax", "fCover", "Profil LAD")
  metrics <- c("rmse", "mae", "mean_diff")

```

```

fwd_scs <- forward_order[forward_order %in% df_scores$scenario]
df_fwd_scores <- df_scores %>%
  filter(scenario %in% fwd_scs) %>%
  mutate(step = match(scenario, forward_order)) %>%
  arrange(step)

df_fwd_long <- purrr::map_dfr(metrics, function(m) {
  vals <- setNames(df_fwd_scores[[m]], df_fwd_scores$scenario)
  steps <- fwd_scs
  purrr::map_dfr(seq_along(steps)[-1], function(i) {
    tibble(variable = fwd_labels[steps[i]], approach = "Forward",
           metric = m, delta = vals[steps[i - 1]] - vals[steps[i]])
  })
}) %>% filter(!is.na(variable))

loo_scs_present <- intersect(names(loo_labels), df_scores$scenario)
df_loo_long <- tibble()
if (length(loo_scs_present) > 0) {
  full_row <- df_scores %>% filter(scenario == "H1f_4_Full_real")
  df_loo_long <- purrr::map_dfr(metrics, function(m) {
    full_val <- if (nrow(full_row) > 0) full_row[[m]] else 0
    df_scores %>%
      filter(scenario %in% loo_scs_present) %>%
      mutate(variable = loo_labels[scenario],
             approach = loo_approach[scenario],
             metric = m,
             delta = !!sym(m) - full_val) %>%
      dplyr::select(variable, approach, metric, delta)
  })
}

approach_levels <- c("Forward", "L00", "L00 (base. uniforme)", "L00 (base. moy-forme)")
approach_colors <- c(
  "Forward" = "#31688e",
  "L00" = "#d8576b",
  "L00 (base. uniforme)" = "#f0a500",
  "L00 (base. moy-forme)" = "#7b2d8b"
)

metric_labels <- c(
  rmse = "ΔRMSE (°C)",
  mae = "ΔMAE (°C)",
  mean_diff = "ΔBias (°C, signé)"
)

df_plot <- bind_rows(df_fwd_long, df_loo_long) %>%
  mutate(
    variable = factor(variable, levels = var_order),
    approach = factor(approach, levels = approach_levels),
    metric_lab = factor(metric_labels[metric], levels = metric_labels)
  )

p <- ggplot(df_plot, aes(x = variable, y = delta, fill = approach)) +
  geom_col(position = position_dodge(width = 0.75), width = 0.65) +
  geom_hline(yintercept = 0, linewidth = 0.4, colour = "grey40") +
  scale_fill_manual(values = approach_colors, name = NULL, drop = TRUE) +
  facet_wrap(~ metric_lab, scales = "free_y", ncol = 1) +
  labs(
    title = "H1 – Robustesse du ranking : Forward vs L00",
    subtitle = "Concordance Forward/L00 = ranking indépendant de l'ordre | 2 barres L00 po",
    x = NULL,

```

```

    y      = "Réduction d'erreur (°C, positif = amélioration)"
  ) +
  theme_bw(base_size = 11) +
  theme(
    axis.text.x      = element_text(angle = 20, hjust = 1),
    strip.text       = element_text(face = "bold"),
    plot.title       = element_text(face = "bold"),
    legend.position  = "top"
  )

  dir.create(dirname(out_path), recursive = TRUE, showWarnings = FALSE)
  save_plot(p, out_path, width = 9, height = 12)
  cat(sprintf(" [h1_rankings] PNG : %s\n", out_path))
  invisible(p)
}

#' Scatter 2x2 : ΔTmax journalier LiDAR-réel (y) vs scénario avec une seule
#' variable dégradée (x). Chaque panneau = une variable remplacée par sa valeur
#' absente (LAI moyen, Hmax moyen, fCover=1, LAD uniforme).
#'
#' @param df_all Dataframe empilé (x, y, date, Delta_Tmax, scenario).
#' @param ref_scenario_name Scénario de référence "full real".
#' @param out_path Chemin PNG de sortie.
#' @return Invisible ggplot.
plot_loo_scatters_2x2 <- function(df_all,
                                  ref_scenario_name = "H1f_4_Full_real",
                                  out_path = "outputs/figures/03_h1_loo_scatters_2x2.png")

  loo_map <- c(
    H1l_dropLAI_mean      = "LAI → moyen",
    H1l_dropHmax_mean     = "Hmax → moyen",
    H1l_dropfCover_mean   = "fCover → 1",
    H1l_dropLAD_uniform   = "LAD → uniforme"
  )
  panel_order <- unname(loo_map)

  df_ref <- df_all %>%
    filter(scenario == ref_scenario_name) %>%
    dplyr::select(x, y, date, Delta_real = Delta_Tmax)

  loo_scs <- intersect(names(loo_map), unique(df_all$scenario))
  if (length(loo_scs) == 0) {
    warning("[plot_loo_scatters_2x2] aucun scénario L00 trouvé dans df_all")
    return(invisible(NULL))
  }

  df_joined <- df_all %>%
    filter(scenario %in% loo_scs) %>%
    inner_join(df_ref, by = c("x", "y", "date")) %>%
    mutate(panel = factor(loo_map[scenario], levels = panel_order))

  lims <- range(c(df_joined$Delta_Tmax, df_joined$Delta_real), na.rm = TRUE)
  x_ann <- lims[1] + 0.05 * diff(lims)
  y_ann <- lims[2] - 0.05 * diff(lims)

  df_stats <- df_joined %>%
    group_by(panel) %>%
    summarise(
      r2      = cor(Delta_Tmax, Delta_real, use = "complete.obs")^2,
      rmse    = sqrt(mean((Delta_Tmax - Delta_real)^2, na.rm = TRUE)),
      bias    = mean(Delta_Tmax - Delta_real, na.rm = TRUE),

```

```

    .groups = "drop"
  ) %>%
  mutate(
    label = sprintf("R²=%.2f\nRMSE=%.2f°C\nBias=%+.2f°C",
                    r2, rmse, bias),
    x_pos = x_ann, y_pos = y_ann
  )

p <- ggplot(df_joined, aes(x = Delta_Tmax, y = Delta_real)) +
  geom_hex(bins = 50) +
  scale_fill_gradient(low = "grey88", high = "midnightblue",
                     trans = "log10", name = "N (log)") +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed",
              colour = "black", linewidth = 0.7) +
  geom_smooth(method = "lm", se = FALSE, colour = "#d73027", linewidth = 1) +
  geom_text(data = df_stats,
            aes(x = x_pos, y = y_pos, label = label),
            hjust = 0, vjust = 1, size = 3, fontface = "bold",
            inherit.aes = FALSE) +
  coord_fixed(xlim = lims, ylim = lims) +
  facet_wrap(~ panel, nrow = 2, ncol = 2) +
  labs(
    title = "Impact d'une variable structurelle – ΔTmax journalier",
    subtitle = "x = scénario avec une variable dégradée | y = LiDAR réel (Full-real)",
    x = expression(Delta * T[max] ~ "scénario dégradé (°C)"),
    y = expression(Delta * T[max] ~ "LiDAR réel (°C)")
  ) +
  theme_bw(base_size = 11) +
  theme(strip.text = element_text(face = "bold"),
        plot.title = element_text(face = "bold"),
        legend.position = "right")

dir.create(dirname(out_path), recursive = TRUE, showWarnings = FALSE)
save_plot(p, out_path, width = 10, height = 10)
cat(sprintf(" [loo_scatters_2x2] PNG : %s\n", out_path))
invisible(p)
}

# ---- Shapley exact attribution (2^4 lattice) -----

#' Extract RMSE values for the 16 coalitions from df_scores.
#'
#' @param df_scores Tibble from score_scenarios_vs_reference (must contain
#'   all 16 H1F_* factorial scenarios).
#' @return Named numeric vector of RMSE values, one per coalition binary string.
extract_shapley_coalitions <- function(df_scores) {
  coalition_to_scenario <- c(
    "0000" = "H1F_m_m_a_u", "1000" = "H1F_r_m_a_u",
    "0100" = "H1F_m_r_a_u", "0010" = "H1F_m_m_r_u",
    "0001" = "H1F_m_m_a_r", "1100" = "H1F_r_r_a_u",
    "1010" = "H1F_r_m_r_u", "1001" = "H1F_r_m_a_r",
    "0110" = "H1F_m_r_r_u", "0101" = "H1F_m_r_a_r",
    "0011" = "H1F_m_m_r_r", "1110" = "H1F_r_r_r_u",
    "1101" = "H1F_r_r_a_r", "1011" = "H1F_r_m_r_r",
    "0111" = "H1F_m_r_r_r", "1111" = "H1F_r_r_r_r"
  )
  rmse_values <- sapply(coalition_to_scenario, function(sc) {
    val <- df_scores$rmse[df_scores$scenario == sc]
    if (length(val) == 0) {
      warning(sprintf("Scenario manquant pour treillis Shapley : %s", sc))
    }
  })
}

```

```

    return(NA_real_)
  }
  val[1]
})
names(rmse_values) <- names(coalition_to_scenario)
if (any(is.na(rmse_values)))
  stop("Treillis Shapley incomplet : certains scenarios factoriel manquent.")
rmse_values
}

#' Compute exact Shapley values from the 2^4 factorial RMSE lattice.
#'
#' @param rmse_values Named numeric vector (16 coalitions) from
#'   extract_shapley_coalitions.
#' @return List with elements $shapley, $rmse_values, $v_empty,
#'   $total_effect, $total_sum, $check_diff.
compute_shapley_exact <- function(rmse_values) {
  v_empty <- as.numeric(rmse_values["0000"])
  v <- function(b) v_empty - as.numeric(rmse_values[b])

  make_bin <- function(bits_on, n = 4) {
    b <- rep("0", n); b[bits_on] <- "1"; paste(b, collapse = "")
  }
  variables <- c("LAI", "Hmax", "fCover", "LAD")

  shapley_vals <- sapply(seq_along(variables), function(var_bit) {
    others <- setdiff(seq_len(4), var_bit)
    total <- 0
    for (s in 0:3) {
      subsets <- if (s == 0) list(integer(0)) else combn(others, s, simplify = FALSE)
      w <- factorial(s) * factorial(4 - s - 1) / factorial(4)
      for (S in subsets)
        total <- total + w * (v(make_bin(c(S, var_bit))) - v(make_bin(S)))
    }
    total
  })
  names(shapley_vals) <- variables

  total_shapley <- sum(shapley_vals)
  total_effect <- v("1111")

  cat("— Valeurs Shapley exactes —\n")
  for (i in seq_along(variables))
    cat(sprintf("  phi(%-6s) = %+.4f °C  (%+.1f%% de v(N))\n",
               variables[i], shapley_vals[i],
               100 * shapley_vals[i] / total_effect))
  cat(sprintf("\n Somme phi = %+.4f | v(N) = %+.4f | écart = %.2e\n\n",
              total_shapley, total_effect, abs(total_shapley - total_effect)))

  list(shapley = shapley_vals, rmse_values = rmse_values,
       v_empty = v_empty, total_effect = total_effect,
       total_sum = total_shapley, check_diff = abs(total_shapley - total_effect))
}

#' Three-panel figure: Shapley bars, Forward/L00/Shapley comparison, coalition lattice.
#'
#' @param shap_res Result of compute_shapley_exact.
#' @param df_scores Tibble from score_scenarios_vs_reference.
#' @param out_path PNG output path.
#' @return Invisible patchwork ggplot.

```

```

plot_shapley_attribution <- function(shap_res, df_scores,
                                   out_path = "outputs/figures/02_shapley_attribution.png")
  variables <- c("LAI", "Hmax", "fCover", "LAD")
  var_order <- c("LAI", "fCover", "Hmax", "LAD")
  rv <- shap_res$rmse_values

  # as.numeric() strips inner names to avoid "LAI.0000" concatenation bug
  forward_gain <- c(
    LAI = as.numeric(rv["0000"] - rv["1000"]),
    Hmax = as.numeric(rv["1000"] - rv["1100"]),
    fCover = as.numeric(rv["1100"] - rv["1110"]),
    LAD = as.numeric(rv["1110"] - rv["1111"])
  )
  loo_loss <- c(
    LAI = as.numeric(rv["0111"] - rv["1111"]),
    Hmax = as.numeric(rv["1011"] - rv["1111"]),
    fCover = as.numeric(rv["1101"] - rv["1111"]),
    LAD = as.numeric(rv["1110"] - rv["1111"])
  )

  # Panel A: Shapley bars
  df_A <- data.frame(
    variable = factor(variables, levels = var_order),
    phi = as.numeric(shap_res$shapley),
    label = sprintf("%+.3f\n(%.0f%%)",
                    shap_res$shapley,
                    100 * shap_res$shapley / shap_res$total_effect)
  )
  pal_vars <- c(LAI = "#440154", fCover = "#31688e", Hmax = "#35b779", LAD = "#fde725")
  pA <- ggplot(df_A, aes(x = variable, y = phi, fill = variable)) +
    geom_col(width = 0.7, colour = "grey20", linewidth = 0.3) +
    geom_hline(yintercept = 0, colour = "grey40", linewidth = 0.4) +
    geom_text(aes(label = label, vjust = ifelse(phi >= 0, -0.3, 1.3)),
              size = 4, fontface = "bold") +
    scale_fill_manual(values = pal_vars) +
    scale_y_continuous(expand = expansion(mult = c(0.15, 0.20))) +
    labs(title = "A. Shapley attribution of canopy structural drivers",
         subtitle = sprintf("Total buffering v(N) = %.3f °C | Σφ = %.4f",
                             shap_res$total_effect, shap_res$total_sum),
         x = NULL, y = expression(phi ~ "(" * degree * "°C")) +
    theme_bw(base_size = 12) +
    theme(legend.position = "none", plot.title = element_text(face = "bold"),
          axis.text.x = element_text(face = "bold", size = 12))

  # Panel B: Forward / L00 / Shapley comparison
  df_B <- data.frame(
    variable = rep(variables, 3),
    method = rep(c("Forward", "L00", "Shapley"), each = 4),
    value = c(as.numeric(forward_gain[variables]),
              as.numeric(loo_loss[variables]),
              as.numeric(shap_res$shapley))
  ) %>% mutate(variable = factor(variable, levels = var_order),
              method = factor(method, levels = c("Forward", "L00", "Shapley")))

  pB <- ggplot(df_B, aes(x = variable, y = value, fill = method)) +
    geom_col(position = position_dodge(0.8), width = 0.7,
            colour = "grey20", linewidth = 0.2) +
    geom_hline(yintercept = 0, colour = "grey40", linewidth = 0.4) +
    scale_fill_manual(values = c(Forward = "#31688e", L00 = "#d8576b", Shapley = "#5ec962"),
                      name = NULL) +

```

```

labs(title = "B. Method comparison : Forward / L00 / Shapley",
      subtitle = "Shapley averages over all permutations – resolves Forward/L00 divergence",
      x = NULL, y = "Attribution (°C)") +
theme_bw(base_size = 12) +
theme(legend.position = "top", plot.title = element_text(face = "bold"),
      axis.text.x = element_text(face = "bold", size = 12))

# Panel C: coalition lattice
df_C <- data.frame(
  coalition = names(rv), rmse = as.numeric(rv),
  v_S = shap_res$v_empty - as.numeric(rv)
) %>% mutate(
  size = factor(sapply(coalition, function(s) sum(as.integer(strsplit(s, "")[[1]]))),
                levels = 0:4, labels = paste0("|S|=", 0:4)),
  label = sapply(coalition, function(s) {
    bits <- as.integer(strsplit(s, "")[[1]])
    lbl <- paste(c("L", "H", "F", "D")[bits == 1], collapse="")
    if (lbl == "") "∅" else lbl
  })
)
pC <- ggplot(df_C, aes(x = size, y = v_S)) +
  geom_jitter(aes(colour = size), width = 0.15, size = 3.5, alpha = 0.85) +
  geom_text(aes(label = label), size = 2.8, vjust = -1.1,
            colour = "grey30", fontface = "bold") +
  geom_hline(yintercept = shap_res$total_effect, linetype = "dashed",
             colour = "red", linewidth = 0.6) +
  annotate("text", x = 0.6, y = shap_res$total_effect,
           label = sprintf("v(N)=%.3f°C", shap_res$total_effect),
           hjust = 0, vjust = -0.5, colour = "red", fontface = "bold", size = 3.5) +
  scale_colour_viridis_d(option = "plasma", end = 0.85) +
  scale_y_continuous(expand = expansion(mult = c(0.05, 0.15))) +
  labs(title = "C. The 2^4 = 16 coalitions of the Shapley lattice",
       subtitle = "v(S) = RMSE(∅) - RMSE(S) | L=LAI, H=Hmax, F=fCover, D=LAD",
       x = "Coalition size |S|", y = expression("v(S)" ~ "(" * degree * "C)")) +
  theme_bw(base_size = 12) +
  theme(legend.position = "none", plot.title = element_text(face = "bold"))

fig <- (pA + pB) / pC +
  patchwork::plot_annotation(
    title = "Shapley attribution of canopy structural drivers on ΔTmax",
    subtitle = "Blois oak forest | MuSICA simulations | Summer 2021-2022",
    theme = theme(plot.title = element_text(face = "bold", size = 14),
                  plot.subtitle = element_text(colour = "grey30"))
  ) +
  patchwork::plot_layout(heights = c(1.2, 1))

dir.create(dirname(out_path), recursive = TRUE, showWarnings = FALSE)
save_plot(fig, out_path, width = 14, height = 11)

csv_path <- sub("\\.png$", ".csv", out_path)
write.csv(data.frame(
  variable = variables,
  shapley = as.numeric(shap_res$shapley),
  shapley_pct = 100 * shap_res$shapley / shap_res$total_effect,
  forward_gain = as.numeric(forward_gain[variables]),
  loo_loss = as.numeric(loo_loss[variables])
), csv_path, row.names = FALSE)
cat(sprintf(" [Shapley] PNG : %s\n [Shapley] CSV : %s\n", out_path, csv_path))
invisible(fig)
}

```

```

# ---- Shapley bootstrap (stability over plot sub-samples) -----

#' Bootstrap Shapley values over random sub-samples of plots.
#'
#' Reuses existing extracted scenario results – no MuSICA re-runs.
#' Each iteration subsamples n_per_boot rows of df_sample, filters
#' df_all_scenarios to those (x,y), rescores, and recomputes Shapley.
#'
#' @param df_all_scenarios Stacked scenario results (must include all 16 H1F_*).
#' @param df_sample        cLHS sample dataframe.
#' @param ref_scenario_name Reference scenario name.
#' @param n_boot           Number of bootstrap iterations (default 5).
#' @param n_per_boot       Plots per iteration (default half of df_sample rows).
#' @return Long tibble with columns: boot_id, variable, phi, phi_pct.
bootstrap_shapley <- function(df_all_scenarios, df_sample,
                             ref_scenario_name,
                             n_boot = 5,
                             n_per_boot = NULL) {
  if (is.null(n_per_boot)) n_per_boot <- ceiling(nrow(df_sample) / 2)
  variables <- c("LAI", "Hmax", "fCover", "LAD")
  boot_list <- vector("list", n_boot)

  for (b in seq_len(n_boot)) {
    idx <- sample(seq_len(nrow(df_sample)), n_per_boot, replace = FALSE)
    sub_xy <- df_sample[idx, c("x", "y")]

    df_sub <- df_all_scenarios %>%
      inner_join(sub_xy, by = c("x", "y"))

    shap_b <- tryCatch({
      df_scores_b <- score_scenarios_vs_reference(df_sub, ref_scenario_name)
      coalitions_b <- extract_shapley_coalitions(df_scores_b)
      capture.output(res_b <- compute_shapley_exact(coalitions_b))
      tibble(
        boot_id = b,
        variable = variables,
        phi = as.numeric(res_b$shapley),
        phi_pct = 100 * res_b$shapley / res_b$total_effect
      )
    }, error = function(e) {
      warning(sprintf("[Shapley bootstrap] boot %d failed: %s", b, e$message))
      NULL
    })

    boot_list[[b]] <- shap_b
    cat(sprintf(" [Shapley bootstrap] boot %d/%d done\n", b, n_boot))
  }
  bind_rows(boot_list)
}

#' Bar + jitter plot of bootstrap Shapley values with mean ± sd.
#'
#' @param df_boot      Result of bootstrap_shapley.
#' @param shap_exact   Exact Shapley result (compute_shapley_exact) – overlaid as diamonds.
#' @param out_path     PNG output path.
#' @return Invisible ggplot.
plot_shapley_bootstrap <- function(df_boot, shap_exact = NULL,
                                   out_path = "outputs/figures/annex/A18_shapley_bootstrap.
  variables <- c("LAI", "Hmax", "fCover", "LAD")
  pal_vars <- c(LAI = "#440154", fCover = "#31688e", Hmax = "#35b779", LAD = "#fde725")

```

```

df_sum <- df_boot %>%
  group_by(variable) %>%
  summarise(
    mean_phi = mean(phi, na.rm = TRUE),
    sd_phi = sd(phi, na.rm = TRUE),
    n_boot = dplyr::n(),
    .groups = "drop"
  ) %>%
  mutate(
    variable = factor(variable, levels = variables),
    ci_lo = mean_phi - sd_phi,
    ci_hi = mean_phi + sd_phi
  )

df_pts <- df_boot %>% mutate(variable = factor(variable, levels = variables))

p <- ggplot(df_sum, aes(x = variable, y = mean_phi, fill = variable)) +
  geom_col(width = 0.6, colour = "grey20", linewidth = 0.3, alpha = 0.85) +
  geom_errorbar(aes(ymin = ci_lo, ymax = ci_hi), width = 0.2, linewidth = 0.8) +
  geom_jitter(data = df_pts, aes(x = variable, y = phi),
    width = 0.12, size = 2.8, alpha = 0.75, colour = "grey25",
    inherit.aes = FALSE) +
  geom_hline(yintercept = 0, colour = "grey40", linewidth = 0.4) +
  scale_fill_manual(values = pal_vars) +
  scale_y_continuous(expand = expansion(mult = c(0.10, 0.20)))

if (!is.null(shap_exact)) {
  df_exact <- tibble(
    variable = factor(names(shap_exact$shapley), levels = variables),
    phi_exact = as.numeric(shap_exact$shapley)
  )
  p <- p + geom_point(data = df_exact,
    aes(x = variable, y = phi_exact),
    shape = 18, size = 6, colour = "#d73027",
    inherit.aes = FALSE)
}

n_b <- dplyr::n_distinct(df_boot$boot_id)
p <- p +
  labs(
    title = sprintf("Shapley bootstrap – stabilité sur %d sous-échantillons", n_b),
    subtitle = "Barres = moyenne ± σ | points = tirages individuels | ♦ = Shapley exact (é",
    x = NULL,
    y = expression(phi ~ "(°C)")
  ) +
  theme_bw(base_size = 12) +
  theme(legend.position = "none",
    plot.title = element_text(face = "bold"),
    axis.text.x = element_text(face = "bold", size = 12))

dir.create(dirname(out_path), recursive = TRUE, showWarnings = FALSE)
save_plot(p, out_path, width = 8, height = 6)
cat(sprintf(" [Shapley bootstrap] PNG : %s\n", out_path))
invisible(p)
}

```

4.9 R/h2_analysis.R — Analyse H2 : forme verticale LAD

```
# =====
# Chapter 1 – H2 analysis: uniform vs real LAD
# =====

#' Pivot H2 results to wide format and compute per-plot, per-day difference.
#'
#' @param df_h2 Stacked H2 scenario results (from extract_all_scenarios).
#' @return Wide tibble with columns: x, y, date, Real, Uniform, diff.
summarise_h2 <- function(df_h2) {
  df_h2 %>% pivot_wider(id_cols = c(x, y, date), names_from = scenario, values_from = Delta_Tmax) %>%
    rename(Real = matches("real_LAD"), Uniform = matches("uniform_LAD")) %>% mutate(diff = Real - Uniform)
}

#' Density plot of ΔTmax distributions for real vs uniform LAD.
#'
#' @param df_h2 Stacked H2 scenario results.
#' @return ggplot object.
plot_h2_distributions <- function(df_h2) {
  ggplot(df_h2, aes(x = Delta_Tmax, fill = scenario, colour = scenario)) + geom_density(alpha = 0.5) +
    scale_fill_manual(values = c("#31688e", "#d8576b")) + scale_colour_manual(values = c("#31688e", "#d8576b")) +
    labs(title = "H2 – ΔTmax distributions: real vs uniform LAD", x = expression(Delta * T[max]))
}

#' Paired scatter plot of ΔTmax for two scenarios (hexbin + stats annotation).
#'
#' @param df_h2_wide Wide H2 tibble (x, y, date, + two scenario columns).
#' @param col_x Name of the x-axis scenario column (default "Real").
#' @param col_y Name of the y-axis scenario column (default "Uniform").
#' @param title Plot title.
#' @param subtitle Plot subtitle.
#' @return ggplot object.
plot_h2_paired <- function(df_h2_wide, col_x = "Real", col_y = "Uniform",
                           title = "H2 – Per-plot, per-day ΔTmax: real vs uniform LAD",
                           subtitle = "Paired comparison: impact of vertical profile SHAPE") {
  x_vals <- df_h2_wide[[col_x]]; y_vals <- df_h2_wide[[col_y]]
  r2_val <- cor(x_vals, y_vals, use = "complete.obs")^2
  rmse_val <- sqrt(mean((y_vals - x_vals)^2, na.rm = TRUE))
  bias_val <- mean(y_vals - x_vals, na.rm = TRUE)
  stats_label <- sprintf("R² = %.2f\nRMSE = %.2f°C\nBias = %.2f°C", r2_val, rmse_val, bias_val)
  axis_lims <- range(c(x_vals, y_vals), na.rm = TRUE)

  df_plot <- df_h2_wide; df_plot$.x <- x_vals; df_plot$.y <- y_vals
  ggplot(df_plot, aes(x = .x, y = .y)) +
    geom_hex(bins = 100) +
    scale_fill_gradient(low = "grey80", high = "midnightblue", trans = "log10", name = "Count") +
    geom_abline(slope = 1, intercept = 0, linetype = "dashed", colour = "black", linewidth = 1) +
    geom_smooth(method = "lm", colour = "#d73027", se = FALSE, linewidth = 1.2) +
    coord_fixed(xlim = axis_lims, ylim = axis_lims) +
    annotate("text", x = -Inf, y = Inf, label = stats_label,
            hjust = -0.1, vjust = 1.2, fontface = "bold", size = 5) +
    labs(title = title, subtitle = subtitle,
         x = bquote(. (col_x) ~ Delta * T[max] ~ (degree * C)),
         y = bquote(. (col_y) ~ Delta * T[max] ~ (degree * C))) +
    theme_bw(base_size = 14) +
    theme(legend.position = "right", plot.title = element_text(face = "bold"))
}

#' Split H2 differences by macroclimate regime (heatwave vs normal).
```

```

#'
#' @param df_h2_wide Wide H2 tibble with diff column.
#' @param df_macro Daily macroclimate.
#' @param threshold Tmax_macro threshold for heatwave (default 30°C).
#' @return Invisible ggplot (also prints summary table).
analyse_h2_distribution <- function(df_h2_wide, df_macro, threshold = 30) {
  df <- df_h2_wide %>% left_join(df_macro, by = "date") %>% mutate(period = ifelse(Tmax_macro >= threshold, "Heatwave", "Normal"))
  summ <- df %>% group_by(period) %>% summarise(n = n(), mean_diff = mean(diff, na.rm = TRUE))
  print(summ)
  p <- ggplot(df, aes(x = diff, fill = period)) + geom_histogram(bins = 80, alpha = 0.6, position = "dodge")
  scale_fill_manual(values = c(normal = "#31688e", heatwave = "#d8576b")) + labs(title = "H2 LAD shape effect")
  print(p)
  invisible(p)
}

#' Seasonal stability of the H2 LAD shape effect (diff by month and regime).
#'
#' Answers: is the  $\pm 1^\circ\text{C}$  real-vs-uniform effect stable across spring/summer/canicule,
#' or is it driven by a single extreme period?
#'
#' @param df_h2_wide Wide H2 tibble with diff column (Real - Uniform).
#' @param df_macro Daily macroclimate dataframe (date, Tmax_macro).
#' @param threshold Heatwave Tmax_macro threshold in °C (default 30).
#' @param out_path PNG output path.
#' @return Invisible patchwork ggplot.
plot_h2_seasonal <- function(df_h2_wide, df_macro, threshold = 30,
                             out_path = "outputs/figures/annex/A17_h2_seasonal.png") {
  df <- df_h2_wide %>%
    inner_join(df_macro %>% dplyr::select(date, Tmax_macro), by = "date") %>%
    mutate(
      month = factor(format(date, "%b"), levels = c("Jun", "Jul", "Aug", "Sep")),
      regime = factor(ifelse(Tmax_macro >= threshold, "Canicule", "Normal"),
                      levels = c("Normal", "Canicule"))
    )

  # Panel A: boxplot by month with mean annotations
  month_sum <- df %>%
    group_by(month) %>%
    summarise(
      mean_diff = mean(diff, na.rm = TRUE),
      n = dplyr::n(),
      .groups = "drop"
    )

  pA <- ggplot(df, aes(x = month, y = diff, fill = month)) +
    geom_boxplot(alpha = 0.5, outlier.shape = NA) +
    geom_hline(yintercept = 0, linetype = "dashed", colour = "grey30") +
    geom_text(
      data = month_sum,
      aes(x = month, y = 1.6,
          label = sprintf("moy=%.2f°C\nn=%d", mean_diff, n)),
      vjust = 0, size = 3.2, fontface = "bold", inherit.aes = FALSE
    ) +
    coord_cartesian(ylim = c(-2, 2)) +
    scale_fill_viridis_d(option = "mako", end = 0.85) +
    labs(
      title = "A. Effet LAD par mois (outliers hors  $\pm 2^\circ\text{C}$  masqués)",
      x = NULL,
      y = "Diff Real - Uniform  $\Delta T_{\text{max}}$  (°C)"
    ) +

```

```

theme_bw(base_size = 11) +
theme(legend.position = "none", plot.title = element_text(face = "bold"))

# Panel B: density by heatwave regime
regime_sum <- df %>%
  group_by(regime) %>%
  summarise(mean_diff = mean(diff, na.rm = TRUE), n = dplyr::n(), .groups = "drop")

pB <- ggplot(df, aes(x = diff, fill = regime, colour = regime)) +
  geom_density(alpha = 0.3, linewidth = 0.9) +
  geom_vline(data = regime_sum,
             aes(xintercept = mean_diff, colour = regime),
             linetype = "dashed", linewidth = 0.9) +
  geom_vline(xintercept = 0, colour = "grey40", linewidth = 0.5) +
  geom_label(
    data = regime_sum %>%
      mutate(y_pos = ifelse(regime == "Canicule", 3.5, 5.5)),
    aes(x = mean_diff, y = y_pos,
        label = sprintf("%s\n%.2f°C  n=%d", regime, mean_diff, n),
        colour = regime),
    fill = "white", label.size = 0.3, size = 3.2, fontface = "bold",
    inherit.aes = FALSE
  ) +
  coord_cartesian(xlim = c(-2, 2)) +
  scale_fill_manual(values = c(Normal = "#31688e", Canicule = "#d8576b")) +
  scale_colour_manual(values = c(Normal = "#31688e", Canicule = "#d8576b")) +
  labs(
    title = "B. Distribution Canicule vs Normal ( $\pm 2^\circ\text{C}$  affiché)",
    x = "Diff Real - Uniform  $\Delta T_{\text{max}}$  ( $^\circ\text{C}$ )",
    y = "Densité",
    fill = NULL, colour = NULL
  ) +
  theme_bw(base_size = 11) +
  theme(legend.position = "none", plot.title = element_text(face = "bold"))

subtitle_str <- if (all(c("Canicule", "Normal") %in% regime_sum$regime)) {
  sprintf("Effet moyen : Canicule = %.2f°C | Normal = %.2f°C",
          regime_sum$mean_diff[regime_sum$regime == "Canicule"],
          regime_sum$mean_diff[regime_sum$regime == "Normal"])
} else {
  paste(sprintf("%s = %.2f°C", regime_sum$regime, regime_sum$mean_diff), collapse = " | ")
}

fig <- pA + pB +
  patchwork::plot_annotation(
    title = "H2 - Stabilité saisonnière de l'effet profil LAD (Real - Uniform)",
    subtitle = subtitle_str,
    theme = theme(plot.title = element_text(face = "bold", size = 13),
                  plot.subtitle = element_text(colour = "grey30"))
  )

dir.create(dirname(out_path), recursive = TRUE, showWarnings = FALSE)
save_plot(fig, out_path, width = 13, height = 6)
cat(sprintf(" [h2_seasonal] PNG : %s\n", out_path))
invisible(fig)
}

#' Compute paired validation metrics between HOB0 observations and simulations.
#'
#' @param hobo_res List with element $daily (id_plot, date, scenario, Delta_obs, Delta_sim).

```

```

#' @return Tibble of per-scenario metrics: n, r2, rmse, mae, bias.
validate_hobo_paired <- function(hobo_res) {
  df <- hobo_res$daily
  key_counts <- df %>% count(id_plot, date, name = "n_sc")
  common_keys <- key_counts %>% filter(n_sc == length(unique(df$scenario)))
  df %>% inner_join(common_keys %>% select(id_plot, date), by = c("id_plot", "date")) %>% group_by(id_plot, date) %>% summarise(
    n = n(), r2 = cor(Delta_obs, Delta_sim, use = "complete.obs")^2, rmse = sqrt(mean((Delta_obs - Delta_sim)^2)),
    mae = mean(abs(Delta_obs - Delta_sim), na.rm = TRUE), bias = mean(Delta_sim - Delta_obs, na.rm = TRUE)
  )
}

#' Analyse la structure de la diff (Real - Uniform) par archétype et métriques structurelles
#'
#' Produit 4 livrables :
#' 1. Boxplot horizontal diff par archétype (PNG)
#' 2. Table heatwave x archétype (CSV + console)
#' 3. Scatter corrélation diff vs métriques structurelles (PNG + CSV)
#' 4. Histogramme amplitude temporelle de l'effet LAD par plot (PNG)
#'
#' @param df_h2_w Dataframe wide H2 (x, y, date, Real, Uniform, diff).
#' @param df_sample Dataframe cLHS (x, y, Cluster, LAI, Hmax, fCover, ...).
#' @param df_macro Dataframe macroclimat journalier (date, Tmax_macro, ...).
#' @param h_median_vec Vecteur numérique H_median par plot (même ordre que df_sample) ; NULL si non disponible.
#' @param fpc_scores Matrice nx3 des scores FPC1/2/3 ; NULL si non disponible.
#' @param out_dir Répertoire de sortie pour PNG et CSV.
#' @return Invisible list $per_plot, $cor_tbl, $tbl_cluster.
analyse_h2_structure <- function(df_h2_w, df_sample, df_macro,
                                h_median_vec = NULL, fpc_scores = NULL,
                                out_dir = "outputs/audit") {
  dir.create(out_dir, recursive = TRUE, showWarnings = FALSE)

  df_meta <- df_sample %>%
    dplyr::select(x, y, Cluster, LAI, Hmax, fCover) %>%
    mutate(Cluster = as.factor(Cluster))

  df_per_plot <- df_h2_w %>%
    group_by(x, y) %>%
    summarise(
      diff_mean = mean(diff, na.rm = TRUE),
      diff_q05 = quantile(diff, 0.05, na.rm = TRUE),
      diff_q95 = quantile(diff, 0.95, na.rm = TRUE),
      n_dates = n(),
      .groups = "drop"
    ) %>%
    inner_join(df_meta, by = c("x", "y")) %>%
    mutate(amplitude = diff_q95 - diff_q05)

  # Livrable 1 – Boxplot diff par cluster
  cl_stats <- df_per_plot %>%
    group_by(Cluster) %>%
    summarise(n = n(), median_diff = median(diff_mean, na.rm = TRUE), .groups = "drop")

  subtitle_1 <- paste(
    sprintf("Cl.%s : n=%d, méd.=%.3f°C",
      cl_stats$Cluster, cl_stats$n, cl_stats$median_diff),
    collapse = " | "
  )

  p1 <- ggplot(df_per_plot,
    aes(x = diff_mean, y = Cluster, fill = Cluster, colour = Cluster)) +

```

```

geom_boxplot(alpha = 0.4, outlier.shape = NA) +
geom_jitter(height = 0.2, alpha = 0.3, size = 1.5) +
geom_vline(xintercept = 0, linetype = "dashed", colour = "grey30") +
scale_fill_viridis_d(option = "turbo") +
scale_colour_viridis_d(option = "turbo") +
labs(
  title = "H2 – Structure de l'effet LAD par cluster (per-plot mean)",
  subtitle = subtitle_1,
  x = "diff Real - Uniform ΔTmax (°C)",
  y = "Cluster"
) +
theme(legend.position = "none")

print(p1)
ggsave(file.path(out_dir, "h2_boxplot_cluster.png"), p1,
  width = 10, height = 6, dpi = 150)

# Livrable 2 – Table heatwave × cluster
df_dates <- df_h2_w %>%
  left_join(df_macro %>% dplyr::select(date, Tmax_macro), by = "date") %>%
  mutate(period = ifelse(Tmax_macro >= 30, "heatwave", "normal")) %>%
  inner_join(df_meta %>% dplyr::select(x, y, Cluster), by = c("x", "y"))

tbl2 <- df_dates %>%
  group_by(Cluster, period) %>%
  summarise(
    n = n(),
    median_diff = median(diff, na.rm = TRUE),
    mean_diff = mean(diff, na.rm = TRUE),
    sd_diff = sd(diff, na.rm = TRUE),
    q05 = quantile(diff, 0.05, na.rm = TRUE),
    q95 = quantile(diff, 0.95, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(Cluster, period)

write.csv(tbl2, file.path(out_dir, "h2_structure_by_cluster.csv"), row.names = FALSE)
cat("\n— H2 Structure : diff par cluster × période —\n")
for (cl in levels(tbl2$Cluster)) {
  sub <- tbl2 %>% filter(Cluster == cl)
  cat(sprintf(" Cluster %s:\n", cl))
  for (i in seq_len(nrow(sub))) {
    cat(sprintf(
      " %-10s n=%5d méd=%+.3f°C moy=%+.3f°C sd=%+.3f°C [q05=%+.3f ; q95=%+.3f]\n",
      sub$period[i], sub$n[i], sub$median_diff[i],
      sub$mean_diff[i], sub$sd_diff[i], sub$q05[i], sub$q95[i]
    ))
  }
}
cat(sprintf(" → CSV : %s/h2_structure_by_cluster.csv\n", out_dir))

# Livrable 3 – Corrélation diff ↔ métriques structurelles
df_corr <- df_per_plot %>% dplyr::select(x, y, diff_mean, LAI, Hmax, fCover)

if (!is.null(h_median_vec) && length(h_median_vec) == nrow(df_sample)) {
  df_hmed <- df_sample %>%
    dplyr::select(x, y) %>%
    mutate(H_median = h_median_vec)
  df_corr <- df_corr %>% left_join(df_hmed, by = c("x", "y"))
}

```

```

if (!is.null(fpc_scores) && nrow(fpc_scores) == nrow(df_sample)) {
  df_fpc <- df_sample %>%
    dplyr::select(x, y) %>%
    mutate(FPC1 = fpc_scores[, 1],
           FPC2 = fpc_scores[, 2],
           FPC3 = fpc_scores[, 3])
  df_corr <- df_corr %>% left_join(df_fpc, by = c("x", "y"))
}

metric_cols <- setdiff(names(df_corr), c("x", "y", "diff_mean"))

cor_rows <- lapply(metric_cols, function(m) {
  x_v <- df_corr[[m]]
  y_v <- df_corr$diff_mean
  ok <- complete.cases(x_v, y_v)
  if (sum(ok) < 5) return(NULL)
  data.frame(
    metric = m,
    pearson = cor(x_v[ok], y_v[ok], method = "pearson"),
    spearman = cor(x_v[ok], y_v[ok], method = "spearman"),
    n = sum(ok)
  )
})
df_cor_tbl <- bind_rows(cor_rows)
write.csv(df_cor_tbl, file.path(out_dir, "h2_diff_correlations.csv"), row.names = FALSE)

annot_df <- df_cor_tbl %>% mutate(label = sprintf("r = %.2f", pearson))

df_long <- df_corr %>%
  pivot_longer(cols = all_of(metric_cols), names_to = "metric", values_to = "metric_val")
  left_join(annot_df %>% dplyr::select(metric, label), by = "metric")

p3 <- ggplot(df_long, aes(x = metric_val, y = diff_mean)) +
  geom_point(alpha = 0.4, size = 1.5, colour = "#31688e") +
  geom_smooth(method = "lm", se = TRUE, colour = "#d8576b",
             fill = "#d8576b", alpha = 0.15) +
  geom_text(
    data = df_long %>% group_by(metric, label) %>% slice(1),
    aes(x = -Inf, y = Inf, label = label),
    hjust = -0.1, vjust = 1.3, fontface = "bold", size = 3.5
  ) +
  facet_wrap(~ metric, scales = "free_x") +
  labs(
    title = "H2 – Diff LAD (real-uniform) vs métriques structurelles par plot",
    x = "Valeur de la métrique",
    y = "diff mean ΔTmax (°C)"
  )

print(p3)
ggsave(file.path(out_dir, "h2_scatter_metrics.png"), p3,
       width = 12, height = 8, dpi = 150)
cat(sprintf(" → CSV corrélations : %s/h2_diff_correlations.csv\n", out_dir))

# Livrable 4 – Distribution amplitude temporelle
pct_gtl <- 100 * mean(df_per_plot$amplitude > 1, na.rm = TRUE)
med_amp <- median(df_per_plot$amplitude, na.rm = TRUE)

p4 <- ggplot(df_per_plot, aes(x = amplitude, fill = Cluster)) +
  geom_histogram(alpha = 0.5, position = "identity", bins = 40) +
  geom_vline(xintercept = med_amp, linetype = "dashed",

```

```

        colour = "black", linewidth = 1) +
scale_fill_viridis_d(option = "turbo") +
annotate("text", x = Inf, y = Inf,
        label = sprintf("%.1f%%\nplots amplitude > 1°C", pct_gt1),
        hjust = 1.1, vjust = 1.3, fontface = "bold", size = 4) +
labs(
  title = "H2 – Amplitude temporelle de l'effet LAD par plot (q95 - q05)",
  subtitle = sprintf("Médiane globale = %.2f°C | %.1f%% plots avec amplitude > 1°C",
                    med_amp, pct_gt1),
  x = "Amplitude de l'effet LAD (q95 - q05) (°C)",
  y = "Nombre de plots"
)

print(p4)
ggsave(file.path(out_dir, "h2_amplitude_histogram.png"), p4,
        width = 10, height = 6, dpi = 150)

cat(sprintf("\n[H2 structure] 5 livrables produits dans %s/\n", out_dir))
invisible(list(per_plot = df_per_plot, cor_tbl = df_cor_tbl,
               tbl_cluster = tbl2))
}

```

4.10 R/gamm.R — Émulateur GAMM

```

# =====
# Chapter 1 – GAMM emulator (structure + macroclimate)
# =====

#' Prepare scaled inputs for the GAMM, integrating macroclimate and shape drivers.
#'
#' @param df_daily      Daily scenario results for the reference scenario.
#' @param df_sample     cLHS sample dataframe.
#' @param fpc_scores    Optional FPC score matrix (n_plots × n_fpc).
#' @param h_median_vec  Optional H_median vector (length = nrow(df_sample)).
#' @return Dataframe ready for fit_reference_gamm, with scaled predictors.
prepare_gamm_data <- function(df_daily, df_sample, fpc_scores = NULL, h_median_vec = NULL) {
  df <- df_daily %>%
    mutate(plot_id = as.factor(paste0("X", x, "_Y", y)),
           date_factor = as.factor(date))

  if (!is.null(h_median_vec)) {
    df_sample$H_median <- h_median_vec
    df <- df %>% left_join(df_sample %>% dplyr::select(x, y, H_median), by = c("x", "y"))
  }

  if ("Cluster" %in% names(df)) df$Cluster <- as.factor(df$Cluster)

  scale_cols <- c("LAI", "Hmax", "fCover", "H_median", "Tmax_macro",
                 "Wind_mean", "Rad_mean", "VPD_mean", "Rain_sum")
  cols_to_scale <- intersect(scale_cols, names(df))
  for (nm in cols_to_scale) df[[paste0(nm, "_sc")]] <- as.numeric(scale(df[[nm]]))

  if (!is.null(fpc_scores)) {
    fpc_df <- as.data.frame(fpc_scores)
    names(fpc_df) <- paste0("FPC", seq_len(ncol(fpc_df)), "_sc")
    fpc_df[] <- lapply(fpc_df, function(v) as.numeric(scale(v)))
    fpc_df$x <- df_sample$x; fpc_df$y <- df_sample$y
    df <- df %>% left_join(fpc_df, by = c("x", "y"))
  }
}

```

```

}
return(df)
}

#' Fit the reference GAMM with toggles for macroclimate and shape predictor.
#'
#' Uses bam() with scat() family and fREML for efficiency on large datasets.
#'
#' @param df_gamm      Dataframe from prepare_gamm_data.
#' @param shape_type    Shape predictor: "CLUSTER", "FPC", "H_MEDIAN", or "NONE".
#' @param use_macroclimate Include ERA5 physical drivers (default TRUE).
#' @return Fitted bam model object.
fit_reference_gamm <- function(df_gamm, shape_type = "NONE", use_macroclimate = TRUE) {
  base_terms <- c("s(LAI_sc)", "s(Hmax_sc)", "s(fCover_sc)")

  clim_terms <- if (use_macroclimate) {
    cat(" [GAMM] Including macroclimate drivers (Tmax, Wind, Rad, VPD, Rain)...\n")
    c("s(Tmax_macro_sc)", "s(Wind_mean_sc)", "s(Rad_mean_sc)", "s(VPD_mean_sc)", "s(Rain_sum")
  } else {
    cat(" [GAMM] Structural drivers only (Macroclimate excluded)...\n")
    character(0)
  }

  shape_terms <- switch(shape_type,
    "CLUSTER" = {
      cat(" [GAMM] Including profile Cluster type as categorical predictor...\n")
      c("Cluster") # terme paramétrique factoriel
    },
    "FPC" = {
      cat(" [GAMM] Including FPC scores as shape predictors...\n")
      c("s(FPC1_sc)", "s(FPC2_sc)", "s(FPC3_sc)")
    },
    "H_MEDIAN" = {
      cat(" [GAMM] Including structural shape (H_median) as predictor...\n")
      c("s(H_median_sc)")
    },
    {
      cat(" [GAMM] Structural macro-metrics only (no specific shape variables)...\n")
      character(0)
    }
  )

  re_terms <- c("s(date_factor, bs='re')", "s(plot_id, bs='re')")
  rhs <- paste(c(base_terms, clim_terms, shape_terms, re_terms), collapse = " + ")
  form <- as.formula(paste("Delta_Tmax ~", rhs))

  bam(form, data = df_gamm, family = scat(), method = "fREML", discrete = TRUE)
}

#' Plot marginal effects of the GAMM predictors.
#'
#' @param gam_model      Fitted bam/gam model.
#' @param shape_type      Shape predictor type used (see fit_reference_gamm).
#' @param use_macroclimate Whether macroclimate terms are present.
#' @return patchwork ggplot.
plot_gamm_marginal_effects <- function(gam_model, shape_type = "NONE", use_macroclimate = TRUE) {
  terms_to_plot <- c("LAI_sc", "Hmax_sc")

  if (shape_type == "CLUSTER") terms_to_plot <- c(terms_to_plot, "Cluster")
  if (shape_type == "FPC") terms_to_plot <- c(terms_to_plot, "FPC1_sc")

```

```

if (shape_type == "H_MEDIAN") terms_to_plot <- c(terms_to_plot, "H_median_sc")
if (use_macroclimate) terms_to_plot <- c(terms_to_plot, "VPD_mean_sc", "Wind_mean_sc", "Rad_mean_sc")

plots <- lapply(terms_to_plot, function(term) {
  pred <- ggpredict(gam_model, terms = term)
  plot(pred) +
    labs(title = paste("Effect of", gsub("_sc", "", term)),
         x = if (term == "Cluster") "Profile cluster type" else paste(gsub("_sc", "", term), "smooth"),
         y = "Predicted ΔTmax (°C)") +
    theme_bw() + theme(plot.title = element_text(size = 10))
})

wrap_plots(plots, ncol = 2) + plot_annotation(title = "GAMM Marginal Effects", subtitle = "Reference GAMM")
}

#' Residual diagnostics for the reference GAMM (QQ, histogram, fitted vs residuals, AIC).
#'
#' @param gam_scatter Fitted scatter() bam model.
#' @param df_gamm Data used to fit the model.
#' @param plot Whether to produce base-graphics plots (default TRUE).
#' @return Dataframe with diagnostic statistics.
diagnose_residuais <- function(gam_scatter, df_gamm, plot = TRUE) {
  gam_gauss <- bam(formula(gam_scatter), data = df_gamm, family = gaussian(), method = "fREML",
    resid_g <- residuals(gam_gauss, type = "deviance")
  idx_sw <- sample(length(resid_g), min(5000, length(resid_g))); sw_test <- shapiro.test(resid_g[idx_sw])
  excess_kurt <- mean((resid_g - mean(resid_g))^4) / sd(resid_g)^4 - 3
  aic_g <- AIC(gam_gauss); aic_s <- AIC(gam_scatter)

  if (plot) {
    op <- par(mfrow = c(2, 2)); on.exit(par(op))
    qqnorm(resid_g, main = "QQ-plot (Gaussian residuals)", pch = 16, cex = 0.3, col = rgb(0, 0, 0, 0.3))
    hist(resid_g, breaks = 80, freq = FALSE, main = "Residual distribution", xlab = "Residuals", ylab = "Density")
    plot(fitted(gam_gauss), resid_g, pch = 16, cex = 0.3, col = rgb(0, 0, 0, 0.3), xlab = "Fitted values", ylab = "Residuals")
    barplot(c(Gaussian = aic_g, `scatter` = aic_s), col = c("lightblue", "salmon"), main = "AIC")
  }

  data.frame(n_obs = length(resid_g), shapiro_W = unname(sw_test$statistic), shapiro_p = sw_test$p.value)
}

#' Diagnostic de concurvit  pour le GAMM de r f rence.
#'
#' Teste la concurvit  globale (full=TRUE) et paire- -paire (full=FALSE) pour
#' les termes de for age macroclimatique (Tmax, Rad, VPD, Wind) physiquement
#' corr l s. L ve une alerte console si le worst pairwise > 0.8 sur ce groupe.
#' Sauvegarde deux CSV dans out_dir.
#'
#' @param gam_model Mod le bam/gam ajust  (sorti de fit_reference_gamm).
#' @param out_dir R pertoire de sortie pour les CSV.
#' @return Invisible list $full et $pairwise.
run_concurvity_audit <- function(gam_model, out_dir = "outputs/audit") {
  dir.create(out_dir, recursive = TRUE, showWarnings = FALSE)

  conc_full <- mgcv::concurvity(gam_model, full = TRUE)
  conc_pair <- mgcv::concurvity(gam_model, full = FALSE)

  cat("\n  Concurvity audit (full = TRUE)   worst-case par terme smooth  \n")
  print(round(conc_full, 3))

  cat("\n  Concurvity audit (full = FALSE)   matrice pairwise worst  \n")
  worst_mat <- conc_pair[["worst"]]
  print(worst_mat)
}

```

```

print(round(worst_mat, 3))

clim_terms <- c("s(Tmax_macro_sc)", "s(Rad_mean_sc)", "s(VPD_mean_sc)", "s(Wind_mean_sc)")
terms_found <- intersect(clim_terms, rownames(worst_mat))

if (length(terms_found) >= 2) {
  sub_mat <- worst_mat[terms_found, terms_found, drop = FALSE]
  diag(sub_mat) <- NA
  max_conc <- max(sub_mat, na.rm = TRUE)

  if (max_conc > 0.8) {
    cat(sprintf(
      "\n[!] CONCURVITE ELEVEE : worst pairwise macroclimat = %.3f > 0.8\n",
      max_conc
    ))
    cat("  Termes concernés :", paste(terms_found, collapse = ", "), "\n")
    cat("  → Envisager de retirer un terme redondant ou d'utiliser",
      "un tenseur produit ti() pour contrôler la collinéarité.\n\n")
  } else {
    cat(sprintf(
      "\n[ok] Concurvité acceptable : worst pairwise macroclimat = %.3f ≤ 0.8\n\n",
      max_conc
    ))
  }
}

write.csv(as.data.frame(conc_full),
  file.path(out_dir, "concurvity_full.csv"), row.names = TRUE)
write.csv(as.data.frame(worst_mat),
  file.path(out_dir, "concurvity_pairwise_worst.csv"), row.names = TRUE)
cat(sprintf(" [concurvity] CSV sauvegardés → %s/\n", out_dir))

invisible(list(full = conc_full, pairwise = conc_pair))
}

```

4.11 R/validation.R — Validation aux capteurs HOBO

```

# =====
# Chapter 1 – HOBO validation: diagnostics and per-scenario validation
# =====

#' Select 3 representative HOBO sensors by buffering capacity.
#'
#' Returns the sensor with the strongest cooling, the median, and the weakest.
#'
#' @param df_hobo_daily Daily HOBO data with id_plot and Delta_obs columns.
#' @return Tibble with 3 rows and columns id_plot, mean_delta, rationale, facet_label.
pick_representative_hobos <- function(df_hobo_daily) {
  df_summary <- df_hobo_daily %>% group_by(id_plot) %>% summarise(mean_delta = mean(Delta_obs))
  n_total <- nrow(df_summary)
  res <- bind_rows(
    df_summary %>% slice(1) %>% mutate(rationale = "Strongest cooling (Max buffer)"),
    df_summary %>% slice(ceiling(n_total / 2)) %>% mutate(rationale = "Median cooling"),
    df_summary %>% slice(n_total) %>% mutate(rationale = "Weakest cooling (Min buffer)")
  ) %>% mutate(facet_label = sprintf("HOBO %s\n[%s : Mean ΔTmax = %.1f°C]", id_plot, rationale))
  return(res)
}

```

```

#' Time-series overlay for 3 selected HOB0s: HOB0 obs, MuSICA real, MuSICA uniform, ERA5.
#'
#' @param df_selected_hobos Tibble from pick_representative_hobos.
#' @param df_hobo           Full daily HOB0 dataset.
#' @param df_musica_real    MuSICA results for real LAD scenario.
#' @param df_musica_uniform MuSICA results for uniform LAD scenario.
#' @param df_macro         Daily macroclimate (ERA5).
#' @return ggplot object.
plot_timeseries_faceted <- function(df_selected_hobos, df_hobo, df_musica_real, df_musica_uni
  ids <- df_selected_hobos$id_plot
  df1 <- df_hobo %>% filter(id_plot %in% ids) %>% transmute(id_plot, date, value = Tmax_obs,
  df2 <- df_musica_real %>% filter(id_plot %in% ids) %>% transmute(id_plot, date, value = Tm
  df3 <- df_musica_uniform %>% filter(id_plot %in% ids) %>% transmute(id_plot, date, value =
  df4 <- tidyr::expand_grid(id_plot = ids, df_macro) %>% transmute(id_plot, date, value = Tm

  df_all <- bind_rows(df1, df2, df3, df4) %>% inner_join(df_selected_hobos %>% dplyr::select

  ggplot(df_all, aes(x = date, y = value, colour = source)) + geom_line(linewidth = 0.6, alp
    scale_colour_manual(values = c("HOB0" = "black", "MuSICA real LAD" = "#31688e", "MuSICA
    labs(title = "Time-series – Selected HOB0s vs MuSICA", subtitle = "Plots chosen by their
}

#' Extract structural covariates for HOB0 sensor locations from LiDAR rasters.
#'
#' @param hobo_geojson Path to the HOB0 GeoJSON file.
#' @param r_stack      LiDAR raster stack (from load_lidar_rasters).
#' @param ids_to_remove HOB0 IDs to exclude.
#' @return Dataframe with id_plot and structural covariates.
build_hobo_inputs <- function(hobo_geojson, r_stack, ids_to_remove) {
  hobo_pts <- st_read(hobo_geojson, quiet = TRUE)
  ext_vals <- terra::extract(r_stack, vect(hobo_pts), xy = TRUE)
  hobo_pts %>% st_drop_geometry() %>% dplyr::select(id_plot) %>% bind_cols(ext_vals) %>% fil
}

#' Extract daily ΔTmax from MuSICA outputs at HOB0 sensor locations.
#'
#' @param scenario_dir Directory with HOB0 NetCDF outputs for one scenario.
#' @param df_macro     Daily macroclimate.
#' @param date_seq     Dates of interest.
#' @return Tibble with id_plot, scenario columns added.
extract_deltatmax_hobo_scenario <- function(scenario_dir, df_macro, date_seq) {
  nc_files <- list.files(scenario_dir, pattern = "\\\\.nc$", full.names = TRUE); if (length(nc
  map_df(nc_files, function(f) {
    id_str <- str_extract(basename(f), "(?<=HOB0_).*(?=\\.nc)"); res <- extract_deltatmax_on
    res %>% mutate(id_plot = id_str, scenario = basename(scenario_dir))
  })
}

#' Run MuSICA at HOB0 locations for a list of scenarios and compute validation metrics.
#'
#' @param df_hobo_inputs Dataframe from build_hobo_inputs.
#' @param df_hobo_daily  Observed daily HOB0 data.
#' @param scenarios      Named list of scenario lists.
#' @param parent_dir     Parent output directory.
#' @param df_macro       Daily macroclimate.
#' @param date_seq       Dates of interest.
#' @param force          Re-run existing outputs (default FALSE).
#' @return List with elements $daily (paired obs/sim) and $metrics (per-scenario stats).
validate_scenarios_at_hobos <- function(df_hobo_inputs, df_hobo_daily, scenarios, parent_dir
  dir.create(parent_dir, recursive = TRUE, showWarnings = FALSE)

```

```

hobo_paths <- character(length(scenarios)); names(hobo_paths) <- names(scenarios)
for (nm in names(scenarios)) {
  sc <- scenarios[[nm]]; out_d <- file.path(parent_dir, sc$name); dir.create(out_d, recurs
  existing <- list.files(out_d, pattern = "\\nc$")
  if (!force && length(existing) >= nrow(df_hobo_inputs)) { hobo_paths[nm] <- out_d; next
  cat(sprintf("\n[HOB0 %s] %d HOB0 plots → %s\n", sc$name, nrow(df_hobo_inputs), out_d))
  for (i in seq_len(nrow(df_hobo_inputs))) {
    plot_row <- df_hobo_inputs[i, ]; sim_id <- sprintf("HOB0_%s", plot_row$id_plot)
    out_file <- file.path(out_d, paste0("musica_out_", sim_id, ".nc")); if (file.exists(out
    run_musica_one(plot_row, sc, out_file, CFG$forcing_file, CFG$musica_cmd)
  }
  hobo_paths[nm] <- out_d
}
df_sims <- map_df(hobo_paths, extract_deltatmax_hobo_scenario, df_macro = df_macro, date_s
df_obs <- df_hobo_daily %>% dplyr::select(id_plot, date, Delta_obs)
df_daily <- df_sims %>% inner_join(df_obs, by = c("id_plot", "date")) %>% rename(Delta_sim
df_metrics <- df_daily %>% group_by(scenario) %>% summarise(n = n(), r2 = cor(Delta_obs, D
list(daily = df_daily, metrics = df_metrics)
}

#' Bar chart of RMSE per scenario at HOB0 locations, coloured by bias.
#'
#' @param df_metrics Tibble from validate_scenarios_at_hobos$metrics.
#' @param title Plot title.
#' @return ggplot object.
plot_hobo_validation <- function(df_metrics, title = "HOB0 validation – RMSE per scenario")
  df_metrics %>%
  mutate(label = sprintf("R²=%.2f bias=%+.2f°C", r2, bias)) %>%
  ggplot(aes(x = reorder(scenario, rmse), y = rmse)) +
  geom_col(fill = "#31688e", alpha = 0.8) +
  geom_text(aes(label = label), hjust = -0.05, size = 3.2, colour = "grey20") +
  coord_flip() +
  expand_limits(y = max(df_metrics$rmse) * 1.35) +
  labs(title = title, x = NULL, y = "RMSE vs observed HOB0 ΔTmax (°C)") +
  theme_bw(base_size = 11) +
  theme(plot.title = element_text(face = "bold"))
}

#' Scatter ΔTmax(sim) vs ΔTmax(obs HOB0) coloured by structural cluster.
#'
#' Key figure: the 3D canopy typology (Cluster) should predict the rank order of
#' cooling refuges without necessarily predicting the absolute amplitude.
#' Faceted by heatwave / normal regime.
#'
#' @param hobo_res List from validate_scenarios_at_hobos ($daily has id_plot, date, De
#' @param df_hobo_inputs Dataframe from build_hobo_inputs (id_plot, x, y).
#' @param df_forest Full forest dataframe with Cluster column (x, y, Cluster).
#' @param scenario_name Scenario to extract from hobo_res$daily (default "REF_all_real").
#' @param threshold Heatwave Tmax_macro threshold in °C (default 30).
#' @param out_path PNG output path.
#' @return Invisible ggplot.
plot_hobo_sim_obs_by_cluster <- function(hobo_res, df_hobo_inputs, df_forest,
  scenario_name = "REF_all_real",
  threshold = 30,
  out_path = "outputs/figures/annex/A19_hobo_cl

# Assign each HOB0 sensor to the nearest forest grid cell's cluster
df_forest_xy <- df_forest %>%
  dplyr::select(x, y, Cluster) %>%
  mutate(Cluster = as.character(Cluster))

```

```

df_hobo_cluster <- df_hobo_inputs %>%
  dplyr::select(id_plot, x, y) %>%
  mutate(
    Cluster = purrr::map2_chr(x, y, function(hx, hy) {
      dists <- (df_forest_xy$x - hx)^2 + (df_forest_xy$y - hy)^2
      df_forest_xy$Cluster[which.min(dists)]
    })
  ) %>%
  dplyr::select(id_plot, Cluster)

# hobo_res$daily already carries Tmax_macro from extract_deltatmax_one;
# a second join would produce Tmax_macro.x/.y and break the mutate below.
df_daily <- hobo_res$daily %>%
  filter(scenario == scenario_name) %>%
  inner_join(df_hobo_cluster, by = "id_plot") %>%
  mutate(
    regime = factor(ifelse(Tmax_macro >= threshold, "Canicule", "Normal"),
      levels = c("Normal", "Canicule")),
    Cluster = factor(Cluster)
  )

if (nrow(df_daily) == 0) {
  warning(sprintf("[plot_hobo_sim_obs_by_cluster] no data for scenario '%s'", scenario_name))
  return(invisible(NULL))
}

lims <- range(c(df_daily$Delta_obs, df_daily$Delta_sim), na.rm = TRUE)

df_stats <- df_daily %>%
  group_by(regime) %>%
  summarise(
    r2 = cor(Delta_obs, Delta_sim, use = "complete.obs")^2,
    rmse = sqrt(mean((Delta_obs - Delta_sim)^2, na.rm = TRUE)),
    bias = mean(Delta_sim - Delta_obs, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  mutate(
    label = sprintf("R²=%.2f\nRMSE=%.2f°C\nBias=%+.2f°C", r2, rmse, bias),
    x_pos = lims[1], y_pos = lims[2]
  )

p <- ggplot(df_daily, aes(x = Delta_obs, y = Delta_sim,
  colour = Cluster, shape = Cluster)) +
  geom_point(alpha = 0.7, size = 2.5) +
  geom_smooth(method = "lm", se = FALSE, linewidth = 1.1,
    aes(group = 1), colour = "grey30") +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed",
    colour = "black", linewidth = 0.7) +
  geom_text(data = df_stats,
    aes(x = x_pos, y = y_pos, label = label),
    hjust = -0.05, vjust = 1.1, fontface = "bold", size = 3.2,
    inherit.aes = FALSE) +
  coord_fixed(xlim = lims, ylim = lims) +
  facet_wrap(~ regime) +
  scale_colour_viridis_d(option = "turbo", name = "Cluster") +
  scale_shape_manual(values = 15:18, name = "Cluster") +
  labs(
    title = sprintf("Validation HOB0 - ΔTmax sim vs obs | Scénario : %s", scenario_name),
    subtitle = "Coloré par cluster structural - la 3D classe les refuges sans prédire l'am",
    x = expression("HOB0 obs" ~ Delta * T[max] ~ (degree*C)),

```

```

    y      = expression("MuSICA sim" ~ Delta * T[max] ~ (degree*C))
  ) +
  theme_bw(base_size = 12) +
  theme(
    legend.position = "bottom",
    plot.title      = element_text(face = "bold"),
    strip.text      = element_text(face = "bold", size = 11)
  )

  dir.create(dirname(out_path), recursive = TRUE, showWarnings = FALSE)
  save_plot(p, out_path, width = 12, height = 6)
  cat(sprintf(" [hobo_cluster_scatter] PNG : %s\n", out_path))
  invisible(p)
}

```

4.12 R/io.R — Entrées/sorties (forcing ERA5, HOBO CSV)

```

# =====
# Chapter 1 – I/O helpers
# =====

#' Force UTC timezone on a NetCDF time variable.
#'
#' @param ncfile Path to the NetCDF file.
#' @param varname Name of the time variable (default "time").
#' @return POSIXct vector in UTC.
force_utc_nc <- function(ncfile, varname = "time") {
  if (!file.exists(ncfile)) stop(paste("File not found:", ncfile))
  nc <- nc_open(ncfile); on.exit(nc_close(nc))
  time_val <- ncvar_get(nc, varname)
  units_str <- ncatt_get(nc, varname, "units")$value
  parts <- strsplit(units_str, " since ")[[1]]
  unit_type <- parts[1]
  origin_str <- gsub("\\s*\\(.*\\)| UTC", "", parts[2])
  origin_dt <- as.POSIXct(origin_str, tz = "UTC", format = "%Y-%m-%d %H:%M:%S")
  mult <- switch(trimws(unit_type),
    "hours" = 3600, "minutes" = 60,
    "seconds" = 1, "days" = 86400, 3600)
  with_tz(origin_dt + (time_val * mult), "UTC")
}

#' Load and aggregate the LiDAR rasters.
#'
#' @param in_dir Directory containing the .tif files.
#' @param agg_factor Aggregation factor (default 2 = 10 m → 20 m).
#' @return List with elements $metrics, $lad, $stack.
load_lidar_rasters <- function(in_dir, agg_factor = 2) {
  r_lai <- rast(file.path(in_dir, "lai_z1_res_10_m.tif"))
  r_vci <- rast(file.path(in_dir, "vci_res_10_m.tif"))
  r_hmax <- rast(file.path(in_dir, "max_res_10_m.tif"))
  r_fcover <- rast(file.path(in_dir, "fCover_res_10_m.tif"))
  r_lad_st <- rast(file.path(in_dir, "lad_profiles_z1_res_10_m.tif"))

  r_metrics <- terra::aggregate(
    c(r_lai, r_vci, r_hmax, r_fcover),
    fact = agg_factor, fun = "mean", na.rm = TRUE
  )
  names(r_metrics) <- c("LAI", "VCI", "Hmax", "fCover")
}

```

```

r_lad_agg <- terra::aggregate(r_lad_st, fact = agg_factor, fun = "mean", na.rm = TRUE)

list(metrics = r_metrics, lad = r_lad_agg, stack = c(r_metrics, r_lad_agg))
}

#' Extract daily macroclimate drivers from ERA5 forcing.
#'
#' Computes Tmax, Wind, Rad, VPD, Rain aggregated to daily resolution.
#'
#' @param forcing_file Path to the ERA5 NetCDF forcing file.
#' @param date_seq Date sequence to filter on.
#' @return Daily tibble with columns: date, Tmax_macro, Wind_mean, Rad_mean, VPD_mean, Rain_
extract_macro_daily <- function(forcing_file, date_seq) {
  nc <- nc_open(forcing_file)

  get_var <- function(v) {
    val <- try(ncvar_get(nc, v), silent = TRUE)
    if (inherits(val, "try-error")) return(rep(NA, nc$dim$time$len))
    return(val)
  }

  macro_tair <- get_var("Tair")
  macro_qair <- get_var("Qair")
  macro_psurf <- get_var("PSurf")
  macro_wind_n <- get_var("Wind_N")
  macro_wind_e <- get_var("Wind_E")
  macro_sdown <- get_var("SWdown")
  macro_rain <- get_var("Rainf")

  nc_close(nc)
  macro_time <- force_utc_nc(forcing_file, "time")

  t_celsius <- macro_tair - 273.15
  wind_abs <- sqrt(macro_wind_n^2 + macro_wind_e^2)

  e_s <- 611.2 * exp((17.67 * t_celsius) / (t_celsius + 243.5))
  e_a <- (macro_qair * macro_psurf) / (0.622 + 0.378 * macro_qair)
  vpd <- pmax(e_s - e_a, 0)

  df_hourly <- data.frame(
    time = macro_time,
    Tair_macro = t_celsius,
    Wind_macro = wind_abs,
    Rad_macro = macro_sdown,
    VPD_macro = vpd,
    Rain_macro = macro_rain
  )

  df_hourly %>%
    filter(as.Date(time) %in% date_seq) %>%
    mutate(date = as.Date(time)) %>%
    group_by(date) %>%
    summarise(
      Tmax_macro = max(Tair_macro, na.rm = TRUE),
      Wind_mean = mean(Wind_macro, na.rm = TRUE),
      Rad_mean = mean(Rad_macro, na.rm = TRUE),
      VPD_mean = mean(VPD_macro, na.rm = TRUE),
      Rain_sum = sum(Rain_macro * 3600, na.rm = TRUE),
      .groups = "drop"
    )

```

```

}

#' Save a ggplot to disk and return the path invisibly.
#'
#' @param p      ggplot object.
#' @param path    Output file path (PNG).
#' @param width   Width in inches (default 10).
#' @param height  Height in inches (default 7).
#' @param dpi     Resolution (default 150).
#' @return Invisible path character.
save_plot <- function(p, path, width = 10, height = 7, dpi = 150) {
  dir.create(dirname(path), recursive = TRUE, showWarnings = FALSE)
  ggsave(path, p, width = width, height = height, dpi = dpi)
  invisible(path)
}

#' Create the standard outputs/ sub-directory tree.
#'
#' @param root Root directory (default "outputs").
#' @return Invisible root path.
setup_outputs_dirs <- function(root = "outputs") {
  for (sd in c("clusters", "h2", "h1", "fpca", "gamm", "hobo", "s2_annex", "audit",
               "figures", "figures/annex")) {
    dir.create(file.path(root, sd), recursive = TRUE, showWarnings = FALSE)
  }
  invisible(root)
}

#' Read HOB0 sub-canopy temperatures and compute daily Delta_obs vs ERA5.
#'
#' @param csv_file    Path to the HOB0 temperature CSV.
#' @param date_seq    Date sequence for filtering.
#' @param df_macro    Daily macroclimate dataframe (from extract_macro_daily).
#' @param ids_to_remove Character vector of HOB0 plot IDs to exclude.
#' @return Daily tibble with columns: id_plot, date, Tmax_obs, Delta_obs.
read_hobo_daily <- function(csv_file, date_seq, df_macro, ids_to_remove) {
  read.csv(csv_file) %>%
    mutate(
      datetime = as.POSIXct(datetime, format = "%Y-%m-%d %H:%M:%S", tz = "UTC"),
      date      = as.Date(datetime)
    ) %>%
    filter(position_sensor == "a", date %in% date_seq,
           !(id_plot %in% ids_to_remove)) %>%
    group_by(id_plot, date) %>%
    summarise(Tmax_obs = max(t_hobo, na.rm = TRUE), .groups = "drop") %>%
    inner_join(df_macro, by = "date") %>%
    mutate(Delta_obs = Tmax_obs - Tmax_macro)
}

```

4.13 R/profiles.R — Profils verticaux MuSICA

```

# =====
# Chapter 1 – Vertical temperature profiles (H2 – by cluster median)
#
# NetCDF MuSICA structure:
#   Tair_z      : dims [nair=15 × time]
#   relative_height : vector [nair] of relative heights (0→1)
#   veget_height_top: absolute canopy height (time series, constant in practice)

```

```

# Absolute Z axis = relative_height × veget_height_top[1]
# =====

#' Extraire le profil vertical de Tmax pour un NetCDF MuSICA.
#'
#' @param nc_file Chemin vers le fichier NetCDF.
#' @param date_seq Dates d'intérêt (filtre sur l'été).
#' @param summary "mean_summer" | "canicule_date" | "median_date"
#' @return Tibble (height_m, Tmax_z) ou NULL si échec.
extract_vertical_tmax_profile <- function(nc_file, date_seq,
                                         summary = "mean_summer") {
  if (is.na(nc_file) || !file.exists(nc_file)) return(NULL)

  nc <- try(nc_open(nc_file), silent = TRUE)
  if (inherits(nc, "try-error")) return(NULL)

  rel_h <- try(ncvar_get(nc, "relative_height"), silent = TRUE)
  h_top <- try(ncvar_get(nc, "veget_height_top"), silent = TRUE)
  raw <- try(get_variable(nc, "Tair_z"), silent = TRUE)
  nc_close(nc)

  if (inherits(raw, "try-error") || is.null(raw)) return(NULL)

  hmax_nc <- if (!inherits(h_top, "try-error")) h_top[1] else NA_real_

  df <- raw %>%
    mutate(
      Tair_sim = Tair_z - 273.15,
      time = time - hours(2),
      date = as.Date(time)
    ) %>%
    filter(date %in% date_seq) %>%
    group_by(nair, date) %>%
    summarise(Tmax_daily = max(Tair_sim, na.rm = TRUE), .groups = "drop")

  df_out <- switch(summary,
    "mean_summer" = df %>%
      group_by(nair) %>%
      summarise(Tmax_z = mean(Tmax_daily, na.rm = TRUE), .groups = "drop"),

    "canicule_date" = {
      d_max <- df %>%
        group_by(date) %>%
        summarise(m = max(Tmax_daily, na.rm = TRUE), .groups = "drop") %>%
        slice_max(m, n = 1, with_ties = FALSE) %>%
        pull(date)
      df %>%
        filter(date == d_max) %>%
        dplyr::select(nair, Tmax_z = Tmax_daily)
    },

    "median_date" = {
      d_med <- df %>%
        group_by(date) %>%
        summarise(m = max(Tmax_daily, na.rm = TRUE), .groups = "drop") %>%
        arrange(m) %>%
        slice(ceiling(n() / 2)) %>%
        pull(date)
      df %>%
        filter(date == d_med) %>%

```

```

    dplyr::select(nair, Tmax_z = Tmax_daily)
  }
)

if (!inherits(rel_h, "try-error") && !is.na(hmax_nc)) {
  df_out$height_m <- rel_h[df_out$nair] * hmax_nc
} else {
  n_layers <- max(df_out$nair, na.rm = TRUE)
  df_out$height_m <- seq(0, hmax_nc, length.out = n_layers)[df_out$nair]
}

df_out
}

#' Identifier le plot le plus proche de la médiane structurale de chaque cluster.
#'
#' Distance euclidienne normalisée sur (LAI, Hmax, fCover) centrées
#' par la médiane du cluster.
#'
#' @param df_sample cLHS avec colonnes (x, y, Cluster, LAI, Hmax, fCover).
#' @return Tibble 1 ligne x cluster : (Cluster, x, y, LAI, Hmax, fCover, plot_id).
select_cluster_median_plots <- function(df_sample) {
  if (!"Cluster" %in% names(df_sample)) {
    warning("[select_cluster_median_plots] colonne 'Cluster' absente de df_sample.")
    return(invisible(NULL))
  }
}

df_sample %>%
  filter(!is.na(Cluster)) %>%
  group_by(Cluster) %>%
  mutate(
    sd_LAI = sd(LAI, na.rm = TRUE),
    sd_Hmax = sd(Hmax, na.rm = TRUE),
    sd_fCover = sd(fCover, na.rm = TRUE),
    d_med = sqrt(
      ((LAI - median(LAI, na.rm = TRUE)) / pmax(sd_LAI, 1e-9))^2 +
      ((Hmax - median(Hmax, na.rm = TRUE)) / pmax(sd_Hmax, 1e-9))^2 +
      ((fCover - median(fCover, na.rm = TRUE)) / pmax(sd_fCover, 1e-9))^2
    )
  ) %>%
  slice_min(d_med, n = 1, with_ties = FALSE) %>%
  ungroup() %>%
  mutate(plot_id = sprintf("X%d_Y%d", round(x), round(y))) %>%
  dplyr::select(Cluster, x, y, LAI, Hmax, fCover, plot_id)
}

#' Visualiser le profil LAD réel + ligne Hmax pour un plot donné.
#'
#' @param plot_row Une ligne de tibble avec colonnes (x, y, Hmax, Cluster,
#' plot_id, fCover) et des colonnes LAD_Layer_*.
#' @param out_path Chemin de sortie PNG.
#' @param annotate Texte optionnel ajouté en sous-titre à la place du défaut.
#' @return Invisible ggplot object.
plot_lad_profile_for_plot <- function(plot_row, out_path, annotate = NULL) {
  lad_cols <- grep("^LAD_Layer_", names(plot_row), value = TRUE)
  if (length(lad_cols) == 0) {
    warning("[plot_lad_profile_for_plot] Aucune colonne LAD_Layer_ dans plot_row.")
    return(invisible(NULL))
  }
}

```

```

hmax <- as.numeric(plot_row$Hmax[1])
n <- length(lad_cols)
z <- seq(0, hmax, length.out = n)
lad <- as.numeric(plot_row[1, lad_cols])

df_lad <- tibble(z_m = z, LAD = lad)

p <- ggplot(df_lad, aes(x = LAD, y = z_m)) +
  geom_path(linewidth = 1.2, colour = "#31688e") +
  geom_point(size = 2, alpha = 0.7, colour = "#31688e") +
  geom_hline(yintercept = hmax, linetype = "dashed",
    colour = "firebrick", linewidth = 0.9) +
  annotate("text",
    x = max(lad, na.rm = TRUE) * 0.7,
    y = hmax,
    label = sprintf("Hmax = %.1f m", hmax),
    vjust = -0.5, colour = "firebrick",
    fontface = "bold", size = 4) +
  labs(
    title = sprintf("Profil LAD – Plot %s (Cluster %s)",
      plot_row$plot_id[1], plot_row$Cluster[1]),
    subtitle = if (!is.null(annotate)) annotate else
      sprintf("LAI = %.2f | Hmax = %.1f m | fCover = %.2f",
        plot_row$LAI[1], hmax, plot_row$fCover[1]),
    x = expression("LAD" ~ (m^2 ~ m^{-3})),
    y = "Hauteur (m)"
  ) +
  theme_bw(base_size = 12)

dir.create(dirname(out_path), recursive = TRUE, showWarnings = FALSE)
save_plot(p, out_path, width = 6, height = 7)
cat(sprintf(" [plot_lad_profile_for_plot] Sauvegardé : %s\n", out_path))
invisible(p)
}

#' Vérifier la robustesse du pic de Tmax proche du sol en Cluster 1
#' sur plusieurs plots (profils "canicule_date" Real vs Uniform LAD).
#'
#' @param df_sample Tibble cLHS (Cluster, x, y, LAI, Hmax, fCover).
#' @param real_lad_dir Dossier NetCDF H2_real_LAD.
#' @param uniform_lad_dir Dossier NetCDF H2_uniform_LAD.
#' @param date_seq Dates d'intérêt.
#' @param out_dir Dossier de sortie.
#' @param n_plots Nombre de plots C1 à comparer (default 3).
#' @return Invisible list(plot, data).
verify_cluster1_peak <- function(df_sample, real_lad_dir, uniform_lad_dir,
  date_seq, out_dir = "outputs/h2",
  n_plots = 3) {
  dir.create(out_dir, recursive = TRUE, showWarnings = FALSE)

  if (!"Cluster" %in% names(df_sample)) {
    warning("[verify_cluster1_peak] Colonne 'Cluster' absente – abandon.")
    return(invisible(NULL))
  }

  cl_all <- df_sample %>%
    filter(as.character(Cluster) == "1") %>%
    mutate(
      sd_LAI = sd(LAI, na.rm = TRUE),
      sd_Hmax = sd(Hmax, na.rm = TRUE),

```

```

sd_fCover = sd(fCover, na.rm = TRUE),
d_med      = sqrt(
  ((LAI      - median(LAI,      na.rm = TRUE)) / pmax(sd_LAI,      1e-9))^2 +
  ((Hmax     - median(Hmax,     na.rm = TRUE)) / pmax(sd_Hmax,     1e-9))^2 +
  ((fCover   - median(fCover,   na.rm = TRUE)) / pmax(sd_fCover,   1e-9))^2
)
) %>%
arrange(d_med)

if (nrow(c1_all) == 0) {
  warning("[verify_cluster1_peak] Aucun plot Cluster 1 trouvé – abandon.")
  return(invisible(NULL))
}

idx_sel <- c(2, 5, 10)
idx_sel <- idx_sel[idx_sel <= nrow(c1_all)]
if (length(idx_sel) == 0) idx_sel <- seq_len(min(n_plots, nrow(c1_all)))
c1_sel <- c1_all[idx_sel, ] %>%
  mutate(plot_id = sprintf("X%d_Y%d", round(x), round(y)))

find_nc <- function(dir, x, y) {
  pat <- sprintf("X%d_Y%d", round(x), round(y))
  hits <- list.files(dir, pattern = pat, full.names = TRUE)
  if (length(hits) == 0) NA_character_ else hits[1]
}

c1_meta <- c1_sel %>%
  rowwise() %>%
  mutate(
    nc_real    = find_nc(real_lad_dir,    x, y),
    nc_uniform = find_nc(uniform_lad_dir, x, y)
  ) %>%
  ungroup() %>%
  filter(!is.na(nc_real) & !is.na(nc_uniform))

if (nrow(c1_meta) == 0) {
  warning("[verify_cluster1_peak] Aucun NetCDF valide pour Cluster 1 – abandon.")
  return(invisible(NULL))
}

df_profiles <- purrr::pmap_dfr(
  c1_meta,
  function(Cluster, x, y, LAI, Hmax, fCover, plot_id, nc_real, nc_uniform, ...) {
    prof_real <- extract_vertical_tmax_profile(nc_real, date_seq, "canicule_date")
    prof_unif <- extract_vertical_tmax_profile(nc_uniform, date_seq, "canicule_date")
    bind_rows(
      if (!is.null(prof_real)) prof_real %>% mutate(scenario = "Real LAD"),
      if (!is.null(prof_unif)) prof_unif %>% mutate(scenario = "Uniform LAD")
    ) %>%
      mutate(Cluster = as.character(Cluster),
             plot_id = plot_id,
             LAI      = LAI,
             Hmax     = Hmax)
  }
)

if (nrow(df_profiles) == 0) {
  warning("[verify_cluster1_peak] Aucun profil extrait pour Cluster 1.")
  return(invisible(NULL))
}

```

```

df_profiles <- df_profiles %>%
  mutate(panel_label = sprintf("Plot %s\nHmax=%.1fm | LAI=%.1f",
                                plot_id, Hmax, LAI))

df_hmax_lines <- df_profiles %>%
  group_by(panel_label) %>%
  summarise(Hmax = first(Hmax), .groups = "drop")

p <- ggplot(df_profiles, aes(x = Tmax_z, y = height_m, colour = scenario)) +
  geom_path(linewidth = 1.2) +
  geom_point(size = 2, alpha = 0.7) +
  geom_hline(data = df_hmax_lines,
             aes(yintercept = Hmax),
             linetype = "dashed", colour = "firebrick",
             linewidth = 0.8, inherit.aes = FALSE) +
  facet_wrap(~ panel_label, nrow = 1, scales = "free_x") +
  scale_colour_manual(
    values = c("Real LAD" = "#31688e", "Uniform LAD" = "#d8576b")
  ) +
  labs(
    title = "Vérification pic sol – Cluster 1 (canicule_date)",
    subtitle = sprintf("%d plots Cluster 1 ordonnés par distance à la médiane",
                       nrow(cl_meta)),
    x = expression(T[max] ~ "simulée" ~ (degree*C)),
    y = "Hauteur (m)",
    colour = "Scénario"
  ) +
  theme_bw(base_size = 12) +
  theme(legend.position = "bottom",
        strip.text = element_text(face = "bold"))

png_path <- file.path(out_dir, "h2_verify_cluster1_peak.png")
csv_path <- file.path(out_dir, "h2_verify_cluster1_peak.csv")

save_plot(p, png_path, width = 14, height = 6)
write.csv(df_profiles, csv_path, row.names = FALSE)
cat(sprintf(" [verify_cluster1_peak] PNG : %s\n", png_path))
cat(sprintf(" [verify_cluster1_peak] CSV : %s\n", csv_path))

invisible(list(plot = p, data = df_profiles))
}

#' Produire la figure en 3 panneaux (un par cluster) : Tmax vs hauteur,
#' Real LAD vs Uniform LAD.
#'
#' @param df_sample      cLHS (besoin de Cluster, LAI, Hmax, fCover, x, y).
#' @param real_lad_dir   Dossier NetCDF du scénario H2_real_LAD.
#' @param uniform_lad_dir Dossier NetCDF du scénario H2_uniform_LAD.
#' @param date_seq      Dates d'intérêt (filtre été).
#' @param out_dir        Dossier de sortie PNG/CSV.
#' @param summary_mode   "mean_summer" | "canicule_date" | "median_date".
#' @return Invisiblement list(plot, data, medians).
plot_vertical_tmax_profiles <- function(df_sample, real_lad_dir,
                                       uniform_lad_dir, date_seq,
                                       out_dir = "outputs/h2",
                                       summary_mode = "mean_summer") {
  dir.create(out_dir, recursive = TRUE, showWarnings = FALSE)

  if (!"Cluster" %in% names(df_sample)) {
    warning("[plot_vertical_tmax_profiles] colonne 'Cluster' absente – abandon.")
  }

```

```

    return(invisible(NULL))
  }

medians <- select_cluster_median_plots(df_sample)
if (is.null(medians) || nrow(medians) == 0) {
  warning("[plot_vertical_tmax_profiles] Aucun plot médian sélectionné.")
  return(invisible(NULL))
}
cat(sprintf(" [vertical profile] %d plots médians :\n", nrow(medians)))
print(as.data.frame(medians[, c("Cluster", "plot_id", "LAI", "Hmax", "fCover")]))

find_nc <- function(dir, x, y) {
  pat <- sprintf("X%d_Y%d", round(x), round(y))
  hits <- list.files(dir, pattern = pat, full.names = TRUE)
  if (length(hits) == 0) NA_character_ else hits[1]
}

profiles_meta <- medians %>%
  rowwise() %>%
  mutate(
    nc_real = find_nc(real_lad_dir, x, y),
    nc_uniform = find_nc(uniform_lad_dir, x, y)
  ) %>%
  ungroup()

missing <- profiles_meta %>%
  filter(is.na(nc_real) | is.na(nc_uniform))
if (nrow(missing) > 0) {
  warning(sprintf(
    "[plot_vertical_tmax_profiles] NetCDF manquants pour %d plot(s) : %s",
    nrow(missing), paste(missing$plot_id, collapse = ", ")
  ))
  print(missing[, c("Cluster", "plot_id", "nc_real", "nc_uniform")])
  profiles_meta <- profiles_meta %>% filter(!is.na(nc_real) & !is.na(nc_uniform))
}
if (nrow(profiles_meta) == 0) {
  warning("[plot_vertical_tmax_profiles] Aucun NetCDF valide – abandon.")
  return(invisible(NULL))
}

df_profiles <- purrr::pmap_dfr(
  profiles_meta,
  function(Cluster, x, y, LAI, Hmax, fCover, plot_id, nc_real, nc_uniform, ...) {
    prof_real <- extract_vertical_tmax_profile(nc_real, date_seq, summary_mode)
    prof_unif <- extract_vertical_tmax_profile(nc_uniform, date_seq, summary_mode)
    bind_rows(
      if (!is.null(prof_real))
        prof_real %>% mutate(scenario = "Real LAD"),
      if (!is.null(prof_unif))
        prof_unif %>% mutate(scenario = "Uniform LAD")
    ) %>%
    mutate(Cluster = as.character(Cluster),
           plot_id = plot_id,
           LAI = LAI,
           Hmax = Hmax)
  }
)

if (nrow(df_profiles) == 0) {
  warning("[plot_vertical_tmax_profiles] Aucun profil extrait.")
}

```

```

    return(invisible(NULL))
  }

df_profiles <- df_profiles %>%
  mutate(panel_label = sprintf("Cluster %s\nHmax = %.1f m | LAI = %.1f",
                                Cluster, Hmax, LAI))

df_hmax_lines <- df_profiles %>%
  group_by(panel_label) %>%
  summarise(Hmax = first(Hmax), .groups = "drop")

p <- ggplot(df_profiles, aes(x = Tmax_z, y = height_m, colour = scenario)) +
  geom_path(linewidth = 1.2) +
  geom_point(size = 2, alpha = 0.7) +
  geom_hline(data = df_hmax_lines,
             aes(yintercept = Hmax),
             linetype = "dashed", colour = "firebrick",
             linewidth = 0.8, inherit.aes = FALSE) +
  facet_wrap(~ panel_label, nrow = 1, scales = "free_x") +
  scale_colour_manual(
    values = c("Real LAD" = "#31688e", "Uniform LAD" = "#d8576b")
  ) +
  labs(
    title = "Profils verticaux de Tmax – Real LAD vs Uniform LAD",
    subtitle = sprintf(
      "Plot médian de chaque cluster | Mode : %s", summary_mode
    ),
    x = expression(T[max] ~ "simulée" ~ (degree*C)),
    y = "Hauteur (m)",
    colour = "Scénario"
  ) +
  theme_bw(base_size = 12) +
  theme(legend.position = "bottom",
        strip.text = element_text(face = "bold"))

png_path <- file.path(out_dir,
  sprintf("h2_vertical_tmax_profiles_%s.png", summary_mode))
csv_path <- file.path(out_dir,
  sprintf("h2_vertical_tmax_profiles_%s.csv", summary_mode))

save_plot(p, png_path, width = 14, height = 6)
write.csv(df_profiles, csv_path, row.names = FALSE)
cat(sprintf(" [vertical profile] PNG : %s\n", png_path))
cat(sprintf(" [vertical profile] CSV : %s\n", csv_path))

if (requireNamespace("patchwork", quietly = TRUE) &&
    length(unique(df_profiles$scenario)) == 2) {
  df_diff <- df_profiles %>%
    tidyr::pivot_wider(
      id_cols = c(Cluster, nair, height_m, panel_label),
      names_from = scenario,
      values_from = Tmax_z
    ) %>%
    dplyr::filter(!is.na(`Real LAD`) & !is.na(`Uniform LAD`)) %>%
    dplyr::mutate(Diff = `Real LAD` - `Uniform LAD`)

  if (nrow(df_diff) > 0) {
    p_diff <- ggplot(df_diff, aes(x = Diff, y = height_m)) +
      geom_path(linewidth = 1.0, colour = "#5ec962") +
      geom_vline(xintercept = 0, linetype = "dotted",

```

```

        colour = "grey50", linewidth = 0.7) +
    geom_hline(data = df_hmax_lines,
              aes(yintercept = Hmax),
              linetype = "dashed", colour = "firebrick",
              linewidth = 0.8, inherit.aes = FALSE) +
    facet_wrap(~ panel_label, nrow = 1, scales = "free_x") +
    labs(
      x = expression(Delta * T[max] ~ "(Real - Uniform)" ~ (degree*C)),
      y = "Hauteur (m)"
    ) +
    theme_bw(base_size = 11) +
    theme(strip.text = element_blank())

p_combined <- p / p_diff + patchwork::plot_layout(heights = c(2, 1))
diff_path <- file.path(out_dir,
  sprintf("h2_vertical_tmax_profiles_%s_with_diff.png", summary_mode))
save_plot(p_combined, diff_path, width = 14, height = 8)
cat(sprintf(" [vertical profile] PNG avec diff : %s\n", diff_path))
}
}

invisible(list(plot = p, data = df_profiles, medians = medians))
}

#' Extraire le profil vertical de  $\Delta T_{max}$  selon deux méthodes de définition du  $T_{max}$ .
#'
#' Méthode 1 – indépendant :  $T_{max}(z) = \max_t(T_{air\_z}(t))$  par couche.
#' Méthode 2 – heure globale : profil à l'heure où la couche supérieure atteint son  $T_{max}$ .
#' Retourne la moyenne sur toutes les dates de date_seq.
#'
#' @param nc_file Chemin NetCDF.
#' @param df_macro Dataframe avec colonnes date et Tmax_macro (ERA5).
#' @param date_seq Dates de la période d'intérêt.
#' @return Tibble (height_m, nair, Delta_Tmax_mean, method) ou NULL.
extract_vertical_delta_both_methods <- function(nc_file, df_macro, date_seq) {
  if (is.na(nc_file) || !file.exists(nc_file)) return(NULL)

  nc <- try(nc_open(nc_file), silent = TRUE)
  if (inherits(nc, "try-error")) return(NULL)
  rel_h <- try(ncvar_get(nc, "relative_height"), silent = TRUE)
  h_top <- try(ncvar_get(nc, "veget_height_top"), silent = TRUE)
  raw <- try(get_variable(nc, "Tair_z"), silent = TRUE)
  nc_close(nc)
  if (inherits(raw, "try-error") || is.null(raw)) return(NULL)

  hmax_nc <- if (!inherits(h_top, "try-error")) h_top[1] else NA_real_

  df_hourly <- raw %>%
    mutate(Tair_C = Tair_z - 273.15,
           time = time - lubridate::hours(2),
           date = as.Date(time)) %>%
    filter(date %in% date_seq) %>%
    inner_join(df_macro %>% dplyr::select(date, Tmax_macro), by = "date")

  top_nair <- max(df_hourly$nair, na.rm = TRUE)

  m1 <- df_hourly %>%
    group_by(nair, date, Tmax_macro) %>%
    summarise(Tmax_z = max(Tair_C, na.rm = TRUE), .groups = "drop") %>%
    mutate(Delta = Tmax_z - Tmax_macro,

```

```

        method = "Indépendant par couche")

peak_times <- df_hourly %>%
  filter(nair == top_nair) %>%
  group_by(date) %>%
  slice_max(Tair_C, n = 1, with_ties = FALSE) %>%
  dplyr::select(date, peak_time = time)

m2 <- df_hourly %>%
  inner_join(peak_times, by = c("date", "time" = "peak_time")) %>%
  group_by(nair, date, Tmax_macro) %>%
  summarise(Tmax_z = first(Tair_C), .groups = "drop") %>%
  mutate(Delta = Tmax_z - Tmax_macro,
         method = "Heure du Tmax global")

df_out <- bind_rows(m1, m2) %>%
  group_by(nair, method) %>%
  summarise(Delta_Tmax_mean = mean(Delta, na.rm = TRUE), .groups = "drop")

if (!inherits(rel_h, "try-error") && !is.na(hmax_nc)) {
  df_out$height_m <- rel_h[df_out$nair] * hmax_nc
} else {
  n_lay <- max(df_out$nair)
  df_out$height_m <- seq(0, hmax_nc, length.out = n_lay)[df_out$nair]
}

df_out
}

#' Profils verticaux de  $\Delta T_{max}$  par cluster – comparaison des deux méthodes Tmax.
#'
#' @param df_sample cLHS avec Cluster.
#' @param real_lad_dir Dossier NetCDF du scénario full-real (H1f4_Full_real ou REF).
#' @param df_macro Dataframe ERA5 avec date et Tmax_macro.
#' @param date_seq Dates de la période.
#' @param out_path Chemin PNG de sortie.
#' @return Invisible list(plot, data).
plot_vertical_delta_by_cluster <- function(df_sample,
                                         real_lad_dir,
                                         df_macro,
                                         date_seq,
                                         out_path = "outputs/figures/06_h2_vertical_delta") {
  if (!"Cluster" %in% names(df_sample)) {
    warning("[plot_vertical_delta_by_cluster] colonne 'Cluster' absente – abandon.")
    return(invisible(NULL))
  }

  medians <- select_cluster_median_plots(df_sample)
  if (is.null(medians) || nrow(medians) == 0) return(invisible(NULL))

  find_nc <- function(dir, x, y) {
    pat <- sprintf("X%d_Y%d", round(x), round(y))
    hits <- list.files(dir, pattern = pat, full.names = TRUE)
    if (length(hits) == 0) NA_character_ else hits[1]
  }

  df_profiles <- medians %>%
    rowwise() %>%
    mutate(nc = find_nc(real_lad_dir, x, y)) %>%
    ungroup() %>%

```

```

filter(!is.na(nc)) %>%
purrr::pmap_dfr(function(Cluster, x, y, LAI, Hmax, plot_id, nc, ...) {
  prof <- extract_vertical_delta_both_methods(nc, df_macro, date_seq)
  if (is.null(prof)) return(NULL)
  prof %>% mutate(Cluster = as.character(Cluster),
                  plot_id = plot_id,
                  Hmax_plot = Hmax,
                  LAI_plot = LAI)
})

if (is.null(df_profiles) || nrow(df_profiles) == 0) {
  warning("[plot_vertical_delta_by_cluster] aucun profil extrait – abandon.")
  return(invisible(NULL))
}

df_profiles <- df_profiles %>%
  mutate(
    method = factor(method, levels = c("Indépendant par couche",
                                         "Heure du Tmax global")),
    panel_label = sprintf("Cluster %s\nHmax=%.1fm | LAI=%.1f",
                          Cluster, Hmax_plot, LAI_plot)
  )

df_hmax <- df_profiles %>%
  group_by(panel_label) %>%
  summarise(Hmax = first(Hmax_plot), .groups = "drop")

p <- ggplot(df_profiles,
            aes(x = Delta_Tmax_mean, y = height_m,
                colour = method, linetype = method)) +
  geom_path(linewidth = 1.1) +
  geom_point(size = 1.8) +
  geom_vline(xintercept = 0, linetype = "dotted",
             colour = "grey50", linewidth = 0.6) +
  geom_hline(data = df_hmax, aes(yintercept = Hmax),
             linetype = "dashed", colour = "firebrick",
             linewidth = 0.8, inherit.aes = FALSE) +
  scale_colour_manual(
    values = c("Indépendant par couche" = "#31688e",
               "Heure du Tmax global" = "#d8576b"),
    name = "Méthode Tmax"
  ) +
  scale_linetype_manual(
    values = c("Indépendant par couche" = "solid",
               "Heure du Tmax global" = "dashed"),
    name = "Méthode Tmax"
  ) +
  facet_wrap(~ panel_label, nrow = 1, scales = "free_x") +
  labs(
    title = "Profils verticaux de  $\Delta T_{max}$  par cluster – comparaison des deux méthodes",
    subtitle = "Bleu = Tmax indépendant par couche | Rouge = profil à l'heure du Tmax de l",
    x = expression(mean ~ Delta * T[max] ~ "moyen été" ~ (degree*C)),
    y = "Hauteur (m)"
  ) +
  theme_bw(base_size = 11) +
  theme(legend.position = "bottom",
        strip.text = element_text(face = "bold"),
        plot.title = element_text(face = "bold"))

dir.create(dirname(out_path), recursive = TRUE, showWarnings = FALSE)

```

```

save_plot(p, out_path, width = 14, height = 7)
csv_path <- sub("\\.png$", ".csv", out_path)
write.csv(df_profiles, csv_path, row.names = FALSE)
cat(sprintf(" [delta_cluster_profiles] PNG : %s\n", out_path))
invisible(list(plot = p, data = df_profiles))
}

#' Profils verticaux Tmax multi-scénarios – figure triptyque comparatif.
#'
#' @param scenarios_dict Liste nommée : list("Etiquette" = "chemin/NetCDF", ...).
#' @param df_sample cLHS (Cluster, x, y, LAI, Hmax, fCover).
#' @param date_seq Dates d'intérêt.
#' @param summary_mode "mean_summer" | "canicule_date" | "median_date".
#' @param out_path Chemin PNG de sortie.
#' @param palette Vecteur de couleurs (longueur = nb scénarios).
#' @return Invisible ggplot.
plot_vertical_tmax_profiles_multi <- function(scenarios_dict, df_sample,
                                             date_seq,
                                             summary_mode = "canicule_date",
                                             out_path = "outputs/figures/07_h2_vertica",
                                             palette = NULL,
                                             diff_from = NULL) {
  dir.create(dirname(out_path), recursive = TRUE, showWarnings = FALSE)

  medians <- select_cluster_median_plots(df_sample)
  if (is.null(medians) || nrow(medians) == 0) {
    warning("[plot_vertical_tmax_profiles_multi] Aucun plot médian – abandon.")
    return(invisible(NULL))
  }

  find_nc <- function(dir, x, y) {
    pat <- sprintf("X%d_Y%d", round(x), round(y))
    hits <- list.files(dir, pattern = pat, full.names = TRUE)
    if (length(hits) == 0) NA_character_ else hits[1]
  }

  df_all <- purrr::imap_dfr(scenarios_dict, function(nc_dir, sc_label) {
    purrr::pmap_dfr(medians, function(Cluster, x, y, LAI, Hmax, fCover, plot_id, ...) {
      nc_file <- find_nc(nc_dir, x, y)
      prof <- extract_vertical_tmax_profile(nc_file, date_seq, summary_mode)
      if (is.null(prof)) return(NULL)
      prof %>% mutate(scenario = sc_label,
                     Cluster = as.character(Cluster),
                     plot_id = plot_id,
                     LAI = LAI,
                     Hmax = Hmax)
    })
  })

  if (nrow(df_all) == 0) {
    warning("[plot_vertical_tmax_profiles_multi] Aucun profil extrait.")
    return(invisible(NULL))
  }

  df_all <- df_all %>%
    mutate(panel_label = sprintf("Cluster %s\nHmax=%.1fm | LAI=%.1f",
                                  Cluster, Hmax, LAI))

  df_hmax <- df_all %>%
    group_by(panel_label) %>%

```

```

    summarise(Hmax = first(Hmax), .groups = "drop")

sc_names <- names(scenarios_dict)
if (is.null(palette)) {
  palette <- setNames(
    c("#31688e", "#d8576b", "#5ec962", "#fca50a", "#440154")[seq_along(sc_names)],
    sc_names
  )
}

annot_layer <- NULL

# Differential mode: show T(scenario) - T(diff_from) instead of absolute T.
# Makes near-zero effects visible when 3 lines would otherwise overlap.
if (!is.null(diff_from) && diff_from %in% sc_names) {
  df_base <- df_all %>%
    filter(scenario == diff_from) %>%
    dplyr::select(panel_label, nair, Tmax_base = Tmax_z)
  df_all <- df_all %>%
    inner_join(df_base, by = c("panel_label", "nair")) %>%
    mutate(Tmax_z = Tmax_z - Tmax_base) %>%
    filter(scenario != diff_from)
  sc_names <- setdiff(sc_names, diff_from)
  palette <- palette[sc_names]
  x_lab <- bquote(Delta * T[max] ~ "vs" ~ .(diff_from) ~ (degree * C))
  vline <- geom_vline(xintercept = 0, linetype = "dashed",
    colour = "grey30", linewidth = 0.6)
  hline_obj <- geom_hline(data = df_hmax,
    aes(yintercept = Hmax),
    linetype = "dashed", colour = "firebrick",
    linewidth = 0.8, inherit.aes = FALSE)
  title_str <- sprintf("Profils verticaux ΔT vs %s – %d scénarios", diff_from, length(sc_n

  df_annot <- df_all %>%
    group_by(panel_label) %>%
    summarise(max_abs = max(abs(Tmax_z), na.rm = TRUE),
      y_pos = max(height_m, na.rm = TRUE) * 0.7,
      .groups = "drop") %>%
    mutate(label = sprintf("|ΔT|max = %.4f°C", max_abs))
  annot_layer <- geom_label(
    data = df_annot,
    aes(y = y_pos, label = label),
    x = -Inf,
    inherit.aes = FALSE,
    hjust = -0.05, vjust = 0.5,
    size = 3.2, fill = "lightyellow", colour = "grey20",
    label.size = 0.4, fontface = "bold"
  )
} else {
  x_lab <- expression(T[max] ~ simulee ~ (degree * C))
  vline <- NULL
  hline_obj <- geom_hline(data = df_hmax,
    aes(yintercept = Hmax),
    linetype = "dashed", colour = "firebrick",
    linewidth = 0.8, inherit.aes = FALSE)
  title_str <- sprintf("Profils verticaux Tmax – comparaison %d scénarios",
    length(sc_names))
}

p <- ggplot(df_all, aes(x = Tmax_z, y = height_m,

```

```

                                colour = scenario, linetype = scenario)) +
geom_path(linewidth = 1.4) +
geom_point(size = 2, alpha = 0.8) +
vline +
hline_obj +
annot_layer +
facet_wrap(~ panel_label, nrow = 1, scales = "free_x") +
scale_colour_manual(values = palette) +
scale_linetype_manual(
  values = setNames(
    c("solid", "dashed", "dotted", "dotdash", "longdash")[seq_along(sc_names)],
    sc_names
  )
) +
labs(
  title    = title_str,
  subtitle = sprintf("Plot médian par cluster | Mode : %s", summary_mode),
  x        = x_lab,
  y        = "Hauteur (m)",
  colour   = "Scénario", linetype = "Scénario"
) +
theme_bw(base_size = 13) +
theme(legend.position = "bottom",
      strip.text       = element_text(face = "bold", size = 12))

save_plot(p, out_path, width = 16, height = 8)
cat(sprintf(" [3-way profile] PNG : %s\n", out_path))
invisible(p)
}

```

4.14 R/sentinel.R — Annexe Sentinel-2/FORMS-H

```

# =====
# Chapter 1 – Sentinel-2 + FORMS-H annex (H3)
# Chapter 3 – Sentinel-2 LAI time series extraction and smoothing
# =====

#' Build the S2 + FORMS-H naive scenario by extracting optical proxy inputs.
#'
#' Replaces LiDAR LAI with Sentinel-2 LAI and LiDAR Hmax with FORMS-H canopy
#' height. Uses a uniform LAD profile. Missing values are imputed by the
#' sample mean.
#'
#' @param df_sample cLHS sample dataframe with x, y columns.
#' @param in_dir    Input directory containing the S2 and FORMS-H rasters.
#' @param agg_factor Aggregation factor (default 2).
#' @return List with elements $scenario (scenario list) and $df_sample (augmented).
make_s2_formsh_scenario <- function(df_sample, in_dir, agg_factor = 2) {
  r_s2      <- rast(file.path(in_dir, "s2lai_summer_atbd_res_10_m.tif"))
  r_formsh  <- rast(file.path(in_dir, "FORMS-H_Blois.tif"))
  r_s2_20   <- terra::aggregate(r_s2, fact = agg_factor, fun = "mean", na.rm = TRUE)
  r_fh_20   <- resample(r_formsh, r_s2_20, method = "bilinear")
  pts <- vect(as.matrix(df_sample[, c("x", "y")]), crs = crs(r_s2_20))
  df_sample$S2_LAI <- as.numeric(terra::extract(r_s2_20, pts)[, 2])
  df_sample$FORMS_H <- as.numeric(terra::extract(r_fh_20, pts)[, 2])
  na_count <- sum(is.na(df_sample$S2_LAI) | is.na(df_sample$FORMS_H))
  if (na_count > 0) {
    df_sample$S2_LAI[is.na(df_sample$S2_LAI)] <- mean(df_sample$S2_LAI, na.rm = TRUE)
  }
}

```

```

df_sample$FORMS_H[is.na(df_sample$FORMS_H)] <- mean(df_sample$FORMS_H, na.rm = TRUE)
}
scenario <- list(
  name       = "ANNEX_S2_FORMSH",
  lai_fn     = function(plot_row) as.numeric(plot_row$S2_LAI),
  hmax_fn    = function(plot_row) as.numeric(plot_row$FORMS_H),
  fcover_fn  = function(plot_row) 1.0,
  lad_fn     = make_lad_uniform
)
list(scenario = scenario, df_sample = df_sample)
}

# =====
# Chapter 3 – S2 LAI time series extraction and smoothing
#
# DATA REQUIREMENT – multi-date S2 LAI stack:
#   Expected at: in_files/s2lai_ts_blois.tif
#   Format:      SpatRaster, one layer per S2 acquisition date.
#                 Layer names must be parseable as ISO dates ("2021-03-15" etc.)
#                 or Julian day strings ("2021-074").
#   Units:       LAI in m²/m², same scale as s2lai_summer_atbd_res_10_m.tif.
#   Resolution:  10 m UTM 31N, same extent as the single-date file.
#   How to produce:
#     → Run NC_Full/02_CODES/Sentinel_2/4_create_s2_metric_time_series.R
#     → Apply Whittaker smoother (see NC_Full/02_CODES/Sentinel_2/5_smooth_s2_ts.R)
#     → Export to in_files/ as s2lai_ts_blois.tif
#   Until available, load_s2_ts() will stop with an informative error.
# =====

#' Load a multi-date Sentinel-2 LAI raster stack from disk.
#'
#' @title Load S2 LAI time series stack
#' @description Reads the multi-layer SpatRaster expected at `s2lai_ts_blois.tif`
#'   inside `in_dir`. Layer names must be ISO dates or "YYYY-DOY" strings.
#'   Returns the raster with names coerced to `Date` objects stored as attributes.
#'
#' @param in_dir Directory containing `s2lai_ts_blois.tif` (default: `in_files/`).
#' @param filename Filename of the stack (default: `s2lai_ts_blois.tif`).
#' @return SpatRaster with layer names as ISO date strings and attribute `dates`
#'   containing the corresponding `Date` vector.
load_s2_ts <- function(in_dir, filename = "s2lai_ts_blois.tif") {
  path <- file.path(in_dir, filename)
  if (!file.exists(path)) {
    stop(sprintf(
      paste0(
        "S2 LAI time series stack not found: %s\n",
        " Produce it from NC_Full/02_CODES/Sentinel_2/ and copy to in_files/.\n",
        " See the DATA REQUIREMENT comment in R/sentinel.R for the expected format."
      ),
      path
    ))
  }
  r <- terra::rast(path)
  raw_names <- names(r)
  # Try ISO format first ("2021-03-15"), then YYYY-DOY ("2021-074")
  dates <- suppressWarnings(as.Date(raw_names))
  if (any(is.na(dates))) {
    dates <- suppressWarnings(as.Date(raw_names, format = "%Y-%j"))
  }
  if (any(is.na(dates))) {

```

```

    stop(sprintf(
      "Cannot parse S2 stack layer names as dates (tried ISO and YYYY-D0Y). Got: %s ...",
      paste(head(raw_names, 3), collapse = ", ")
    ))
  }
  names(r) <- as.character(dates)
  attr(r, "dates") <- dates
  r
}

#' Extract Sentinel-2 LAI time series for all sample plots.
#'
#' @title Extract per-plot S2 LAI time series
#' @description For each plot in `df_sample`, extracts the LAI value at each
#' acquisition date from the multi-layer raster stack. Returns a long-format
#' data.frame suitable for passing to `smooth_s2_ts()`. Cloud-masked pixels
#' (NA) are retained so the smoother can handle gaps.
#'
#' @param df_sample cLHS sample dataframe with `x` and `y` columns (UTM 31N).
#' @param s2_stack SpatRaster from `load_s2_ts()` with ISO-date layer names.
#' @return Long data.frame with columns `plot_id, date, doy, lai`.
#' `plot_id` is `"X{round(x)}_Y{round(y)}"`, consistent with `plot_id_from_row()`.
#' @seealso [load_s2_ts()], [smooth_s2_ts()], [plot_id_from_row()]
extract_s2_ts_per_plot <- function(df_sample, s2_stack) {
  dates <- as.Date(names(s2_stack))
  pts <- terra::vect(as.matrix(df_sample[, c("x", "y")]),
    crs = terra::crs(s2_stack))
  vals <- as.data.frame(terra::extract(s2_stack, pts))
  vals <- vals[, -1, drop = FALSE] # drop terra's ID column
  colnames(vals) <- as.character(dates)

  plot_ids <- sprintf("X%d_Y%d",
    round(as.numeric(df_sample$x)),
    round(as.numeric(df_sample$y)))

  long <- do.call(rbind, lapply(seq_along(dates), function(i) {
    data.frame(
      plot_id = plot_ids,
      date = dates[i],
      doy = as.integer(format(dates[i], "%j")),
      lai = vals[[i]],
      stringsAsFactors = FALSE
    )
  })))
  long[order(long$plot_id, long$date), ]
}

#' Smooth S2 LAI time series to daily resolution using a cyclic GAM spline.
#'
#' @title Smooth per-plot S2 LAI time series
#' @description Fits a GAM with a cyclic cubic regression spline (period 365 days)
#' per plot, interpolating sparse S2 acquisitions to a complete 1–365 DOY grid.
#' NA observations (cloud contamination) are dropped before fitting.
#' Plots with fewer than `min_obs` valid observations receive a warning and are
#' excluded from the output.
#'
#' @param df_ts Long data.frame from `extract_s2_ts_per_plot()` with columns
#' `plot_id, doy, lai`. NAs allowed.
#' @param k Number of basis functions for the cyclic spline (default 10).
#' Reduce if fewer acquisitions are available.

```

```

#' @param min_obs Minimum number of non-NA observations required to fit (default 4).
#' @return Named list: `plot_id → data.frame(doy, lai)` with 365 rows each.
#' Plots that fail to fit are omitted (warning issued).
#' @seealso [extract_s2_ts_per_plot()], [make_phenology_from_s2()]
smooth_s2_ts <- function(df_ts, k = 10, min_obs = 4) {
  ids <- unique(df_ts$plot_id)
  out <- lapply(ids, function(pid) {
    sub <- df_ts[df_ts$plot_id == pid & !is.na(df_ts$lai), ]
    if (nrow(sub) < min_obs) {
      warning(sprintf(
        "Plot '%s': only %d valid S2 obs (need >= %d) – excluded.",
        pid, nrow(sub), min_obs
      ))
      return(NULL)
    }
    fit <- tryCatch(
      mgcv::gam(lai ~ s(doy, bs = "cc", k = min(k, nrow(sub) - 1L)),
        data = sub, knots = list(doy = c(1, 365))),
      error = function(e) {
        warning(sprintf("Plot '%s': GAM failed (%s) – excluded.", pid, e$message))
        NULL
      }
    )
    if (is.null(fit)) return(NULL)
    data.frame(doy = 1:365,
      lai = pmax(as.numeric(predict(fit, data.frame(doy = 1:365))), 0))
  })
  names(out) <- ids
  Filter(Negate(is.null), out)
}

```